

填空题

ハードウェア、ディスク、パーティション、ファイルシステム

CPUに関する情報が格納されたファイルは？

/proc/cpuinfo

起動中のプロセスやカーネルが認識しているハードウェアの情報が格納されている疑似的なファイルシステムであるディレクトリは？

/proc/

ハードディスク、USBメモリーなどが属しているUSBのデバイスクラスは？（先頭4文字のみとする）

Mass

IRQに関する情報が格納されているファイルは？

/proc/interrupts

指定したカーネルモジュールだけをロードするコマンドは？

insmod

I/Oポートアドレスの情報が格納されているファイルは？

/proc/ioports

スピーカー、マイクなどが属しているUSBのデバイスクラスは？（先頭5文字のみとする）

Audio

メモリの使用状況が格納されたファイルは？

/proc/meminfo

ホットプラグデバイスを接続した際に、デバイスファイルを動的に作成する仕組みは？

udev

SCSIデバイスに関する情報が格納されたファイルは？

/proc/scsi/scsi

ロードされているカーネルモジュールの情報を表示するコマンドは？また、ロードされているカーネルモジュールの情報が格納されたファイルは？

lsmod

/proc/modules

知识点总结

- `/proc/` 目录：Linux 内核的“实时监控室”，所有的系统信息（CPU、内存、I/O）都在这里以文件形式呈现。
 - **USB 类定义**：按功能分类，存储叫 **Mass**，音频叫 **Audio**。
 - **模块管理**：`insmod` 是手动强行插入模块，`lsmod` 是列出模块。
 - **设备动态管理**：`udev` 负责在你插拔设备（ホットプラグ）时自动生成对应的设备文件。
-

选项解析与详细说明

1. `/proc/` 目录及其下的文件

- `/proc/`：这是一个 疑似（ぎじ）ファイルシステム（伪文件系统）。它不占硬盘空间，只存在于内存中。
- `/proc/interrupts`：存储 IRQ（中断请求）信息。
- `/proc/ioports`：存储 I/O ポート（端口）地址。
- `/proc/meminfo`：存储 メモリ（内存）使用情况。
- `/proc/scsi/scsi`：存储 SCSI デバイス 列表信息。
- `/proc/modules`：存储当前 ロード（加载）的内核模块列表（和 `lsmod` 命令看到的内容一样）。

1. USB 设备类

- **Mass (Storage)**: 对应硬盘、U盘。
- **Audio**: 对应麦克风、喇叭。

3. 命令与机制

- `insmod`：只能 ロード（加载）指定模块，不考虑依赖关系。
 - `lsmod`：リスト（列表）显示已加载的模块。
 - `udev`：关键词是 動的（どうてき / 动态）和 作成。
-

快速记忆排查表（秒杀考点）

关键词 (日语)	对应答案 (填空)	记忆钩子
疑似ファイルシステム / ディレクトリ	<code>/proc/</code>	系统的“过程 (Process)”目录
ハードディスク / USBメモリ (4字)	Mass	Massive (大量存储)
スピーカー / マイク (5字)	Audio	声音就是 Audio
IRQ	<code>/proc/interrupts</code>	Interrupt = 中断 = IRQ
I/Oポート	<code>/proc/ioports</code>	原名直译
ホットプラグ / 動的に作成	udev	user device (用户设备管理)
モジュール情報表示 (命令)	lsmod	ls (list) + mod (module)

小学生级记忆法

- 路径类：**只要题目问“信息放在哪个**文件**或**目录**”，答案开头 90% 是 `/proc/`，后面接题目里的英文。
 - 问 IRQ 就填 `interrupts`，问内存就填 `meminfo`。
- USB 类：**
 - 存数据的（硬盘/U盘）很大 → **Mass**。
 - 出声音的（耳机/麦克风）→ **Audio**。
- udev：**看到「**ホットプラグ (Hot-plug)**」（热插拔），就想成是“有个叫 **u** (你) 的 **dev** (设备) 插进来了”。

考试小秘籍

- 注意字数限制：**题目里明确要求「**先頭4文字**」（Mass）或「**先頭5文字**」（Audio），多写一个字母都不得分！
- 路径完整性：**`/proc/` 最后的斜杠在目录题里最好带上；`/proc/scsi/scsi` 这种重复的路径要死记硬背一下，它是 SCSI 里的 SCSI 文件。
- lsmod vs /proc/modules：**如果题目问 **コマンド（命令）** 填 `lsmod`；问 **ファイル（文件）** 填 `/proc/modules`。

依存関係を考慮して「kmod」というカーネルモジュールをアンロードするコマンドは？（オプションは短縮形）

`modprobe -r kmod`

指定したカーネルモジュールだけをアンロードするコマンドは？

`rmmod`

キーボード、マウスなどが属しているUSBのデバイスクラスは？（略称とする）

HID

依存関係を考慮してカーネルモジュールをロードまたはアンロードするコマンドは？

modprobe

接続されたUSBデバイスの情報を表示するコマンドは？また、接続されたUSBデバイスの情報が格納されたファイルは？

lsusb

/proc/bus/usb/devices

モデム、TA(ターミナルアダプタ)などが属しているUSBのデバイスクラスは？（先頭3文字のみとする）

ACM

PCIデバイスの情報を表示するコマンドは？ また、PCIデバイスの情報が格納されたファイルは？

lspci

/proc/bus/pci/devices

デバイスが使用中のDMAチャネルに関する情報が格納されたファイルは？

/proc/dma

MBR方式のハードディスク「/dev/sda」デバイスのパーティションテーブルを確認したい。適切なコマンドは？

fdisk -l /dev/sda

GPT形式のハードディスクにおいて、パーティションの作成、削除、変更および情報表示を行うことができるコマンドは？

gdisk

fdiskコマンドのサブコマンドで、パーティションタイプの一覧を表示するものは？

l

MBR形式のハードディスクである「/dev/sda」デバイスのパーティションを操作したい。適切なコマンドは？

fdisk /dev/sda

知识点总结

- 模块管理的高级与基础：`modprobe` 聪明（管依赖），`rmmod/insmod` 简单（不管依赖）。
 - 总线查询命令：`lsusb` (USB) 和 `lspci` (PCI) 是对应的，“ls + 总线名”。
 - 分区工具的派系：`fdisk` 属于 MBR 老派系，`gdisk` 属于 GPT 新派系。
-

选项解析与详细说明

4. 内核模块卸载的细节

- `modprobe -r` : 这里的 `-r` 代表 **Remove**。因为它考虑「依存關係」，所以它是卸载时的首选高级工具。
- `rmmod` : 缩写自 **Remove Module**。它很死板，只卸载你指定的那个，不管别的。

2. USB 与 PCI 的“文件柜”

- 命令: `lsusb` (显示USB设备), `lspci` (显示PCI设备)。
- 文件: 它们的信息都在 `/proc/bus/` 下。
 - USB 在 `/proc/bus/usb/devices`。
 - PCI 在 `/proc/bus/pci/devices`。

3. MBR 与 GPT 的分区博弈

- `fdisk` : 针对 **MBR** 方式。查看列表用 `-l` (list)。
- `gdisk` : 针对 **GPT** 方式 (可以记成 **GPT + fdisk = gdisk**)。
- `fdisk` 内部指令:
 - `l` : 这里的 `l` 代表 **list**, 用来列出所有的分区类型 ID (比如 83 是 Linux, 82 是 Swap)。

快速记忆排查表

核心关键词	对应工具/路径	记忆点
依存關係 + アンロード	<code>modprobe -r</code>	聪明地移走 (Remove)
指定したのみ + アンロード	<code>rmmod</code>	直接移走 (rm)
USB クラス (键盘/鼠标)	HID	H uman I nterface D evice
USB クラス (调制解调器)	ACM	A bstract C ontrol M odel
PCI 情報表示 (命令/文件)	<code>lspci</code> / <code>/proc/bus/pci/devices</code>	路径里有 bus (总线)
MBR 分区操作	<code>fdisk</code>	F ixed D isk (老牌工具)
GPT 分区操作	<code>gdisk</code>	G 代表 GPT

小学生级记忆法

1. USB 设备类名:

- 键盘鼠标是给人用的 → **HID** (Human) 。
- 拨号上网 (Modem) 需要 **A** (阿) **C** (才) **M** (们) → **ACM**。

2. 分区命令：

- 看到 **MBR** 就选 **fdisk**。
- 看到 **GPT** 就选 **gdisk**。
- 想看分区列表详情，命令后面加个 **-l** (Look 一下) 。

3. 子命令 **l**：

- 在 **fdisk** 里面，如果你不知道分区类型的代号，按 **l** (list) 就能看到一长串列表。



考试小秘籍

- 参数大小写：**fdisk -l** 是小写的 **l**，不是数字 **1**。
- 设备路径：在操作分区时，必须指定设备名，如 **/dev/sda**。「**/dev/**」绝对不能漏。
- **ACM** 的坑：题目问“先头 3 个字母”，记准是 **ACM**。
- **DMA** 路径：DMA 的信息直接在 **/proc/dma**，不在 **/proc/bus/** 下，别记混了。

システムの起動には必須で無いプログラムやライブラリが格納されたディレクトリは？

/usr/

論理ボリュームを作成するまでのコマンドは？（使用順に3つ）

pvcreate vgcreate lvcreate

以下の特徴を持つソフトウェアの名称は？（英字で回答）

- GUIでの操作が可能
- GPT形式のHDDからの起動をサポート
- BIOSの後継

UEFI

MBR形式、GPT形式のパーティションテーブルをサポートするパーティション操作コマンドは？

parted

追加のパッケージが格納され、ルートパーティションとは別のパーティションを割り当てた方がよいディレクトリは？

/opt/

一般ユーザ用のコマンドが格納されていて、ルートパーティションから分割できないディレクトリは？

/bin/

fdiskコマンドのサブコマンドで、パーティションを作成するものは？

n

fdiskコマンドのサブコマンドで、パーティションを削除するものは？

d

一時ファイルが格納されていて、再起動すると中身が削除され、システムによってはパーティションを分割する事が推奨されるディレクトリは？

tmp

物理的な記憶デバイスの領域をまとめ、それを論理的なひとつの大きな記憶デバイスとして扱うことで、物理的なパーティションの仕様に由来する制限を回避できるシステムの名称は？(略称で)

LVM

3番目のSCSIハードディスクの2番目のパーティションを表すデバイスファイルは？

/dev/sdc2

システム管理者用のコマンドが格納されていて、ルートパーティションから分割できないディレクトリは？

/sbin/

fdiskコマンドのサブコマンドで、パーティションテーブルの変更を保存して終了するものは？

w

💡 知识点总结

- **FHS (Filesystem Hierarchy Standard)**: Linux 目录的排布规则，区分了哪些是启动必需的（如 `/bin`），哪些是可选或附加的（如 `/usr`，`/opt`）。
- **LVM (Logical Volume Manager)**: 打破物理分区限制的弹性存储技术。
- **UEFI**: 新一代的硬件接口，取代了古老的 BIOS。
- **分区操作指令**: `fdisk` 内部单字母指令。

🔍 选项解析与详细说明

1. 目录结构辨析

- `/bin/` & `/sbin/`: 系统启动和单用户模式下**必需**的命令。`bin` 给一般用户，`sbin` 给管理员 (superuser)。
- `/usr/`: **U**ser **S**ystem **R**esources。存放不影响启动的次要程序和库。
- `/opt/`: **O**ptional。存放第三方、额外的软件包。

- `/tmp/` : Temporary。存放临时文件，重启通常会清空。

2. LVM 的三级火箭

- PV (Physical Volume)→VG (Volume Group)→LV (Logical Volume)
- 必须按此顺序创建：先格式化物理硬盘（pv），再捆绑成池（vg），最后切出逻辑分区（lv）。

3. 设备文件命名规则

- `/dev/sd` + [硬盘序号] + [分区号]
- 第一块硬盘是 `a`，第二块是 `b`，第三块是 `c`。
- 所以：3番目 (c) + 2番目分区 (2) = `/dev/sdc2`。

快速记忆排查表

核心关键词	对应路径/命令	记忆点
起動に必須でない	<code>/usr/</code>	User 用的，非内核必需
追加パッケージ	<code>/opt/</code>	Optional (可选)
管理者用コマンド	<code>/sbin/</code>	S 代表 Superuser
BIOSの後継 / GPT対応	UEFI	现代电脑的启动方式
MBR/GPT 両方サポート	<code>parted</code>	功能最全的分区工具
3つ順番 (LVM)	PV (Physical Volume)→VG (Volume Group)→LV (Logical Volume)	Please Visit Very Good LuV (首字母顺序)

小学生级记忆法

1. `fdisk` 子命令“三字经”：
 - **n** (New): 新建。
 - **d** (Delete): 删除。
 - **w** (Write): 写入并保存退出 (Write and quit)。
2. LVM 顺序：
 - 想象盖房子：先买**皮**（PV - 物理）、再组**个**（VG - 卷组）、最后分**间**（LV - 逻辑）。
3. 目录分割原则：
 - **不能分**：`/bin`，`/sbin`，`/etc`，`/lib`，`/dev`（这些是命根子）。
 - **推荐分**：`/home`，`/var`，`/tmp`，`/boot`，`/usr`，`/opt`（这些是身外之物）。



考试小秘籍

- **LVM 命令拼写**：全部都是 `xxxcreate`，千万别漏掉 `create`。
- **fdisk 退出**：想保存用 `w`，想后悔（不保存）退出用 `q` (Quit)。
- **UEFI vs BIOS**：看到 **GPT** 和 **GUI** 两个词同时出现，基本就是考 **UEFI**。
- **sdsc2 陷阱**：考试时看清是 SCSI (sd) 还是 IDE (hd)。现在基本都是 sd 开头。

MBR形式のパーティションテーブルを使用するHDDのパーティションの作成、削除、変更および情報表示を行うことが出来るコマンドは？

fdisk

基本パーティションはMBR方式のハードディスクに最大何個作成することが出来るか？（数字のみ回答）

4

アプリケーションのログファイルや一時的な電子メールファイルなど、可変ファイル群を格納するディレクトリは？

/var/

デバイスファイルが格納されていて、ルートパーティションから分割できないディレクトリは？

/dev/

ユーザ数百人が利用すると想定されるファイルサーバを構築したい。ルートパーティションとは別のパーティションとして構成した方がよいと考えられるディレクトリは？

/home/

512MBのメモリを搭載したシステムで、適切なスワップ領域のサイズは？（数字で）

1GB

共有ライブラリが格納されていて、ルートパーティションから分割できないディレクトリは？

/lib/

fdiskコマンドのサブコマンドで、パーティションタイプ（システムID）を変更するものは？

t

システム固有の設定ファイルが格納されていて、ルートパーティションから分割できないディレクトリは？

/etc/

以下の特徴を持つ領域の名前は？（アルファベットの略称で）

- UEFIシステムでファームウェアが読み込まれた後、最初にアクセスされる
- FATでフォーマットされる

- ブートローダや起動に必要なドライバなどが置かれている

ESP

Linuxカーネルなど、システムを起動するために必須のファイルが格納されていて、システムによってはパーティションを分割する事が推奨されるディレクトリは？

/boot/

物理メモリが1GBの場合、スワップ領域を物理メモリの何倍程度のサイズにする事が推奨されるか。（数字1文字で）

2倍

知识点总结

- **MBR 的局限性**：这种古老的分区格式最多只能有 **4个** 基本分区（主分区）。
- **可变数据存储**：`/var` 专门存放大小会动态变化的文件（日志、邮件）。
- **不可分割的目录**：`/etc`、`/bin`、`/sbin`、`/lib`、`/dev` 必须和根目录（`/`）在一起，否则系统无法引导。
- **Swap 规划**：内存越小，Swap 倍数越大；一般建议为内存的 **2倍**。

选项解析与详细说明

4. 目录分区建议（为什么有的要分开？）

- `/home/`：存放用户数据。如果用户多了，数据会爆满，分开分区可以防止用户数据填满根目录导致系统崩溃。
- `/var/`：存放日志（Log）。由于日志一直写入，最容易占满空间，必须监控或独立分区。
- `/boot/`：存放内核。为了引导安全，通常建议分一个几百 MB 的独立小分区。

2. Swap（スワップ）计算原则

- 题目 1：**512MB 内存** → 属于小内存，按 **2倍** 计算 → **1GB**。
- 题目 2：**1GB 内存** → 推荐倍数是 **2倍**。

3. 特殊区域：ESP

- **ESP (EFI System Partition)**：它是 UEFI 启动的“桥头堡”。
- **特点**：必须是 **FAT** 格式，存放着引导加载程序（Bootloader）。

快速记忆排查表

关键词 (日本語)	对应路径/数字	记忆点
可変ファイル (ログ/メール)	/var/	Variable (可变的)
設定ファイル	/etc/	Etc = 杂项/配置
共有ライブラリ	/lib/	Library (库)
基本パーティション最大	4	MBR 的极限就是 4
UEFI / FAT / 最初にアクセス	ESP	EFI System Partition
スワップサイズ (1GB内存)	2倍	经典“两倍”原则

🧠 小学生级记忆法

- 1. 不可分割的“五虎将”：
 - /etc (设置), /bin (常用命令), /sbin (管理命令), /lib (库文件), /dev (设备文件)。
 - 口诀：“机密 (etc) 命令 (bin/sbin) 库 (lib) 设备 (dev) ，根区 (/) 永远不作对。”（意思是它们必须留在根分区）。
- 2. fdisk 修改类型：
 - 想要改变 Type（类型），就按 t。
- 3. MBR 分区数：
 - 想象一张桌子只有 4 条腿，断了一条（用了4个主分区）就不能再加了。

📝 考试小秘籍

- /var vs /tmp：虽然都存临时数据，但题目提到「ログ（日志）」或「メール（邮件）」时，百分之百是 /var。
- MBR 的 4 个分区：如果需要超过 4 个分区，必须把其中一个主分区改为“扩展分区”，然后在里面建逻辑分区。但题目问“最大基本分区数”，答案永远是 4。
- ESP 的格式：考试有时会问 ESP 必须是什么格式，记住是 FAT（通常是 FAT32）。

1番目のSCSIハードディスクの、最初の論理パーティションを表すデバイスファイルは？

/dev/sda5

fdiskコマンドのサブコマンドで、パーティションテーブルを表示するものは？

p

fdiskコマンドのサブコマンドで、パーティションテーブルの変更を保存せずに終了するのは？

q

fdiskコマンドのサブコマンドで、サブコマンドのメニューを表示するものは？

m

mke2fsコマンドでext3ファイルシステムを「/dev/sda2」に作成したい。適切なコマンドは？（オプションは引数を取らないものを指定すること）

mke2fs -j /dev/sda2

スワップ領域を作成するコマンドは？

mkswap

ブロックデバイスをツリー状に一覧表示するコマンドは？

lsblk

mkfsコマンドでext2ファイルシステムを「/dev/hda1」に作成したい。適切なコマンドは？（オプションは明示的に指定すること）

mkfs -t ext2 /dev/hda1

メモリ上に作成できる一時的なファイルシステムは？

tmpfs

シリコングラフィクス社が開発した動的inodeの機能を持つファイルシステムは？

XFS

mke2fsコマンドでext2ファイルシステムを「/dev/hda1」に作成したい。適切なコマンドは？（オプションは明示的に指定すること）

mke2fs -t ext2 /dev/hda1

IBMが開発した動的inodeの機能を持つファイルシステムは？

JFS

以下のコマンドを実行した時に作成されるファイルシステムは？

mkfs /dev/hda1

ext2

mkfsコマンドまたはmke2fsコマンドでファイルシステムを作成する際、不良ブロックがないか検査をしてから作成したい。適切なオプションは？

-C

知识点总结

- **逻辑分区编号**：在 MBR 磁盘中，逻辑分区的编号永远从 **5** 开始（1-4 留给主分区和扩展分区）。
 - **mke2fs** 与 **mkfs**：这两个命令用于格式化分区。**mke2fs** 是专门针对 ext 家族的工具。
 - **文件系统门派**：XFS 属于 SGI 公司，JFS 属于 IBM 公司。
-

选项解析与详细说明

1. 为什么是 `/dev/sda5` ?

- 1番目 (a): 第一块磁盘。
- 最初の論理パーティション (5): 无论你前面用了几个主分区，第一个逻辑分区的号码固定是 5。

2. 文件系统大家族

- ext2: 默认的文件系统（不带日志）。
- ext3: `ext2` + Journaling (ジャーナリング)。在 `mke2fs` 中，选项 `-j` 代表添加日志（即转为 ext3）。
- XFS: 高性能，由 SGI (Silicon Graphics) 开发。
- JFS: 由 IBM 开发。
- tmpfs: 存在于 内存 (メモリ) 中的临时文件系统，速度极快。

3. 创建命令对比

- `mkfs -t [类型]` : 通用格式。
- `mke2fs -t [类型]` : ext 专用。
- `mkswap` : 专门用于创建 Swap 交换空间（注意：创建后还需 `swapon` 才能使用）。

文件系统创建与识别表

功能/特点	命令/选项	记忆点
逻辑分区起始号	5	1-4 是主分区保留位
ext3 (带日志)	<code>mke2fs -j</code>	<code>j</code> = Journal (日志)
检查坏块 (不良ブロック)	<code>-c</code>	<code>c</code> = Check
SGI 开发	XFS	SGI → XFS
IBM 开发	JFS	IBM → JFS
默认 mkfs 类型	ext2	不加参数默认老版本
树状显示设备	<code>lsblk</code>	<code>ls</code> (列表) + <code>blk</code> (Block/块设备)

小学生级记忆法

1. **逻辑分区编号**：主分区是“领导”，占了 1、2、3、4 号位；逻辑分区是“员工”，只能从 5 号开始排。

2. **fdisk 子命令**：

- **p** (Print): 打印（显示）分区表。
- **q** (Quit): 放弃保存退出。
- **m** (Menu): 菜单/帮助。

3. **厂商对应**：

- **S**GI → **X**FS（想象 S 和 X 都有弯曲的曲线）。
- **I**BM → **J**FS（I 和 J 在字母表里是邻居）。

考试小秘籍

- **-j** 的特殊性：题目问 `mke2fs` 创建 `ext3` 且“选项不带参数”时，必须用 `-j`。如果你用 `-t ext3`，那 `ext3` 本身就是一个参数。
- `/dev/hda` vs `/dev/sda`：老题目用 `hd` (IDE)，新题目用 `sd` (SCSI/SATA)。看清题目，不要写错。
- `lsblk` 的形状：题目关键词「ツリー状（树状）」是 `lsblk` 的标志性特征。

次世代のLinux標準ファイルシステムとなるべく開発中で、以下の特徴を持つファイルシステムは？

- ・マルチデバイスに対応している
- ・サブボリューム単位でのスナップショット機能がある

Btrfs

以前のLinuxで標準的に使用されていたジャーナリングファイルシステムではないファイルシステムは？

ext2

fdiskコマンドで作成したパーティションにext系のファイルシステムを作成できるコマンドは？（短い方から先に2つ）

mkfs mke2fs

ブロックデバイスの情報（デバイス名や、UUID、ファイルシステムのタイプ）を表示するコマンドは？

blkid

Linuxの標準的なファイルシステムで、inodeの数が制限されるファイルシステムは？（古い方から3つ）

ext2 ext3 ext4

有効化してあるスワップ領域を無効化するコマンドは？

swapoff

作成済のスワップ領域を有効化するコマンドは？

swapon

umountコマンドのオプションのうち、「/etc/mtab」ファイルに記載されているファイルシステムを全てアンマウントするオプションは？

-a

「/dev/sr0」として認識されている内蔵CD-ROMドライブを「/mnt/cdrom」にマウントしたい。なお、これを実現する為のエントリは「/etc/fstab」ファイルに追加していない。適切なコマンドは？ (オプション、引数を省略して記述すること)

mount /dev/sr0 /mnt/cdrom

mountコマンドのオプションのうち、ファイルシステムの種類を指定するオプションは？

-t

mountコマンドのオプションのうち、「/etc/fstab」ファイルに記載されているファイルシステムを全てマウントするオプションは？

-a

ファイルシステムのマウントの設定を行うファイルは？ (絶対パスで回答)

/etc/fstab

「/dev/hdc」として認識されているCD-ROMドライブを「/mnt/cdrom」にマウントしたい。なお、これを実現する為のエントリは既に「/etc/fstab」ファイルに追加してある。適切なコマンドは？ (オプション、引数を省略せずに記述すること)

mount -t iso9660 /dev/hdc /mnt/cdrom

💡 知识点总结

- **Btrfs (Butter FS)**: 集成了快照、多设备支持的新型文件系统，被视为下一代标准。
- **ext 家族**: Linux 的老牌支柱。ext2 是唯一的“非日志”系统，ext2/3/4 都有 **inode** 数量限制。
- **挂载管理**: mount 负责“连接”设备到目录，/etc/fstab 是自动挂载的“剧本”。

🔍 选项解析与详细说明

1. 文件系统特性

- **Btrfs**: 关键词是 **マルチデバイス（多设备）** 和 **スナップショット（快照）**。
- **ext2**: 它是 **ジャーナリング（日志功能）** 诞生前的标准，安全性不如后辈，但更简单。
- **inode 限制**: `ext2/3/4` 在创建时就固定了 **inode** 数量，如果小文件太多，即使硬盘有空间，inode 用完也无法存文件。

2. 挂载命令的细节

- `mount -a`: 读取 `/etc/fstab`，把里面**没挂载的都挂上**。
- `umount -a`: 参考 `/etc/mtab`，把所有能卸载的**都卸载**。
- `mount -t [类型]`: `t` 代表 **Type**。比如 CD-ROM 对应的类型是 `iso9660`。

3. 查询与 Swap

- `blkid`: 查看设备的 **UUID**（唯一身份 ID）和文件系统类型，非常有用的诊断工具。
- `swapon / swapoff`: 就像开关一样，开启或关闭交换空间。

文件系统与挂载对照表

核心关键词 (日本語)	对应答案	记忆点
スナップショット / 次世代	Btrfs	Better Tree FS
ジャーナリングではない	ext2	老古董没日志
UUID / タイプを表示	blkid	Blk (Block) + ID
inode 数が制限される	ext2 ext3 ext4	ext 家族三兄弟
全てマウント/アンマウント	-a	all (全部)
ファイルシステムの設定	/etc/fstab	file system table

小学生级记忆法

1. **Btrfs 记忆**: 看到“次世代”和“快照 (Snapshot)” ，就想它是“**Better** (更好)” 的文件系统 → **Btrfs**。
2. **blkid 记忆**: 想看硬盘的“身份证 (ID)” ，就用 `blkid` 。
3. **mount 命令的“简写原则”** :
 - 如果 `/etc/fstab` 里写好了，`mount` 后面只跟设备名**或**挂载点就行。
 - 如果没有写好，就必须写成 `mount [设备] [挂载点]` 。

- **注意 CD-ROM 类型**：考试中如果要求明确指定 CD-ROM 的类型，记得写 `iso9660`。
- **/etc/fstab 拼写**：这是高频考点，`fstab` 意思是 **File System Table**，千万别拼错。
- **umount 拼写**：注意！命令是 `umount`，中间没有 'n'，很多人会错写成 `unmount`。
- **-a 选项**：只要题目说「**全て（全部）**」，选项基本都是 `-a`。

ext3ファイルシステムの「/dev/sda1」を「/export」に読み取り専用でマウントしたい。適切なコマンドは？ (オプションはアルファベット順に指定)

`mount -o ro -t ext3 /dev/sda1 /export`

「/etc/fstab」ファイルを編集して、ext3ファイルシステムの「/dev/sda1」をシステム起動時から「/boot」に自動マウントするよう設定したい。適切な記述は？ (マウントオプションのアルファベット順に2つ)なお、「/dev/sda1」をdumpコマンドの対象とし、fsckは優先度を2とすること。

`/dev/sda1 /boot ext3 auto 1 2`

`/dev/sda1 /boot ext3 defaults 1 2`

「/etc/fstab」ファイルに記述するマウントオプションで、入出力を非同期で行うものは？

`async`

現在マウントされているxfs形式のファイルシステムを全てアンマウントしたい。適切なコマンドは？ (オプションはアルファベット順にまとめて指定)

`umount -at xfs`

「/etc/fstab」ファイルに記述するマウントオプションで、「mount -a」コマンドを実行した際にマウントしないように指定するものは？

`noauto`

「/etc/fstab」ファイルに記述するマウントオプションで、「mount -a」コマンドを実行した際にマウントするように指定するものは？

`auto`

「/etc/fstab」ファイルに記述するマウントオプションで、SUIDとSGIDを有効化するものは？

`suid`

mountコマンドのオプションのうち、続けてマウントオプションを指定する為のオプションは？

`-o`

「/etc/fstab」ファイルに記述するマウントオプションで、一般ユーザでマウントを可能にし、本人のみアンマウント出来るようにするものは？

`user`

以下のコマンドを実行して「/dev/sdb2」を「/data」にマウントした。「/dev/sdb2」を正しく（エラーの出ないように）アンマウントできるコマンドは？

mount -t xfs /dev/sdb2 /data(ファイルシステムを指定しない方法とする。また引数が短い方から先に)

umount /data

umount /dev/sdb2

「/etc/fstab」ファイルに記述するマウントオプションで、読み書きを許可してマウントするものは？

rw

ファイルシステムをマウントすることが出来るコマンドは？

mount

💡 知识点总结

掌握 `mount` 命令的进阶选项以及 `/etc/fstab` 配置文件的详细参数，实现对文件系统的精细化管理（如权限控制、读写策略）。

🔍 选项解析与详细说明

这一组题目的核心在于「マウントオプション（挂载选项）」。

- `/etc/fstab` 的六个字段：设备名、挂载点、文件系统类型、**挂载选项**、dump 备份标志、fsck 检测顺序。
- `mount -o` : 这里的 `o` 代表 **Options**。后面可以跟多个具体的挂载参数（用逗号隔开）。
- 卸载 (`umount`) : 可以指定 **デバイス名**（设备名 `/dev/sdb2`）或 **マウントポイント**（挂载点 `/data`）进行卸载。

📊 `/etc/fstab` 常用选项对照表

考试中对这些选项的“功能描述”考察非常细致：

选项 (日本語)	功能描述 (中文)	记忆点
ro	読み取り専用 (Read-Only)	只读不写
rw	読み書き許可 (Read-Write)	能读能写
async	非同期入出力	Async (不步调一致，提升性能)
auto	mount -a 时自动挂载	默认选项
noauto	mount -a 时不自动挂载	必须手动挂
user	一般用户可挂载，仅限本人卸载	权限下放
users	一般用户可挂载，任何人可卸载	这里的 s 代表复数 (大家)
defaults	默认选项组合	包含 rw, suid, dev, exec, auto, nouser, async

🧠 小学生级记忆法

- 字母顺序 (アルファベット順): 题目经常要求“按字母顺序”。比如 `ro` 和 `ext3`，在命令里要排好队。`mount -o ro -t ext3`。
- `/etc/fstab` 尾巴上的两个数字：
 - 倒数第二个 (dump): 通常是 `0` (不备份) 或 `1` (备份)。
 - 最后一位 (fsck): `0` 不检查，`1` 根分区最优先，`2` 其他分区后检查。
- 卸载命令 `umount`: 想象它很“大方”，你告诉它“那是哪台机器 (设备名)”或者“那个门口 (挂载点)”，它都能帮你把连接切断。

✍️ 考试小秘籍

- `-at` 的合并使用: 题目要求卸载所有 xfs，`umount -a -t xfs` 可以合并写成 `umount -at xfs`，且符合字母顺序。
- `defaults` 的坑: 在写 `/etc/fstab` 配置时，如果你不确定写什么，通常写 `defaults`。它包含了 `auto`，所以这两者在很多考试场景下都是正确答案。
- 注意 `umount` 拼写: 再次提醒，没有第二个 `n`！

現在マウントされているファイルシステムを確認できる、/procディレクトリ内のファイルは？（絶対パスで、ファイルパスの短い順に2つ）

/proc/mounts

/proc/self/mounts

「/etc/fstab」ファイルに記述するマウントオプションで、一般ユーザによるマウントを禁止するのは？

nouser

現在マウントされているファイルシステムの一覧を単体で表示するコマンドは？（dfコマンド以外で）

mount

「/etc/fstab」ファイルに記述するマウントオプションで、バイナリの実行を禁止するのは？

noexec

「/etc/fstab」ファイルに記述するマウントオプションで、読み取り専用でマウントするのは？

ro

「/dev/hdc」として認識されているCD-ROMドライブを「/mnt/cdrom」にマウントしたい。なお、これを実現する為のエントリは既に「/etc/fstab」ファイルに追加してある。適切なコマンドは？（引数のアルファベット順に2つ。引数はできるだけ省略する事）

mount /dev/hdc

mount /mnt/cdrom

「/etc/fstab」ファイルに記述するマウントオプションで、バイナリの実行を許可するのは？

exec

「/etc/fstab」ファイルに記述するマウントオプションで、一般ユーザでマウントを可能にし、誰でもアンマウント出来るようにするものは？

users

ファイルシステムをアンマウントすることが出来るコマンドは？

umount

💡 知识点总结

掌握查看当前挂载状态的方法，以及如何通过 `no` 前缀开关 `/etc/fstab` 中的安全限制选项。

🔍 选项解析与详细说明

这一组题目的重点在于「マウントの制限（挂载限制）」和「情報の確認（信息确认）」。

• 查看挂载信息：

- **命令**：直接输入 `mount`（不带参数）可以列出所有当前挂载的文件系统。
- **文件**：内核在 `/proc/mounts` 中实时记录挂载情况。`/proc/self/mounts` 指向当前进程的挂载视图，虽然路径长一点，但内容基本一致。

• 权限控制选项：

- **user vs users**：`user`（单数）只允许挂载者本人卸载；`users`（复数）允许所有人（anyone）卸载。

- **no 前缀**：在 Linux 挂载选项中，很多功能通过加 **no** 来关闭。例如 **exec**（允许执行二进制文件）的反义词就是 **noexec**；**user** 的反义词是 **nouser**。

 挂载权限与安全选项对照表

功能 (日本語)	允许/开启	禁止/关闭 (加 no)	记忆点
执行程序	exec	noexec	Execute (执行)
一般用户挂载	user / users	nouser	默认通常是 nouser
读写权限	rw	ro	Read Only (只读)
SUID权限	suid	nosuid	安全增强常用 nosuid

 小学生级记忆法

1. **省略挂载法**：如果 **/etc/fstab** 里已经写好了“剧本”，**mount** 命令就可以偷懒。
 - 只要告诉它**“用哪个设备”（**/dev/hdc**）或者“去哪个门口”**（**/mnt/cdrom**），它就能自动对应上。
2. **users 的复数意义**：
 - **user** (单数)：我很自私，我挂载的只有我能卸载。
 - **users** (复数)：大家都是好朋友，谁都能帮我卸载。
3. **/proc/mounts 的短路径**：
 - 题目要求按路径**“短到长”**排列，先写 **/proc/mounts**（12字符），再写 **/proc/self/mounts**（17字符）。

 考试小秘籍

- a. **mount 命令的独立性**：题目问“**単体で表示（单体显示）**”且排除 **df** 时，想都不用想，直接填 **mount**。
- b. **省略参数的条件**：考试时注意前提条件。只有在「**既に /etc/fstab に追加してある（已经在fstab中添加）**」的情况下，才能省略另一半参数。
- c. **绝对路径要求**：问文件位置时，题目明确要求「**絶対パス（绝对路径）**」，开头的 **/** 一定要写。

パッケージ情報の検索・参照などは行えないが、パッケージ情報のデータベースを使って依存関係を調整しながらパッケージのインストール・アンインストールなどを行えるDebianパッケージの管理用コマンドは？

apt-get

dpkgコマンドを使用して、「**procmal**」パッケージを設定ファイルを含めて全てアンインストールしたい。適切なコマンドは？（アクションが短い方から先に2つ）

dpkg -P procmal / dpkg --purge procmal

apt-getコマンドを使用して、ディストリビューションを最新版にしたい。適切なコマンドは？

apt-get dist-upgrade

apt-cacheコマンドのサブコマンドのうち、指定したパッケージへ依存しているパッケージを含めた詳細情報を表示するサブコマンドは？

showpkg

yumコマンドを使用して、「**Emacs**」パッケージグループをインストールしたい。適切なコマンドは？

yum groupinstall Emacs

apt-getコマンドのサブコマンドのうち、インストール済みの全パッケージとカーネルをアップグレードするサブコマンドは？

dist-upgrade

apt-getコマンドを使用して、「**apache2**」パッケージをアンインストールしたい。適切なコマンドは？

apt-get remove apache2

APTツールにおいて特定のファイルが含まれているパッケージを検索するコマンドは？

apt-file

dpkgコマンドを使用して、インストール済みのパッケージを一覧表示させたい。適切なコマンドは？（アクションが短い方から先に2つ）

dpkg -l / dpkg --list

rpmコマンドの併用オプションのうち、依存関係を無視するオプションは？

--nodeps

知识点总结

通过专门的包管理工具，实现对软件及其「依存関係（依赖关系）」的自动化安装、卸载、查询与升级。

选项解析与详细说明

4. Debian 系 (APT/dpkg)

- `apt-get` : 经典管理工具。它能处理依赖, 但不能像 `apt-cache` 那样灵活搜索。
- `apt-cache showpkg` : 这里的 `showpkg` 会显示非常详尽的信息, 包括哪些包依赖于它 (逆向依赖)。
- `apt-file` : 这是一个特殊的工具, 当你只知道文件名却不知道它属于哪个包时, 用它来搜索。
- `dpkg --purge (-P)` : 卸载的“最高境界”。`remove` 只删程序, `purge` 连「設定ファイル (配置文件)」一起抹除。

2. Red Hat 系 (YUM/rpm)

- `yum groupinstall` : YUM 可以按“组”安装 (比如整个“开发工具组”或“Emacs组”), 非常方便。
- `rpm --nodeps` : `no deps` (no dependencies) 的缩写。这是一个危险的选项, 强制系统忽略依赖关系进行操作。

包管理命令功能对照表

操作系统	基础工具 (不理依赖)	高级工具 (自动处理依赖)	搜索工具
Debian / Ubuntu	<code>dpkg</code>	<code>apt-get</code>	<code>apt-cache / apt-file</code>
Red Hat / CentOS	<code>rpm</code>	<code>yum</code> (或 <code>dnf</code>)	<code>yum search</code>

小学生级记忆法

1. Purge (清除):

- 看到「設定ファイルを含めて (包含配置文件)」, 就要用 `Purge`。
- `Purge` 和 `Pprocmail` 开头字母一样, 顺便记住了缩写是 `-P`。

2. `dist-upgrade` (发行版升级):

- 看到「ディストリビューションを最新版 (发行版最新)」或「カーネル (内核)」升级, 就选带 `dist` 的那个。

3. `--nodeps` :

- 忽略依赖（**no dependencies**）。

4. **apt-file**：

- 题目问「**特定のファイル（特定文件）**」在哪个包里？找“文件（File）”就用 **apt-file**。

考试小秘籍

a. **upgrade** vs **dist-upgrade**：

- **upgrade** 只升版本。
- **dist-upgrade** 会聪明地处理依赖关系的变化，甚至会安装新包或删除旧包（常用于内核升级）。

b. **dpkg** 的参数大小写：

- **-l** (小写L)：List，列出包。
- **-L** (大写L)：显示该包安装了哪些文件到系统。
- **-P** (大写P)：Purge，彻底清除。

c. **apt-cache** 的特殊性：

- 它只负责「**参照・検索**」（查询），不负责修改系统（安装/卸载）。

dpkgツールにおいてインストール済みのパッケージを、インストールした時と同じように再設定するコマンドは？

dpkg-reconfigure

apt-getコマンドのサブコマンドのうち、パッケージのデータベースを最新版に更新するサブコマンドは？

update

rpmコマンドの併用オプションのうち、MD5によるファイルの改ざんを検査しないオプションは？

--nomd5

rpmコマンドの併用オプションのうち、設定ファイルを一覧表示するオプションは？（短い方から先に2つ）

-c / --configfiles

yumコマンドを使用して、アップデート可能なパッケージを確認したい。適切なコマンドは？

yum check-update

dpkgコマンドのアクションのうち、インストールが完了していない不完全なパッケージを表示するアクションは？（短い方から先に2つ）

-C / --audit

dpkgコマンドのアクションのうち、パッケージをインストールするアクションは？（短い方から先に2つ）

-i / --install

yumコマンドのサブコマンドのうち、パッケージをインストールするサブコマンドは？

install

yumコマンドを使用して、アップデート可能な全てのパッケージをアップデートしたい。適切なコマンドは？

yum update

apt-cacheコマンドのサブコマンドのうち、指定したパッケージのサイズやファイル名などを含めた情報を表示するサブコマンドは？

show

rpmコマンドのオプションのうち、指定したパッケージがインストールされているかを照会するオプションは？（短い方から先に2つ）

-q / --query

apt-fileコマンドで、「docker.io」パッケージに含まれているファイルの一覧を表示させたい。空欄に入る適切なアクションは？（アルファベット順に2つ）

apt-file _____ docker.io

list / show

💡 知识点总结

通过专门的子命令和选项，实现对已安装软件包的二次配置、安全校验（改ざん検査）以及版本同步。

🔍 选项解析与详细说明

3. Debian 系 (dpkg/apt) 的深度操作

- **dpkg-reconfigure** : 这是一个非常有用的工具。当软件包安装好后，如果你想重新回答安装时的那些配置问题（比如时区、键盘布局），就用它。
- **dpkg --audit (-C)** : 用于“审计”。它会扫描系统中那些因为断电或报错导致「不完全（不完整）」安装的包。
- **apt-get update** : 注意！这个命令 **不升级软件**，它只是从服务器下载最新的「データベース（数据库）」列表，告诉你哪些包有更新。

- `apt-cache show` : 显示包的详细描述、版本、大小等信息。

2. Red Hat 系 (RPM/YUM) 的高级检索

- `rpm -qc` : 这是一个组合选项。 `-q` (query) 是查询, `-c` (`--configfiles`) 专门列出「設定ファイル (配置文件)」 的路径。
- `--nomd5` : 跳过 MD5 校验。MD5 常用于检查文件是否被「改ざん (篡改)」。
- `yum check-update` : 仅仅是“检查”, 不会真的下载安装。
- `yum update` : 实际执行更新操作。

软件包“动作”对照表

目的 (日本語)	dpkg (Debian)	rpm (Red Hat)	yum / apt (高级)
安装	<code>-i / --install</code>	<code>-i / -ivh</code>	<code>install</code>
查询是否安装	(使用 <code>-l</code>)	<code>-q / --query</code>	<code>list installed</code>
仅查配置文件	(使用 <code>-L</code>)	<code>-c / --configfiles</code>	-
不完全包检查	<code>-C / --audit</code>	<code>-Va</code> (全校验)	-
包含文件列表	<code>apt-file list/show</code>	<code>rpm -ql</code>	-

小学生级记忆法

1. `reconfigure` (重配置):
 - 看到「再設定 (さいせってい)」, 对应的英文就是 **re-configure**。
2. `audit` (审计):
 - **Audit** 本意就是审计、检查。发现系统里有“坏账” (不完整的包), 就得靠 **Audit** (`-C`)。
3. `configfiles` 记忆:
 - 配置文件叫 **Config**, 所以选项是 `-c`。
4. `update` (数据库更新):
 - 电脑想知道超市有什么新货, 得先 **update** 下清单 (数据库)。

考试小秘籍

- a. `apt-get update` 还是 `upgrade` ? :
 - 题目问「データベース (数据库)」更新 → `update`。
 - 题目问「パッケージ本体 (包本身)」更新 → `upgrade`。

b. `dpkg` の大小写陷阱:

- `-c` (小写): 列出 `.deb` 软件包文件中的内容 (安装前看)。
- `-C` (大写): 检查不完整的包 (安装后审计)。
- `-L` (大写): 查看已安装包的文件列表。

c. `apt-file` 的动作:

- 搜索文件列表时, `list` 和 `show` 都可以。考试如果要求写两个, 记得这对组合。

rpmコマンドの併用オプションのうち、詳細情報を表示するオプションは?

-V

YUMツールではネットワーク経由で最新のパッケージを取得できる。パッケージの取得元(リポジトリ)を設定するファイルが格納されているディレクトリは? (絶対パスで回答)

/etc/yum.repos.d/

dpkgコマンドのアクションのうち、指定したインストール済みパッケージの詳細情報を表示するアクションは? (短い方から先に2つ)

-s / --status

rpmコマンドのオプションのうち、パッケージの検査を行うオプションは? (短い方から先に2つ)

-V / --verify

yumコマンドのサブコマンドのうち、パッケージをアンインストールするサブコマンドは?

remove

apt-getコマンドを使用して、「apache2」パッケージをインストールしたい。適切なコマンドは?

apt-get install apache2

dpkgコマンドのオプションのうち、同バージョンが既にインストールされている場合はインストール行わないオプションは?

-E

RPMツールにおいてパッケージのインストール・アンインストールなど、基本的なパッケージ管理を行うコマンドは?

rpm

apt-fileコマンドのアクションのうち、パッケージ情報を最新版に更新するアクションは?

update

yumコマンドのサブコマンドのうち、パッケージの詳細な情報を表示するサブコマンドは?

info

yumコマンドを使用して、「emacs」パッケージが依存しているパッケージの名を一覧表示させたい。適切なコマンドは？

yum deplist emacs

dpkgコマンドのアクションのうち、指定したパッケージからインストールされたファイルを一覧表示するアクションは？（短い方から先に2つ）

-L / --listfiles

rpmコマンドのオプションのうち、パッケージをアップグレードする(なければインストールしない)オプションは？（短い方から先に2つ）

-F / --freshen

rpmコマンドのオプションのうち、パッケージをアンインストールするオプションは？（短い方から先に2つ）

-e / --erase

apt-cacheコマンドのサブコマンドのうち、キーワードを含むパッケージを検索するサブコマンドは？

search

💡 知识点总结

深入掌握 RPM、YUM、dpkg、APT 等工具的查询、验证及依赖分析的高级指令，确保能够精确管理软件包的生命周期。

🔍 选项解析与详细说明

这一组题目聚焦于包管理器的“进阶动作”，例如如何查看「詳細情報」以及如何「検証（验证）」已安装文件的完整性。

- **RPM 验证与更新：**
 - `-V (--verify)`: 将已安装文件与原始包进行对比，检查是否有文件丢失或被篡改。
 - `-F (--freshen)`: 这是“补丁模式”，只有在系统中已经安装了旧版本时才会升级。
 - `-U (Upgrade)`: 与 `-F` 不同，如果没装旧版，`-U` 会直接安装。
- **YUM 仓库配置：**
 - `/etc/yum.repos.d/`: 所有的 `.repo` 配置文件都放在这里。「`repos.d`」是 `repository directory` 的缩写。
- **dpkg 查询细节：**
 - `-s (--status)`: 查看包的状态（已安装、未安装等）和详细元数据。

- `-L (--listfiles)`: 查看这个包到底把文件装到了系统的哪些位置。
- 依赖分析:
 - `yum deplist`: 列出所有 **dependency list**，非常直观地查看依赖树。

软件包管理“动作”速查表

功能 (日本語)	Red Hat (RPM/YUM)	Debian (dpkg/APT)	记忆钩子
アンインストール (卸载)	<code>-e / remove</code>	<code>remove / purge</code>	Erase (抹除)
アップグレード (仅限已安装)	<code>-F / --freshen</code>	-	Fresh (保持新鲜)
詳細情報を表示	<code>-v / info</code>	<code>-s / --status</code>	Verbose (冗长的)
パッケージ検索	<code>search</code>	<code>search</code>	常用 search
ファイルの検証	<code>-V / --verify</code>	(使用 debsums)	Verify (验证)
依存関係の一覧	<code>deplist</code>	<code>showpkg</code>	Dependency List

小学生级记忆法

1. `-F (Freshen)` 记忆:

- 想象你在给花浇水。如果本来就有花（已安装旧版），水才有意义（升级）；如果没有花，浇水也没用（不安装）。

2. `-E (Skip if installed)` 记忆:

- **E** 代表 **Exist**（存在）。如果已经 **Exist** 了，就跳过不装了。

3. YUM 目录名:

- `/etc/yum.repos.d/`。别忘了中间那个 `.d`，它是目录的标志。

4. `-e (Erase)` 记忆:

- 卸载就是把软件 **Erase**（擦掉）。

考试小秘籍

a. `-v` 还是 `-s` ? :

- 在 `rpm` 里，如果要看详细过程，用 `-v`。
- 在 `dpkg` 里，如果要看包的具体状态信息，用 `-s` (Status)。

b. `deplist` vs `showpkg` :

- YUM 用 `deplist`。
- APT/apt-cache 用 `showpkg`（这个之前出现过，注意区分）。

c. **-F** 的特殊要求:

- 考试看到「なければインストールしない（如果没有就不安装）」，这是判定 **-F** (freshen) 的唯一标准，千万别填成 **-U**。

yumコマンドのサブコマンドのうち、パッケージをアップデートするサブコマンドは?

update

yumコマンドのサブコマンドのうち、指定したキーワードでパッケージを検索するサブコマンドは?

search

dpkgコマンドのオプションのうち、ディレクトリを再帰的に処理するオプションは?

-R

apt-cacheコマンドのサブコマンドのうち、指定したパッケージが依存しているパッケージの名前を一覧表示するサブコマンドは?

depends

rpmコマンドの併用オプションのうち、指定したパッケージの変更履歴を表示するオプションは?

--changelog

rpmコマンドの併用オプションのうち、指定したパッケージに含まれるファイルを表示するオプションは? (短い方から先に2つ)

-l / --list

rpmコマンドの併用オプションのうち、インストール済みの全パッケージを表示するオプションは? (短い方から先に2つ)

-a / --all

APTツールではネットワーク経由で最新のパッケージを取得できるがパッケージの取得元(リポジトリ)を設定するファイルは? (絶対パスで回答)

/etc/apt/sources.list

rpmコマンドのオプションのうち、パッケージをアップグレード(なければインストール)するオプションは? (短い方から先に2つ)

-U / --upgrade

rpmコマンドの併用オプションのうち、照会対象をパッケージファイルとするオプションは? (短い方から先に2つ)

-p / --package

dpkgコマンドのアクションのうち、インストール済みのパッケージを一覧表示するアクションは? (短い方から先に2つ)

-l / --list

rpmコマンドを使用して、インストールされている「postfix」パッケージの詳細情報を調べたい。適切なコマンドは?(オプション、併用オプションの順でまとめて指定。また、それらは省略した形を使用)

rpm -qi postfix

apt-cacheコマンドを使用して、「ssh」パッケージが依存しているパッケージの名前一覧表示させたい。適切なコマンドは?

apt-cache depends ssh

パッケージ情報のデータベースを使って依存関係を調整しながらRPMパッケージのインストール・アンインストールなどを行えるコマンドは?

yum

apt-getコマンドのサブコマンドのうち、パッケージをアンインストールするサブコマンドは?

remove

rpmコマンドを使用して、インストールされている全パッケージの検査を行いたい。適切なコマンドは?(オプション、併用オプションの順でまとめて指定。また、それらは省略した形を使用)

rpm -Va

rpmコマンドの併用オプションのうち、進行状況を「#」で表示するオプションは?(短い方から先に2つ)

-h / --hash

apt-cacheコマンドを使用して、キーワード「ssh」を含むパッケージを表示させたい。適切なコマンドは?

apt-cache search ssh

rpmコマンドの併用オプションのうち、指定したファイルのインストール元のパッケージを表示するオプションは?(短い方から先に2つ)

-f / --file

dpkgコマンドのアクションのうち、設定ファイルを含め完全にパッケージをアンインストールするアクションは?(短い方から先に2つ)

-P / --purge

dpkgコマンドのアクションのうち、設定ファイルを残してパッケージをアンインストールするアクションは?(短い方から先に2つ)

-r / --remove

 知识点总结

🔍 选项解析与详细说明

这组题目主要考察的是“深度查询”能力：

- **RPM 组合选项：**
 - `-qi` : info。查看已安装包的详细说明（版本、来源等）。
 - `-ql` : list。查看包里的文件都装到哪去了。
 - `-qf` : file。逆向查询：根据一个文件找出它属于哪个包。
 - `-Va` : Verify + all。对系统里所有的包进行一次大检查。
- **APT 的依赖查询：**
 - `depends` : 直接列出“我依赖谁”。
- **仓库配置文件路径：**
 - **Debian 系:** `/etc/apt/sources.list` （源列表）。
 - **Red Hat 系:** `/etc/yum.repos.d/` （仓库目录）。

📊 核心操作对照表（秒杀专用）

功能 (日本語)	RPM 选项 (短格式)	APT/YUM 子命令	记忆钩子
全量显示	<code>-qa / -l (dpkg)</code>	list	all / list
文件溯源	<code>-qf</code>	-	file
依赖查询	-	<code>depends / deplist</code>	Depends
安装进度条	<code>-h</code>	-	# 号像 hash
升级(包含新装)	<code>-U</code>	upgrade	Upgrade
配置文件位置	<code>/etc/yum.repos.d/</code>	<code>/etc/apt/sources.list</code>	Sources (源头)

🧠 小学生级记忆法

1. **# 号进度条 (`-h`):**
 - 看到题目说用「#」显示进度，就想这个符号叫 Hash，所以选 `-h`。
2. **`sources.list` (源列表):**
 - 软件的“水源”在哪里？在 **sources** 列表里。

3. `-R` (Recursive):

- 只要是 Linux 里的“递归（处理目录下所有文件）”，大写 `-R` 几乎是通解。

4. `depends` (依赖):

- 英文直译。问依赖关系，子命令就是 `depends`。

考试小秘籍

a. `-U` 与 `-F` 再次强调:

- 题目如果说「なければインストール」（如果没有就安装）→ 选 `-U`。
- 题目如果说「なければインストールしない」（如果没有就不装）→ 选 `-F`。

b. 绝对路径的“斜杠”:

- `/etc/apt/sources.list` 这里的 `s` 是复数，别忘了加。

c. `apt-get` vs `apt-cache`:

- 动作类（改动系统）用 `get`，查询类（不改系统）用 `cache`。

rpmコマンドを使用して、インストール前に「postfix-1.i386.rpm」ファイルからインストールされるファイルを調べたい。適切なコマンドは? (オプションを先に指定し、併用オプションはアルファベット順でまとめて指定。また、それらは省略した形を使用)

rpm -qlp postfix-1.i386.rpm

apt-getコマンドのサブコマンドのうち、既存のパッケージの削除や新規パッケージの追加をせずに、インストール済みの全パッケージを対象に更新できるものをアップグレードするサブコマンドは?

upgrade

rpmコマンドのオプションのうち、パッケージをインストールするオプションは? (短い方から先に2つ)

-i / --install

yumコマンドのサブコマンドのうち、パッケージグループを一覧表示するサブコマンドは?

grouplist

apt-fileコマンドで、パスに「docker」が入っているファイルを含むパッケージを検索したい。空欄に入る適切なアクションは? (アルファベット順に2つ)

apt-file _____ docker

find / search

apt-getコマンドのサブコマンドのうち、過去に取得したパッケージファイルを削除するサブコマンドは?

clean

dpkgコマンドのオプションのうち、新バージョンが既にインストールされている場合はインストールを行わないオプションは？

-G

yumコマンドのサブコマンドのうち、パッケージを一覧表示するサブコマンドは？

list

APTツールにおいてapt-getとapt-cacheの機能を統合したコマンドは？

apt

dpkgツールにおいてパッケージのインストール・アンインストールなど、基本的なパッケージ管理を行うコマンドは？

dpkg

APTツールにおいてパッケージ情報の検索・参照などを行うコマンドは？

apt-cache

dpkgツールの設定ファイルは？（絶対パスで回答）

/etc/dpkg/dpkg.cfg

yumコマンドのサブコマンドのうち、パッケージグループをインストールするサブコマンドは？

groupinstall

rpmコマンドを使用して、「postfix」パッケージがインストールされているかどうか確認したい。適切なコマンドは？（オプションが短い方から先に2つ）

rpm -q postfix / rpm --query postfix

dpkgコマンドのアクションのうち、指定したファイルのインストール元のパッケージを表示するアクションは？（短い方から先に2つ）

-S / --search

YUMツールの設定ファイルは？（絶対パスで回答）

/etc/yum.conf

apt-getコマンドのサブコマンドのうち、ディストリビューションを最新版にするサブコマンドは？

dist-upgrade

rpmコマンドの併用オプションのうち、実際には実行せずにテストを実行するオプションは？

--test

yumコマンドのサブコマンドのうち、アップデート可能なパッケージを表示するサブコマンドは？

check-update

apt-getコマンドのサブコマンドのうち、パッケージをインストールまたはアップグレードするサブコマンドは？

install

rpmコマンドを使用して、依存関係を無視して「postfix」パッケージを強制的にアンインストールしたい。適切なコマンドは？(オプション、併用オプションの順で指定。また、オプションは省略した形を使用)

rpm -e --nodeps postfix

rpmコマンドの併用オプションのうち、指定したパッケージの詳細を表示するオプションは？(短い方から先に2つ)

-i / --info

💡 知识点总结

掌握包管理器 (RPM, YUM, dpkg, APT) 的高级过滤选项、测试模式 (--test) 以及全局配置文件 (.conf/ .cfg) 的存放位置。

🔍 选项解析与详细说明

- RPM 预查询 (`-qlp`) :
 - `-q` : 查询 (Query)。
 - `-l` : 列表 (List)。
 - `-p` : 针对包文件 (Package file)。当你手里只有一个 `.rpm` 文件还没安装时，必须加 `-p` 才能查里面的内容。
- 清理缓存 (`clean`) :
 - `apt-get` 在安装时会将 `.deb` 下载到本地，长期不清理会占用空间。使用 `clean` 可以将其清除。
- 强制操作 (`--nodeps` , `--test`) :
 - `--nodeps` : 忽略依赖关系硬干（强制卸载/安装）。
 - `--test` : 模拟运行，看会不会报错，但「**実際には実行しない**（实际上不执行）」。
- 配置文件路径：
 - YUM 的核心配置在 `/etc/yum.conf`（注意和存储仓库地址的 `.repos.d` 目录区分）。
 - dpkg 的核心配置在 `/etc/dpkg/dpkg.cfg`。

📊 核心命令与子命令速查表

功能 (日本語)	命令/子命令	记忆点
安装前查询包内容	<code>rpm -qlp</code>	p = Package file (未装)
全量更新 (不删/不增包)	<code>upgrade</code>	稳健升级
清理下载缓存	<code>clean</code>	Clean (打扫)
不处理新版本 (跳过)	<code>-G</code>	G = Greater (已有更大的版本)
测试运行	<code>--test</code>	只看不练
忽略依赖强制卸载	<code>rpm -e --nodeps</code>	no dependencies

🧠 小学生级记忆法

1. `-G` (Don't downgrade) 记忆:

- 看到题目说“如果已经装了新版本就不装了”，这就是 **G** (Great / 高版本) 保护模式，防止你“降级”。

2. `-S` (Search) 逆向查询:

- 在 `dpkg` 里，根据文件找包，动作是大写的 `-S` (Search)。

3. `yum.conf` 和 `dpkg.cfg` :

- 两个工具的配置文件名不一样：YUM 用 `.conf` (Common / 通用后缀)，dpkg 用 `.cfg` (ConFiG)。

4. `upgrade` vs `dist-upgrade` :

- 普通升级（不删包）叫 `upgrade` ；
- 彻底升级（可能会删旧包保新包，甚至升内核）叫 `dist-upgrade` 。

📝 考试小秘籍

a. `-qlp` 的顺序:

- 题目要求「併用オプションはアルファベット順（并发选项按字母顺）」：`l` 在 `p` 前面，所以是 `qlp`（`q` 是主选项放在最前面）。

b. `apt` 命令的诞生:

- 考试问“整合了 `apt-get` 和 `apt-cache` 的命令是什么？”，答案就是三个字母：`apt`。

c. YUM 分组操作:

- 只要看到「パッケージグループ（包组）」，命令里一定带 `group`。查列表是 `grouplist`，安装是 `groupinstall`。

ファイル・ディレクトリの操作と管理(主題1.02)

chmodコマンドを使用して記号でパーミッションを設定する際、権限の追加を意味する記号は?

+

ファイルやディレクトリを新規作成した際の、デフォルトのパーミッションを調整するコマンドは?

umask

ファイルとディレクトリを新規作成した際のデフォルトのパーミッションを、それぞれ「600」と「700」にしたい。適切なコマンドは?

umask 077

ファイルとディレクトリを新規作成した際のデフォルトのパーミッションを、それぞれ「640」と「750」にしたい。適切なコマンドは?

umask 027

「dir」ディレクトリ内において、自分以外のユーザが所有するファイルを削除できないようにしたい。なお、「dir」ディレクトリのパーミッションは755である。適切なコマンドは (パーミッションの指定には、1つ目は数値、2つ目は記号3文字を使用すること)

chmod 1755 dir / chmod o+t dir

ディレクトリに適用すると、ディレクトリ内の内容を参照できるパーミッションは? (記号1文字)

r

「file」ファイルのパーミッションを、所有者は読み取りと書き込みのみが可能で、所有グループとその他のユーザは読み取りのみが出来るように設定したい。適切なコマンドは? (パーミッションを、1つ目は数値、2つ目は上記の条件を訳するように順に指定する事)

chmod 644 file / chmod u=rw,go=r file

ファイルとディレクトリを新規作成した際のデフォルトのパーミッションを、それぞれ「644」と「755」にしたい。適切なコマンドは?

umask 022

プログラム「file」をどのユーザが実行しても、プログラムの所有者の権限で実行させたい。なお、「file」のパーミッションは755である。適切なコマンドは (パーミッションの指定には、1つ目は数値、2つ目は記号3文字を使用すること)

chmod 4755 file / chmod u+s file

ディレクトリに適用すると、そのディレクトリへ移動できるパーミッションは? (記号1文字)

x

掌握 Linux 中的权限管理（Permission），包括基础的读/写/执行、特殊的 SUID/Sticky Bit，以及决定初始权限的 `umask` 遮罩值。

🔍 选项解析与详细说明

这一组题目涉及 Linux 安全权限的核心：

- 基础权限 (Basic Permissions):
 - `r` (Read): 文件可读，目录可查看内容（`ls`）。
 - `w` (Write): 文件可修改，目录可增删文件。
 - `x` (Execute): 文件可运行，目录可进入（`cd`）。
- 特殊权限 (Special Permissions):
 - SUID (Set User ID): 数值 4000。让执行者临时拥有文件所有者的权限。
 - Sticky Bit: 数值 1000。常用于 `/tmp`，防止用户删除别人的文件。
- 默认权限遮罩 (`umask`):
 - 计算公式：
 - 目录默认：777 - `umask` = 结果
 - 文件默认：666 - `umask` = 结果（文件通常默认不给 `x`）
 - 例子： `umask 022` → 目录 755，文件 644。

📊 `umask` 常用设置对照表

目的 (日本語)	umask 值	文件权限	目录权限	记忆点
标准权限	22	644	755	最常用的设置
组禁止写入	27	640	750	0代表不扣除，2代表扣除写，7全扣
完全私有	77	600	700	只有所有者能动

🧠 小学生级记忆法

- 目录的 `r` 与 `x`：
 - `r` (Read) = 站在门口看一眼屋里有什么（`ls`）。
 - `x` (eXecute) = 抬腿跨进屋里（`cd`）。
- `umask` 计算小技巧：

- 把 `umask` 看成是“减法过滤器”。你想减掉什么权限，就写什么数字。
- 比如想减掉 **others** 的所有权限（7），就在最后一位写 7。

3. 特殊权限代号：

- **SUID** 是所有者的权力 → `u+s`。数值是 4。
- **Sticky Bit** 贴在目录上 → `o+t`（想成“T”是 Trap 陷阱，防止别人乱动）。数值是 1。

考试小秘籍

a. `umask` 的三位数字：

- 考试中经常给出目标权限，反求 `umask`。记住：**777 减去 目录权限 = umask**。

b. 符号表示法的 `=` 与 `+`：

- `u=rw` 是“设置为 rw”，不管之前是什么，现在只给 rw。
- `u+s` 是“额外增加 s 权限”，保留之前的权限。

c. 目录权限的特殊性：

- 题目问“进入目录（移动）”选 `x`；问“查看目录内容（参照）”选 `r`。这两个非常容易混淆，请务必记牢。

chmodコマンドを使用して記号でパーミッションを設定する際、権限を一度に指定する記号は？

=

chmodコマンドを使用して記号でパーミッションを設定する際、所有者を表す文字は？

u

「file」ファイルの所有ユーザをtestからstaffに変更したい。適切なコマンドは？

chown staff file

chmodコマンドを使用して記号でパーミッションを設定する際、読み取り権を表す文字は？

r

以下のような内容を入力するコマンドは？ -rw-r--r-- 1 test staff 24 Jul 21 13:38 index.html

ls -l index.html

chmodコマンドを使用して記号でパーミッションを設定する際、書き込み権を表す文字は？

w

chmodコマンドを使用して記号でパーミッションを設定する際、実行権を表す文字は？

x

「dir」ディレクトリおよびその配下にあるファイルの所有グループを「gp」に変更したい。適切なコマンドは？（コマンドが長い方から先に2つ、「:」は使用しないこと）

```
chown -R .gp dir / chgrp -R gp dir
```

ファイルやディレクトリのパーミッションを変更するコマンドは？

chmod

💡 知识点总结

深入掌握权限管理工具 `chmod`、`chown`、`chgrp` 的用法，包括符号模式（u, g, o, a）、特殊权限位（t）以及递归处理（-R）。

🔍 选项解析与详细说明

这一组题目专注于**所有权（Ownership）与符号模式（Symbolic Mode）**的细节：

- 符号模式的对象与操作：
 - 对象： `u` (user/所有者), `g` (group/属组), `o` (others/其他), `a` (all/所有人)。
 - 操作： `+` (增加), `-` (删除), `=` (精确覆盖设定)。
- 权限位字母：
 - `r` (read), `w` (write), `x` (execute), `t` (sticky bit)。
- 所有权变更命令：
 - `chown` : 变更**所有者**（也可以同时变属组）。
 - `chgrp` : 专门用于变更**属组**。
- 递归操作 (`-R`) :
 - 当题目提到「配下にあるファイル（及其下属文件）」时，必须使用大写的 `-R` (Recursive) 选项。

📊 权限管理命令速查表

操作需求	推荐命令	关键选项/参数
改变权限	chmod	u+x, 755, -R
改变所有者	chown	user:group, user
仅改变属组	chgrp	group
查看当前权限	ls -l	输出 -rwxr-xr-x 格式

🧠 小学生级记忆法

1. 符号对象的“拼音”记忆：

- **u** = **U**ser (我/所有者)
- **g** = **G**roup (我们/组)
- **o** = **O**ther (别人)
- **a** = **A**ll (大家)

2. `chown` 与 `chgrp` 的区别：

- `chown` 是 **OW**ner 的缩写 → 换主人。
- `chgrp` 是 **GRouP** 的缩写 → 换组。

3. 递归的 `-R`：

- 想象一个大写的 **R** 像一个循环的圈，代表把文件夹里里外外全部处理一遍。

4. `=` (等号) 的霸道：

- `+` 和 `-` 是商量着来，在原有基础上加减；`=` 是霸道总裁，不管以前是什么，现在必须“等于”我给的权限。

考试小秘籍

a. `chown` 变组的特殊写法：

- 虽然 `chgrp` 专门变组，但 `chown` 也可以。题目要求不使用 `:` 时，可以使用 `.group` 的格式（如 `chown .gp file`）。

b. 符号法变权限的技巧：

- 从 `644` (`rw-r--r--`) 到 `755` (`rw-r-xr-x`)，实际上是给 **所有人 (a)** 加上了 **执行权 (x)**，所以是 `a+x`。

c. Sticky Bit 的符号：

- 记牢 `t` 代表 Sticky Bit。如果题目问“记号”，它是 `t`；如果问“数值”，它是 `1`（加在三位权限数之前，如 `1755`）。

chmodコマンドを使用して記号でパーミッションを設定する際、権限の削除を意味する記号は？

-

chmodコマンドを使用して記号でパーミッションを設定する際、スティッキービットを表す文字は？

t

「file」ファイルのパーミッションを644から755に変更したい。適切なコマンドは？（パーミッションを1つ目は数値、2つ目は記号含め3文字で指定すること）

chmod 755 file / chmod a+x file

「dir」ディレクトリおよびその配下にあるファイルのパーミッションを755に設定したい。適切なコマンドは？(パーミッションの指定には数値を使用すること)

chmod -R 755 dir

「file」の所有グループのみをtestからgpに変更したい。適切なコマンドは？(chownコマンドは使用しないこと)

chgrp gp file

「file」の所有ユーザをtestからstaffに、所有グループをtestからgpに変更したい。適切なコマンドは？(「:」を使用するほうから先に2つ)

chown staff:gp file / chown staff.gp file

「dir」ディレクトリのパーミッションを700から500に変更したい。適切なコマンドは？(パーミッションを1つ目は数値、2つ目は-記号を用いて3文字、3つ目は=記号を用いて4文字で指定)

chmod 500 dir / chmod u-w dir / chmod u=rx dir

指定したファイルまたはディレクトリの所有グループを変更するコマンドは？ただし所有ユーザの変更はできないコマンドとする。

chgrp

chmodコマンドを使用して記号でパーミッションを設定する際、全てのユーザを表す文字は？

a

chmodコマンドを使用して記号でパーミッションを設定する際、SUIDまたはSGIDを表す文字は？

s

プログラム「file」をどのユーザが実行しても、プログラムを所有するグループの権限で実行させたい。なお、「file」のパーミッションは755である。適切なコマンドは？(パーミッションの指定には、1つ目は数値、2つ目は記号3文字を使用すること)

chmod 2755 file / chmod g+s file

ディレクトリに適用すると、ディレクトリ内にファイルやディレクトリを作成、また削除できるパーミッションは？(記号1文字)

w

chownコマンドを使用し「file」の所有グループのみをtestからgpに変更したい。適切なコマンドは？(「:」を使用するほうから先に2つ)

chown :gp file / chown .gp file

chmodコマンドを使用して記号でパーミッションを設定する際、その他のユーザを表す文字は？

o

chmodコマンドを使用して記号でパーミッションを設定する際、所有グループを表す文字は？

g

指定したファイルまたはディレクトリの所有ユーザや所有グループを変更するコマンドは？

chown

💡 知识点总结

精通 `chown`、`chgrp` 的多功能用法，以及 `chmod` 中特殊的权限位（SUID/SGID/Sticky Bit）及其符号表示法。

🔍 选项解析与详细说明

这一组题目进一步挖掘了权限变更的灵活语法：

- `chown` 的全能性：
 - 它可以同时改变所有者和属组：`chown staff:gp file`。
 - 也可以只改变属组：`chown :gp file`（注意冒号前的空位）。
 - 支持 `:` 和 `.` 两种分隔符。
- 特殊权限位的符号与数值：
 - **SUID**：所有者执行权增强（`u+s`），数值位 **4**（如 `4755`）。
 - **SGID**：属组执行权增强（`g+s`），数值位 **2**（如 `2755`）。
 - **Sticky Bit**：目录防删保护（`o+t`），数值位 **1**（如 `1755`）。
- 权限增减的逻辑：
 - 从 `700` (`rwX-----`) 到 `500` (`r-x-----`)，实际上是减去了写入权（`u-w`），或者重新设定为读和执行（`u=rX`）。

📊 权限变更“多种写法”对照表

需求 (日本語)	数值写法	符号写法 (+/-)	符号写法 (=)
700→500	<code>chmod 500</code>	<code>chmod u-w</code>	<code>chmod u=rX</code>
755→ 755 (加 SGID)	<code>chmod 2755</code>	<code>chmod g+s</code>	<code>chmod g=rws</code>
644→755	<code>chmod 755</code>	<code>chmod a+X</code>	<code>chmod a=rwx</code>

🧠 小学生级记忆法

1. SGID 记忆:

- 题目说「**所有するグループの権限で実行**」→ 关键词是“Group”，所以对应 **g+s**。
- 数值记法: SUID是老大(4)，SGID是老二(2)，Sticky是老三(1)。

2. chown 只改组:

- 想象 **chown [主人]:[组]**，现在主人不变，就把前面空出来 → **:gp**。

3. 目录的 w 权限:

- 在文件上，**w** 是改内容；在目录上，**w** 是**改名单**（创建、删除文件）。

4. 符号对象全家福:

- **u** (User), **g** (Group), **o** (Others), **a** (All)。

考试小秘籍

a. SGID 的判定:

- 看到「**グループの権限で実行**」，立刻反应出符号 **g+s** 和数值 **2**（放在三位权限数前）。

b. 权限简写要求:

- 如果题目要求“-记号 3 文字”，如 **700 -> 500**，答案是 **u-w**。
- 如果要求“=记号 4 文字”，则是 **u=rx**。

c. chown 的分隔符:

- 考试时如果没限制，用 **:** 是最标准、最通用的写法。如果要求写两种，别忘了可以用 **.**。

rmdirコマンドを使用してカレントディレクトリ内にある「dir」ディレクトリと、その子ディレクトリの「dir/dir2」を削除したい。適切なコマンドは？

rmdir -p dir/dir2

lsコマンドのオプションのうち、詳細情報を表示するオプションは？

-l

rmコマンドのオプションのうち、確認無しで強制的に削除するオプションは？

-f

rmコマンドのオプションのうち、削除するかどうか確認するオプションは？

-i

配下にあるファイルやディレクトリも含めて「dir1」ディレクトリごと、新規で「dir2」ディレクトリという名前でコピーしたい。適切なコマンドは？（オプションは小文字で指定）

```
cp -r dir1 dir2
```

カレントディレクトリに「dir/dir2」ディレクトリを作成したい。適切なコマンドは？ なお、「dir」ディレクトリは存在しない。

```
mkdir -p dir/dir2
```

touchコマンドのオプションのうち、アクセス時刻を変更するオプションは？

-a

lsコマンドのオプションのうち、ファイルタイプをつけて表示するオプションは？

-F

ファイルの種別を表示するコマンドは？

file

touchコマンドのオプションのうち、修正時刻を変更するオプションは？

-m

cpコマンドのオプションのうち、コピー先に同名のファイルがある場合に上書きするかどうか確認するオプションは？

-i

「dir1」ディレクトリにあるファイルやディレクトリを、「dir2」ディレクトリに移動したい。適切なコマンドは？

```
mv dir1/* dir2
```

lsコマンドのオプションのうち、inode番号を表示するオプションは？

-i

lsコマンドのオプションのうち、隠しファイルも含めて表示するオプションは？（但し、カレントディレクトリ（.）と親ディレクトリ（..）を表示しないものとする）

-A

「/home」ディレクトリの詳細情報のみを表示させたい。適切なコマンドは？（オプションはアルファベット順にまとめて指定）

```
ls -dl /home
```

知识点总结

掌握 Linux 基础文件管理操作（创建、删除、移动、复制）以及 `ls`、`touch` 等高频命令的常用选项。

选项解析与详细说明

这一组题目集中在文件和目录的日常维护命令上：

- 目录层级处理 (`-p`):
 - `mkdir -p` : 递归创建目录。如果父目录不存在，会一并创建 (parents) 。
 - `rmdir -p` : 递归删除空目录。删除子目录后，如果父目录也变空，则一并删除。
- `ls` 的多样化视图:
 - `-l` : long。显示权限、所有者、大小、时间等详细信息。
 - `-F` : Flassify。在文件名后加标识 (如 `/` 代表目录，`*` 代表可执行文件) 。
 - `-A` : 几乎 All。显示隐藏文件 (以 `.` 开头)，但不显示 `.` 和 `..` 。
 - `-d` : directory。只列出目录本身的信息，而不是列出目录下的内容。
- 安全与强制选项:
 - `-i` : interactive。操作前询问“你确定吗？” (`rm` , `cp` , `mv` 通用) 。
 - `-f` : force。强制执行，不提示，不报错。
- 时间戳管理 (`touch`):
 - `-a` : access time (访问时间) 。
 - `-m` : modification time (修改时间) 。

 `ls` 常用选项对照表

选项	功能描述 (日本語)	记忆钩子
<code>-l</code>	詳細情報を表示	Long (长格式)
<code>-a</code>	全てのファイルを表示	All (含 . 和 ..)
<code>-A</code>	. と .. 以外の全ファイルを表示	Almost all (几乎全部)
<code>-F</code>	ファイル種別を表示	Format/Indicator
<code>-i</code>	inode 番号を表示	inode
<code>-d</code>	ディレクトリ自体の情報を表示	Directory itself

 小学生级记忆法

1. 递归的 `-p` (Parents):
 - 不管是创建还是删除，只要涉及“带着爸爸（父目录）一起”，就用 `-p` 。
2. `ls -F` 的尾巴:
 - 执行 `ls -F` 就像给文件名戴个帽子：文件夹戴 `/`，程序戴 `*` 。

3. touch 时间识别:

- a = access (访问)
- m = modify (修改)

4. 复制目录的 -r:

- r = recursive (递归)。复制文件夹不是搬一张纸，是搬整个箱子及其内部，所以要递归。

考试小秘籍

a. ls -a vs ls -A:

- 题目如果说“包括隐藏文件，但不包括 . 和 ..”，一定要选大写的 -A。

b. 查看目录本身:

- 很多同学查目录信息会直接写 ls -l /home，但这会列出 /home 里的所有用户文件夹。要看 /home 这份“档案”本身，必须加 -d。

c. rmdir 的局限性:

- 记住 rmdir 只能删除空目录。如果目录里有文件，必须改用 rm -r。

cpコマンドのオプションのうち、ファイルの属性を保持したままコピーするオプションは？

-p

mvコマンドのオプションのうち、移動先ディレクトリに同名のファイルやディレクトリがある場合でも、確認無しで強制的に上書きするオプションは？

-f

アクセス権が700の「dir」ディレクトリを作成したい。適切なコマンドは？

mkdir -m 700 dir

「dir1」ディレクトリを、配下にあるファイルやディレクトリも含めて削除するコマンドは？ なお、「dir1」ディレクトリは空ではない。

rm -r dir1

「file」ファイルのタイムスタンプを現在の時刻に変更したい。適切なコマンドは？

touch file

ファイルやディレクトリをコピーするコマンドは？

cp

空のディレクトリで「touch file」を実行したとき、「file」のファイルサイズは？（数字のみで回答）

0

空のディレクトリのみを削除できるコマンドは？

rmkdir

mvコマンドのオプションのうち、移動先ディレクトリに同名のファイルやディレクトリがある場合は、上書きするかどうか確認するオプションは？

-i

cpコマンドのオプションのうち、ディレクトリをコピーするオプションは？ (小文字の方から先に2つ)

-r / -R

ファイルやディレクトリを移動、または名前を変更するコマンドは？

mv

ファイルやディレクトリを削除するコマンドは？

rm

rmコマンドのオプションのうち、ディレクトリ全体を再帰的に削除するオプションは？ (小文字の方から先に2つ)

-r / -R

touchコマンドのオプションのうち、アクセス時刻と修正時刻を指定した時刻に変更するオプションは？

-t

lsコマンドのオプションのうち、隠しファイルも含めて全て表示するオプションは？

-a

ディレクトリ内のファイルやディレクトリを一覧表示するコマンドは？

ls

rmコマンドやtouchコマンドのオプションのうち、オプションリストを終了し、これ以降の引数をオプション以外のものとして扱うものは？ (記号2文字)

--

新規ディレクトリを作成できるコマンドは？

mkdir

cpコマンドのオプションのうち、コピー先に既存のファイルがあり上書きできない場合はそのファイルを削除してからコピーするオプションは？

-f

lsコマンドのオプションのうち、サブディレクトリの内容も再帰的に表示するオプションは？

-R

💡 知识点总结

深入掌握文件管理基础命令（`cp`，`mv`，`rm`，`touch`，`ls`，`mkdir`）的高级选项，特别是涉及属性保留、递归处理及强制操作的参数。

🔍 选项解析与详细说明

这一组题目进一步挖掘了文件操作的细节，特别是对“元数据（属性/时间）”的处理：

- 属性与时间维护:
 - `cp -p` : `preserve`。复制时保留文件的所有者、所属组、权限和时间戳（タイムスタンプ）。
 - `touch -t` : `time`。允许使用 `[[CC]YY]MMDDhhmm[.ss]` 格式指定特定的时刻。
 - `touch` (无参数): 如果文件已存在，则将时间更新为**当前时刻**；如果不存在，则创建一个**0字节**的空文件。
- 递归操作的字母陷阱:
 - 对于 `cp` 和 `rm`，`-r` 和 `-R` (Recursive) 通常是通用的。
 - 但对于 `ls`，查看子目录内容必须使用大写的 `-R`（小写 `-r` 是逆序排列的意思）。
- 强制与安全:
 - `-f` (**force**): 在 `cp` 中，如果无法覆盖则先删除旧文件再复制；在 `mv` 和 `rm` 中则是直接无提示操作。
- 特殊符号 `--` :
 - 用于告诉命令“选项到此为止”。这在处理以横杠 `-` 开头的文件名时非常有用，例如 `rm - -filename`。

📊 核心选项对比表

命令	递归处理 (Recursive)	强制处理 (Force)	交互确认 (Interactive)
cp	-r / -R	-f	-i
rm	-r / -R	-f	-i
mv	(无需递归)	-f	-i
ls	-R (仅大写)	-	-

🧠 小学生级记忆法

1. `cp -p` (Preserve):

- 想象成 “Perfect copy”，不但内容要，属性也要一模一样。

2. `mkdir -m` (Mode):

- 创建时直接给它戴个 “Mode（权限模式）” 帽子。

3. `--` (双横杠):

- 它是命令的 “终点线”，跑过这条线的所有东西都当做普通文件名。

4. `ls -R` (大写的递归):

- 想象 `ls` 的递归工作量很大，所以要用大写的 R。

考试小秘籍

a. `touch` 后的文件大小:

- 考试常识: `touch` 一个不存在的文件，它的大小永远是 0。

b. `ls` 的递归大小写:

- 极其重要的考点! `ls -R` 是递归，`ls -r` 是 reverse（倒序）。

c. `cp` 的属性保留:

- 题目问 「属性を保持（保持属性）」，首选是 `-p`。

d. 创建目录并设权限:

- 如果题目要求一步到位创建权限为 700 的目录，一定要用 `mkdir -m 700`，而不是先创建再 `chmod`。

「ls -l」コマンドでシンボリックリンクのパーミッションの先頭に表示される文字は?

l

「dir」ディレクトリを、bzip2形式で圧縮した「dir.tar.bz2」というアーカイブにしたい。適切なコマンドは? (オプションは、「-」を省略しアルファベット順にまとめて指定)

`tar cfj dir.tar.bz2 dir`

「/home/test/file」ファイルのシンボリックリンクを「/export/file」というファイル名で作成したい。適切なコマンドは? なお、「/home」と「/export」は異なったファイルシステムである。

`ln -s /home/test/file /export/file`

「file.gz」ファイルを展開するコマンドは? (アルファベット順に2つ)

`gunzip file.gz / gzip -d file.gz`

ddコマンドのオプションのうち、入力ファイルから出力ファイルへブロックをコピーする回数を指定するオプションは?

count

「file.xz」 ファイルを展開するコマンドは？（アルファベット順に2つ）

unxz file.xz / xz -d file.xz

「file」 ファイルをxz形式で圧縮したい。適切なコマンドは？

xz file

lsコマンドのオプションのうち、inode番号を表示するオプションは？

-i

「file」 ファイルをgzip形式で「file.gz」というファイルに圧縮したい。また、圧縮前の元ファイルも残したい。適切なコマンドは？（リダイレクト「>」を使用して答えよ）

gzip -c file > file.gz

「file」 ファイルをbzip2形式で「file.bz2」というファイルに圧縮したい。また、圧縮前の元ファイルも残したい。適切なコマンドは？（リダイレクト「>」を使用して答えよ）

bzip2 -c file > file.bz2

💡 知识点总结

- **リンク (Link):** シンボリックリンク (Symbolic Link) 相当于快捷方式，首字母标识为 `l`，可以跨文件系统。
- **アーカイブ (Archive):** `tar` 是归档工具。`j` 对应 bzip2，`z` 对应 gzip，`J` 对应 xz。
- **圧縮と展開:** 压缩工具默认会删除原文件。若要保留原文件，需使用 `-c` 选项配合重定向 `>`。
- **dd コマンド:** 用于按块复制文件或设备，`count` 控制复制的次数。

🔍 选项解析与详细说明

3. 链接的奥秘

- `ln -s`: 创建符号链接（シンボリックリンク）。题目提到「**異なったファイルシステム（不同文件系统）**」，这是在暗示只能用符号链接，因为硬链接（ハードリンク）不能跨分区。
- `ls -li`: 重点在于查看 **inode**。硬链接共享同一个 inode，而符号链接有自己独立的 inode。

2. 压缩工具全家桶

- `gzip` / `gunzip`: 最常用。扩展名 `.gz`。
- `bzip2` / `bunzip2`: 压缩率比 gzip 高。扩展名 `.bz2`。
- `xz` / `unxz`: 现代压缩率之王。扩展名 `.xz`。
- **共通点:** 展开时都可以用「工具名 `-d`」（decompress）。

3. Tar 归档魔法

- `tar cfj` :
 - `c` : create (创建)
 - `f` : file (指定文件名)
 - `j` : 使用 **bzip2** 压缩。
 - **注意**: 这里的选项字母顺序按题目要求是 `cfj` (按字母表 a-z 顺序)。

压缩格式对比表

格式	压缩命令	展开命令 (简写)	展开命令 (通用)	Tar 对应选项
gzip	gzip	gunzip	gzip -d	z
bzip2	bzip2	bunzip2	bzip2 -d	j
xz	xz	unxz	xz -d	J (大写)

小学生级记忆法

1. `-c` 选项 (Keep the file):
 - 压缩工具默认很“贪吃”，压完就把原文件“吃了”（删除）。
 - 给它加个 `-c` (Console/Cat)，让它把结果吐到屏幕上，我们再用 `>` 把它接住存起来，原文件就保住了。
2. Tar 选项记忆:
 - `z` 像 gzip 的尾巴。
 - `j` 对应 bzip2 (bzip2)。
 - `J` (大写) 对应最新的 xz。
3. dd 的 count:
 - 就像数数：1次、2次、3次... 对应的就是 **count**。

考试小秘籍

- **重定向保留原件**: 题目问“元ファイルも残したい（也想留下原文件）”时，一定要反射性想到 `-c` 和 `>`。
- **dd 常用三兄弟**: `if` (输入文件), `of` (输出文件), `bs` (块大小), `count` (数量)。

- **tar 选项顺序:** 考试有时对选项顺序有严格要求（如字母序），虽然平时练习 `tar c j f` 也可以，但按题意写 `c f j` 最稳。

file1のハードリンクをfile2として作成したい。適切なコマンドは？

```
ln file1 file2
```

「aa」「bb」「cc」の3ファイルを1つにまとめて「hoge.tar」というアーカイブにしたい。適切なコマンドは？（オプションは、「-」を省略しアルファベット順にまとめて指定）

```
tar cf hoge.tar aa bb cc
```

xz形式で圧縮したアーカイブファイル「dir.tar.xz」をカレントディレクトリに展開したい。適切なコマンドは？（オプションは、「-」を省略しアルファベット順にまとめて指定）

```
tar fJx dir.tar.xz
```

xz形式の圧縮ファイル「file.xz」の情報を確認したい。適切なコマンドは？

```
xz -l file.xz
```

現在のディレクトリにあるworkdirをxzで圧縮されたtarアーカイブ（ファイル名：foo.tar.xz）にしたい。適切なコマンドは？（オプションは、「-」を省略しアルファベット順にまとめて指定）

```
tar cfJ foo.tar.xz workdir
```

「file.bz2」ファイルを展開するコマンドは？（アルファベット順に2つ）

```
bunzip2 file.bz2 / bzip2 -d file.bz2
```

file1のシンボリックリンクをfile2として作成したい。適切なコマンドは？

```
ln -s file1 file2
```

「file」ファイルをbzip2形式で圧縮したい。適切なコマンドは？

```
bzip2 file
```

「dir」ディレクトリ内の全てのファイルをそれぞれgzip形式で圧縮したい。適切なコマンドは？

```
gzip -r dir
```

「boot.img」というイメージファイルをUSBメモリ（/dev/sdc）にコピーしたい。適切なコマンドは？

```
dd if=boot.img of=/dev/sdc
```

💡 知识点总结

- **链接 (Link):** `ln` 默认创建 **ハードリンク (Hard Link)**，加 `-s` 则是 **シンボリックリンク (Symbolic Link)**。
- **tar 选项字母:**

- `z` → gzip
 - `j` → bzip2
 - `J` (大写) → xz
 - **dd 命令参数**: `if` (input file/输入文件), `of` (output file/输出文件)。常用于制作启动盘或备份分区。
 - **目录压缩**: `gzip -r` 会递归地对目录下的**每个文件**分别进行压缩。
-

选项解析与详细说明

4. 链接的区别

- `ln file1 file2`: file2 和 file1 指向同一个 inode。如果删除原文件，数据依然可以通过链接访问。
- `ln -s file1 file2`: file2 只是一个路径指向。如果原文件被删或移动，链接就会失效（断开）。

2. tar 的“字母组合拳”

- **创建与解压**: `c` (Create/创建), `x` (eXtract/解压)。
- **指定文件**: `f` (File) 必须紧跟在文件名之前，但按照题目要求的“字母序”，位置会有所变动（如 `cfJ` 或 `fJx`）。
- **xz 格式**: 一定要记住是 **大写的 J**。

3. 查看压缩包信息

- `xz -l`: list。可以不解压就看到压缩包文件的大小、压缩率等 **情報（信息）**。

4. 磁盘克隆 dd

- `dd if=boot.img of=/dev/sdc`: 这是向物理设备写入数据的标准方法。`if` 是源，`of` 是目的地。
-

tar 常用选项与字母序参考

需求 (日本語)	主操作	压缩格式	选项字母序 (题目要求)
アーカイブ作成 (xz)	c	J	cfJ
アーカイブ展開 (xz)	x	J	fJx
アーカイブ作成 (无压缩)	c	-	cf

🧠 小学生级记忆法

1. xz 的 J 记忆：

- 字母 `x` 的发音最后有点像 `z`，而字母 `J` 长得像个钩子，能把最大的压缩率（xz）钩住。

2. ln -s (Symbolic)：

- `s` 就是 soft（软链接）或者是 shortcut（快捷方式）。

3. dd 的 if 和 of：

- `if = in` (进去的内容)
- `of = out` (出来的目的地)

📝 考试小秘籍

- 字母顺序坑：题目明确要求「アルファベット順（字母顺序）」。
 - 比如 `fJx`：`f` (6), `J` (10), `x` (24)。
 - 比如 `c f J`：`c` (3), `f` (6), `J` (10)。
- `gzip -r` 的陷阱：
 - 注意！`gzip -r dir` 不会把目录变成一个压缩包文件，而是把目录里的文件 一个个单独 压成 `.gz`。要打包必须用 `tar`。
- `dd` 的权限：
 - 写入 `/dev/sdc` 这种设备文件通常需要 root 权限（虽然题目只考命令本身）。

ファイルからデバイス、デバイスからファイル、またデバイス間でのファイルのコピーを行うことができるコマンドは？

dd

下記説明に該当する圧縮形式を答えよ

- bzip2より圧縮率がよい
- Linuxカーネルソースの配布形式に使われている

xz

gzip形式で圧縮したアーカイブファイル「test.tar.gz」の内容を展開せずに表示したい。下線部にあてはまる適切なgzipコマンドは？（オプションはアルファベット順にまとめて指定）

_____ test.tar.gz | tar ftv -

gzip -cd test.tar.gz | tar ftv -

lsコマンドのオプションのうち、ハードリンクの数を確認できるオプションは？

-l

ddコマンドのオプションのうち、入出力のブロックサイズ(一度に読み書きするバイト数)を指定するオプションは?

bs

「dir.tar」というアーカイブの内容を表示できるコマンドは? なお、アーカイブは展開せず、詳細情報を表示すること。(オプションは、「-」を省略しアルファベット順にまとめて指定)

tar ftv dir.tar

サイズを指定してファイルを作成できるコマンドは?

dd

gzip形式で圧縮したアーカイブファイル「dir.tar.gz」をカレントディレクトリに展開したい。適切なコマンドは? (オプションは、「-」を省略しアルファベット順にまとめて指定)

tar fxz dir.tar.gz

同じファイルの実体を指す全てのハードリンクは何の番号が同じか? (アルファベットの小文字のみで)

inode

gzip形式で圧縮したアーカイブファイル「test.tar.gz」の内容を展開せずに表示したい。また、処理の詳細情報も表示したい。下線部にあてはまる適切なtarコマンドは? (オプションは、「-」を省略しアルファベット順にまとめて指定)

_____ test.tar.gz

tar ftvz **test.tar.gz**

「/home/dir」ディレクトリのシンボリックリンクを「/link」という名で作成したい。適切なコマンドは?

ln -s /home/dir /link

「file」ファイルをxz形式で「file.xz」というファイルに圧縮したい。また、圧縮前の元ファイルも残したい。適切なコマンドは? (リダイレクト「>」を使用せずに答えよ)

xz -k file

「file」ファイルをgzip形式で圧縮したい。適切なコマンドは?

gzip file

「hoge.tar」というアーカイブを展開したい。適切なコマンドは? (オプションは、「-」を省略しアルファベット順にまとめて指定)

tar fx hoge.tar

 知识点总结

- **dd 的全能性**：它不仅仅是复制文件，由于它是在 **块 (Block)** 级别操作，所以能处理设备文件（如硬盘、U盘、光盘镜像）。
 - **xz 格式**：目前 Linux 内核 (Kernel) 源码的标准分发格式，压缩率极高，选项通常与 **gzip** 类似（如 **-k** 保留原文件）。
 - **tar 预览**：**t** 选项用于 **list**（查看列表），通常配合 **v**（显示详细信息）和 **f**（指定文件）。
 - **保留原文件**：除了重定向，**xz** 支持专用的 **-k** (**keep**) 选项。
-

选项解析与详细说明

1. dd 的关键参数

- **bs** : block size。指定一次读写的大小。如果想加快速度，通常设置为 **bs=4M**。
- **文件到设备**：**dd** 是 Linux 下制作启动盘的常用手段，因为它能跨越文件系统直接写入设备。

5. 不解压预览内容

- **tar ftv** :
 - **t** : 查看内容。
 - **v** : 详细信息（类似 **ls -l**）。
 - **f** : 指定文件。
 - **tar ftvz** : 针对 **.tar.gz** 增加 **z** 选项直接查看。
- **gzip -cd** :
 - **-c** : 将结果输出到标准输出 (stdout)。
 - **-d** : 解压。
 - 这两个组合在一起可以把压缩包的内容“流”式传输给下一个命令（如 **tar**）。

3. 硬链接的灵魂：inode

- 多个硬链接在磁盘上其实是同一个“实体的分身”，它们的 **inode** 号码完全一致。通过 **ls -l** 可以看到硬链接的数量。
-

归档/压缩进阶操作表

操作需求	对应命令/选项	核心逻辑
创建特定大小文件	dd	使用 bs 和 count 组合
保留原文件 (xz)	xz -k	keep (保持)
预览 tar.gz 内容	tar ftvz	t = List, v = Verbose, z = gzip
通过管道预览内容	***`gzip -cd ...	tar ftv -`**
跨文件系统链接	ln -s	硬链接不可跨分区，软链接可以

🧠 小学生级记忆法

1. `dd` 是“搬砖工”：
 - 它不管文件是什么，只管一块一块（`bs`）地搬，还能数数搬了多少块（`count`）。
2. `tar` 的 `t` 选项：
 - 把 `t` 想成 Table of contents（目录），所以它只看目录，不拆箱（不解压）。
3. `xz -k` (Keep):
 - `xz` 比较有礼貌，加个 `-k` 就会把原来的文件 `keep` 住，不删掉。
4. `ls -l` 查链接：
 - 在 `ls -l` 输出的权限位后面，紧跟着的那个数字就是 **硬件链接数**。

📝 考试小秘籍

- 管道符号 `-` 的意义：
 - 在 `tar ftv -` 中，最后的那个横杠 `-` 非常关键，它告诉 `tar`：“请从上一个命令传过来的管道里读取数据”。
- 字母顺序 (Alphabetical Order):
 - 题目多次要求选项按字母顺序排列。
 - 例如：`fxz` (f=6, x=24, z=26)、`ftv` (f=6, t=20, v=22)。
- 内核源码格式：
 - 只要看到关键词「カーネルソース（内核源码）」，答案几乎锁定是 `xz`。

findコマンドの検索式で、ファイルの最終更新時刻を条件として検索するものは？

-mtime

指定したコマンドが格納されている場所を、環境変数PATHで設定しているディレクトリから検索し、その場所を絶対パスで表示するコマンドは？

which

指定したディレクトリ以下からファイルやディレクトリを検索するコマンドは？（データベースを使用しないものとする）

find

指定したコマンド、またそのマニュアル、ソースコードが格納されている場所を検索し、絶対パスで表示するコマンドは？

whereis

Linuxの標準的なディレクトリの構造を定めた規格のことを何というか？（アルファベットの大文字3文字で）

FHS

findコマンドでカレントディレクトリ以下から、アクセス権が775のディレクトリを検索したい。適切なコマンドは？（検索起点ディレクトリは省略しない。検索式はファイルの種類から先に指定する事。アクションは省略）

find . -type d -perm 775

locateコマンドが利用する検索用のデータベースを更新するコマンドは？

updatedb

全てのユーザが使用できるプログラムを、バイナリ形式の実行ファイルとして作成した。そのプログラムはこのホストでのみ使用する。バイナリを格納するディレクトリとして適切なものは？

/usr/local/bin/

findコマンドの検索式で、ファイルのサイズを条件として検索するものは？

-size

findコマンドで検索したファイルに対してコマンドを実行したい。適切なアクションは？

-exec

全てのユーザが使用できるプログラムを作成した。そのプログラムはこのホストでのみ使用する。プログラムの設定ファイルを格納するディレクトリとして適切なものは？

/usr/local/etc/

findコマンドの検索式で、ファイルの所有者を条件として検索するものは？

-user

マニュアルページが格納されているディレクトリは？

/usr/share/man/

findコマンドの検索式で、指定した階層までを検索対象とするものは？

-maxdepth

findコマンドによる検索結果をNULL文字区切りで得るアクションは？

-print0

💡 知识点总结

- 查找工具三剑客：

- `find` : 实时、深度搜索。功能最强，支持时间、权限、大小等多种条件。
- `locate` : 基于数据库。速度极快，但需要更新数据库（`updatedb`）。
- `which` / `whereis` : 专门查找命令（二进制文件、源码、手册）。

- FHS (Filesystem Hierarchy Standard):

- `/usr/local/` : 存放**本主机特有**（该主机独用）且**手动安装**的软件（及其配置、可执行文件）。
- `/usr/share/man/` : 标准的手册页（Manual）存放地。

🔍 选项解析与详细说明

1. `find` 命令的搜索条件（表达式）

- `-mtime` : **m**odification **t**ime。根据最后修改时间查找。
- `-size` : 根据文件大小查找。
- `-user` : 根据文件的所有者查找。
- `-perm` : **p**ermission。根据权限查找（如 `775`）。
- `-type` : 指定类型（`f` 为文件，`d` 为目录）。
- `-maxdepth` : 限制搜索的目录深度，防止递归太深。

2. `find` 的高级动作（Action）

- `-exec` : 对找到的每一个文件执行指定的命令。
- `-print0` : 将结果以 **NULL** 字符分隔。这在处理带有空格的文件名时非常有用，常配合 `xargs` `-0` 使用。

4. 命令定位工具对比

- `which` : 只在 `$PATH` 中找可执行文件。
- `whereis` : 除了二进制文件，还会找 **ソース (Source)** 和 **マニュアル (Manual)**。



查找命令功能对照表

工具	搜索方式	优势	局限
find	实时扫描硬盘	条件极多，非常精准	速度相对较慢
locate	检索数据库	速度极快	无法搜到新创建的文件 (需 updatedb)
which	搜索 \$PATH	快速找到命令路径	只能找可执行文件
whereis	搜索特定目录	一次性找齐源码和手册	搜索范围有限



小学生级记忆法

1. which vs whereis :

- which (哪一个) 是问 “哪一个命令会被执行”。
- where is (在哪里) 是问 “这个命令的所有东西（源码、手册）都在哪里”。

2. updatedb (更新数据库):

- locate 像是在查字典，如果新词（新文件）没写进去，就查不到。所以要用 **update** 手段更新这本字典。

3. /usr/local/ (本地独有):

- 只要看到关键词 「このホストでのみ (仅限此主机)」，路径里一定有 **local**。

4. -mtime (Modification Time):

- m 表示修改，time 表示时间。



考试小秘籍

◦ find 的参数顺序:

- 题目要求 「検索式はファイルの種類から先に（搜索表达式先指定文件类型）」。因此必须先写 **-type d** 再写 **-perm 775**。

◦ FHS 的缩写:

- 题目要求大写 3 字母，务必写成 **FHS**。

◦ /usr/local/ 结尾的斜杠:

- 询问目录时，通常习惯在结尾加上 **/**，如 **/usr/local/bin/**。

- **NULL 分隔符：**

- 只要看到「NULL文字区切り」，脑子里立刻反应出 `-print0`（那个是零，不是大写字母 O）。

findコマンドでファイルを検索する際、検索結果を改行区切りで表示するアクションは？

`-print`

指定したコマンドが通常のコマンドか、シェルの組み込みコマンドか、エイリアスかなどの情報を表示するコマンドは？

`type`

検索用データベースを使用して高速でファイル検索を行うコマンドは？

`locate`

findコマンドで「httpd.conf」ファイルを「/etc」ディレクトリ以下から検索したい。適切なコマンドは（アクションは省略）？

`find /etc -name httpd.conf`

findコマンドの検索式で、ファイルの種類を条件として検索するものは？

`-type`

findコマンドの検索式で、指定した階層から下を検索対象とするものは？

`-mindepth`

システムの起動には必須ではないコマンドの共有ライブラリが格納されているディレクトリは？

`/usr/lib/`

updatedbコマンドの設定ファイルは？（絶対パスで回答）

`/etc/updatedb.conf`

cpコマンドの場所を絶対パスで表示したい。適切なコマンドは？（コマンド全体が短い方から先に2つ）

`which cp / whereis cp`

カレントディレクトリ以下の全てのファイルについて、それぞれの単語数を表示したい。下線部にあてはまるコマンドとアクションは？

`_____ -type f _____ wc -w {} \;`

`find -type f -exec wc -w {} \;`

findコマンドの検索式で、アクセス権を条件として検索するものは？

`-perm`

findコマンドの検索式で、シンボリックリンクを検索するものは？

-type l

システムの起動には必須ではない、一般ユーザ用のコマンドが格納されているディレクトリは？

/usr/bin/

findコマンドの検索式で、ファイルの最終アクセス日を条件として検索するものは？

-atime

findコマンドの検索式で、ファイル名を条件として検索するものは？

-name

findコマンドで「/home」ディレクトリ以下から、所有者が「test」ユーザのファイルやディレクトリを検索したい。適切なコマンドは？（アクションは省略）

find /home -user test

システムの起動には必須ではない、システム管理用のコマンドが格納されているディレクトリは？

/usr/bin/

💡 知识点总结

- **find** 的时间参数：
 - **-atime** : access **time**（访问时间）。
 - **-mtime** : modification **time**（修改时间）。
- 目录分工 (FHS):
 - **/usr** : 存放**非启动必需**（システムの起動には必須ではない）的资源。
 - **/usr/bin** : 一般用户命令。
 - **/usr/sbin** : 系统管理命令（管理员用）。
 - **/usr/lib** : 程序运行所需的库文件。
- 命令属性查询：
 - **type** : 用于区分命令是 **Alias**（别名）、**Built-in**（内置）还是 **File**（外部文件）。

🔍 选项解析与详细说明

1. **find** 的核心用法

- **-print** : **find** 的默认动作。如果你不写任何动作，它会自动执行 **-print**，将结果以**换行符**分隔输出。
- **-exec ... {} \;** :

- `{}` : 代表 `find` 找到的每一个文件名。
- `\;` : 代表命令的结束。
- 层级限制:
 - `-mindepth` : 从哪一层以下开始找。
 - `-maxdepth` : 到哪一层以上停止找。

5. 定位与数据库

- `locate` : 利用索引数据库进行“秒搜”，比 `find` 快得多。
- `/etc/updatedb.conf` : 规定了 `updatedb` 哪些目录该搜，哪些目录（如 `/tmp` 或网络挂载点）该排除。

3. FHS 目录结构

- `/bin` vs `/usr/bin` :
 - `/bin` 里的命令是救命用的（单用户模式下也要有）。
 - `/usr/bin` 里的命令是平时工作用的（如 `gcc` , `python`）。

`find` 搜索条件 (Expression) 速查表

搜索条件 (日本語)	选项	记忆点
ファイル名	<code>-name</code>	Name
所有者	<code>-user</code>	User
アクセス権	<code>-perm</code>	Permission
ファイル種別	<code>-type</code>	f (file), d (dir), l (link)
アクセス日	<code>-atime</code>	Access Time
最小階層	<code>-mindepth</code>	Minimum Depth

小学生级记忆法

1. `type` 命令 (你是哪一类?):
 - 当你想问一个命令：“你到底是亲生的（内置），还是外来的（外部文件），还是绰号（别名）？”就用 `type`。
2. `-atime` vs `-mtime` :
 - Atime = Access (看了它)。

- Mtime = Modify (改了它)。

3. `/usr` 目录的含义：

- 把 `/usr` 想成是 **U**ser **S**ystem **R**esources。系统启动不需要它，但用户干活离不开它。

考试小秘籍

a. `-exec` 的固定格式：

- 考试填空时，千万别漏掉空格！`{ } \;` 之间有空格，`\` 后面紧跟分号。

b. `which` vs `whereis` 的长度：

- 题目要求“短い方から（短的在前）”：`which` (5字) < `whereis` (7字)。

c. `l` 字母的含义：

- 在 `-type l` 中，`l` 代表 **S**ymbolic **L**ink（符号链接）。

d. `/usr/sbin` 的特殊性：

- `s` 代表 **S**uperuser 或 **S**ystem。看到“系统管理用”且“非启动必需”，首选 `/usr/sbin/`。

Linuxのインストールと仮想マシン・コンテナの利用(主題1.01)

サーバ「server」にユーザ「user1」としてSSH接続するコマンドは？（オプション無し）

ssh user1@server

SSHサーバからログアウトするコマンドは？（アルファベット順に2つ）

exit/logout

SSHクライアントの「`~/.ssh/`」ディレクトリにあるSSH接続先のホスト名やIPアドレス、公開鍵が格納されるファイル名は？

known_hosts

暗号化を用いた、リモートホストの遠隔操作作用のプロトコルは？

SSH

OpenSSHにおいてSSHサーバの公開鍵/秘密鍵が保存されるディレクトリは？

/etc/ssh/

OpenSSHにおけるSSHクライアントの設定ファイルは？（絶対パスで）

etc/ssh/ssh_config

SSHサーバの「`~/.ssh/`」ディレクトリにあるSSH接続ユーザの公開鍵が格納されるファイル名は？

authorized_keys

SSHサーバからログアウトするキー操作で、Ctrlキーと同時に押すキーは？（アルファベット1文字）

D

OpenSSHにおけるSSHサーバの設定ファイルは？（絶対パスで）

etc/ssh/sshd_config

SSHのユーザ認証に使用する公開鍵と秘密鍵のペアを作成するコマンドは？

ssh-keygen

ssh-keygenコマンド実行時にユーザのホームディレクトリに作成され、公開鍵と秘密鍵が格納されるディレクトリは？

.ssh/

プロセスIDが10のプロセスをクリーンアップせずに強制的に終了させたい。適切なコマンドは？（-s オプションを併用し、シグナル名は略さずに大文字で）

kill -s SIGKILL 10

実行中の全てのbashプロセスのプロセスIDを表示したい。実行するコマンドは？（パイプを使わず1コマンドで）

pgrep bash

端末の切断時にプロセスを終了する、または設定ファイルを再度読み込ませるシグナル名は？（省略した形、大文字で）

HUP

💡 知识点总结

- **SSH 安全机制**：基于公钥加密技术。客户端保存服务器的公钥（`known_hosts`），服务器保存客户端的公钥（`authorized_keys`）。
- **OpenSSH 配置**：遵循“`d` 代表 Daemon（守护进程）”的命名规则。
- **进程信号 (Signals)**：Linux 通过信号来控制进程。`SIGKILL` (9) 是最强杀招，`SIGHUP` (1) 则是优雅的重启或通知。

🔍 选项解析与详细说明

4. SSH 密钥对与文件分布

- `ssh-keygen`：生成 **秘密鍵 (Private Key)** 和 **公開鍵 (Public Key)**。默认放在 `~/.ssh/`。
- `known_hosts`：存放在 **客户端**。当你第一次连接某服务器时，它的“指纹”会记在这里。
- `authorized_keys`：存放在 **服务器端**。你想免密登录哪台机器，就把你的公钥塞进那台机器的这个文件里。

2. SSH 配置文件

- `/etc/ssh/ssh_config`: 客户端（你连接别人时）的全局配置。
- `/etc/ssh/sshd_config`: 服务端（别人连接你时）的守护进程配置。「d」是核心区别。

3. 进程控制与信号

- `SIGKILL` (9): 强制终止，进程无法忽略，也不会进行任何清理工作（如保存数据）。
- `SIGHUP` (1): Hang UP。传统上是终端断开信号，现在常用于让服务「設定ファイルを再度読み込ませる（重新读取配置）」。
- `pgrep`: 专门根据进程名来查找进程 ID (PID) 的利器，比 `ps | grep` 更简洁。

SSH 文件与信号对照表

分类	名称	存放位置/数值	作用
客户端配置	ssh_config	/etc/ssh/	连接外面的设置
服务器配置	sshd_config	/etc/ssh/	接受连接的设置
连接过的主机	known_hosts	~/.ssh/	验证服务器身份
授权的公钥	authorized_keys	~/.ssh/	验证客户端身份
强制杀进程	SIGKILL	9	不清理，直接杀
重新加载	HUP	1	重读配置/终端断开

小学生级记忆法

1. `authorized_keys` (授权钥匙):
 - 想象你要进服务器家门，服务器得把你的公钥放在它的“白名单”里，这个名单就是 `authorized`（授权过的）。
2. `sshd_config` 的 `d`:
 - `d` = Daemon（大魔王/守护进程）。大魔王在服务器上守着门，所以服务器配置带 `d`。
3. `SIGKILL` (杀无赦):
 - 它是信号里的核武器，不管进程在干嘛，立刻物理消失。
4. `Ctrl + D` (Done):
 - 做完了（Done），就退出。



考试小秘籍

a. `kill` 命令的信号指定：

- 题目要求「シグナル名は略せずに大文字で（信号名不缩写且大写）」：务必写 `SIGKILL` 而不是 `KILL` 或 `9`。

b. 配置文件绝对路径：

- `/etc/ssh/` 路径中的斜杠不要写错。注意题目要求「絶対パス」。

c. SSH 目录权限 (常识)：

- 考试有时会问权限。`~/.ssh/` 目录通常必须是 `700`，`authorized_keys` 必须是 `600`，否则 SSH 会因为不安全而拒绝连接。

プロセスの稼働状況を一定時間ごとに更新して表示するコマンドは？

top

動作中のプロセスの親子関係をつリー状に表示できるコマンドは？（オプション無しでツリー状に表示できるコマンドとする）

pstree

起動中のコンテナでコマンドを実行する Docker のコマンドラインは？ docker _____

exec

virsh コマンドのサブコマンドで、XML 形式のファイルから仮想マシンを作成し新規の仮想マシンを起動するものは？

create

virsh コマンドのサブコマンドで、一時停止中の仮想マシンを再開するものは？

resume

コンテナの一覧を表示する Docker のコマンドラインは？ docker _____

ps

Docker Hub からイメージをダウンロードする Docker のコマンドラインは？ docker _____

pull

virsh コマンドのサブコマンドで、仮想マシンを起動するものは？

start

コンテナを強制的に停止する Docker のコマンドラインは？ docker _____

kill

Dockerイメージ「ubuntu」を使用してコンテナを作成・起動して、コンテナにログインしたい。以下のコマンドの空欄に入るオプションは？（アルファベット順に2つ）# docker run -() ubuntu

it

virshコマンドのサブコマンドで、仮想マシンのコンソールに接続するものは？

console

Docker Hubにイメージをアップロードする Dockerのコマンドラインは？ docker _____

push

Linuxに標準搭載されているカーネルベースの仮想化ソリューションは？

KVM

virshコマンドのサブコマンドで、仮想マシンの一覧を表示するものは？

list

virshコマンドのサブコマンドで、仮想マシンを一時停止するものは？

suspend

这组题目将传统的**进程管理（Process Management）与现代的虚拟化（KVM/virsh）及容器技术（Docker）**结合在了一起。这是 Linux 运维从物理机迈向云端的必经之路。

知识点总结

- **进程状态监控**： `top` 是动态监控的标配， `pstree` 则是理清进程血缘关系（父子关系）的利器。
- **虚拟化管理 (KVM/virsh)**： KVM 是 Linux 的“亲儿子”虚拟化方案。通过 `virsh` 命令行工具，可以实现对虚拟机的全生命周期管理（启动、停止、暂停、恢复）。
- **容器技术 (Docker)**： 容器是轻量化的虚拟化。Docker 的命令逻辑非常像 Linux 原生命令（如 `ps`、`kill`、`exec`）。

选项解析与详细说明

4. 传统的进程视图

- `top` : Table Of Processes。它会实时刷新，让你看到哪个进程在“吃” CPU 或内存。
- `pstree` : 以树状结构显示。通过它你可以看到 `systemd` 是如何开启所有子进程的。

2. 虚拟机管理 (virsh)

- `create` : 根据 XML 配置文件的定义，**创建并启动**一个新的虚拟机。

- `suspend` vs `resume` :
 - `suspend` 将虚拟机挂起（暂停到内存）。
 - `resume` 让挂起的机器起死回生。
- `list` : 查看当前运行中的虚拟机状态。

3. 容器魔术 (Docker)

- `docker run -it` :
 - `-i` : interactive（交互式）。
 - `-t` : tty（分配终端）。
 - 想进入 Ubuntu 容器“聊天”，这两个选项是金牌搭档。
- `docker pull/push` : 像 Git 一样，从仓库（Docker Hub）拉取或上传镜像。
- `docker exec` : 在已经跑起来的容器里“加戏”，执行额外的命令。

虚拟化 vs 容器命令对照表

操作功能 (日本語)	virsh (虚拟机)	Docker (容器)
表示列表	<code>list</code>	<code>ps</code>
启动/运行	<code>start</code>	<code>run</code>
强制停止	<code>destroy</code>	<code>kill</code>
进入/连接	<code>console</code>	<code>exec / attach</code>
创建新实例	<code>create</code>	<code>run</code>

小学生级记忆法

1. `suspend` 与 `resume` :
 - 把虚拟机看成在听音乐。`suspend` 是按下了“暂停键”，`resume` 是按下了“继续播放”。
2. Docker 的 `ps` :
 - Linux 查进程叫 `ps`，Docker 查“跑着的容器”也叫 `ps`（容器本质上就是一个被隔离的进程）。
3. `pull` 与 `push` :
 - **Pull** (拉) 过来用，**Push** (推) 出去给别人用。

4. `virsh console` :

- 想象你要去控制台（Console）前坐下办公，所以连接虚拟机终端用 `console`。

考试小秘籍

a. Docker `run` vs `exec` :

- `run` 是从无到有（启动新容器）。
- `exec` 是对已有容器操作（进入运行中的容器）。

b. `virsh create` vs `define` :

- 考试中常考 `create`，它会立即启动机器。而 `define` 只是注册配置，不启动。

c. `-it` 顺序:

- 虽然技术上 `-ti` 也可以，但题目要求「アルファベット順（字母顺序）」，所以必须写 `it`。

d. KVM 的地位:

- 看到「標準搭載（标准搭载）」和「カーネルベース（基于内核）」，不要犹豫，答案就是 `KVM`。

virt-managerやvirshコマンドなどが利用する、オープンソースの仮想化管理用のライブラリは？

libvirt

virshコマンドのサブコマンドで、仮想マシンの定義ファイルをXML形式で出力するものは？

dumpxml

Dockerレジストリにログインする Dockerのコマンドラインは？ docker _____

login

コンテナの標準入出力に接続する Dockerのコマンドラインは？ docker _____

attach

virshコマンドのサブコマンドで、仮想マシンを強制終了するものは？

attach

既存のコンテナを起動する Dockerのコマンドラインは？ docker _____

start

コンテナを削除する Dockerのコマンドラインは？ docker _____

rm

コンテナを停止する Dockerのコマンドラインは？ docker _____

stop

virsh コマンドのサブコマンドで、仮想マシンを再起動するものは？

reboot

virsh コマンドのサブコマンドで、仮想マシンを正常に終了するものは？

shutdown

Docker イメージの一覧を表示する Docker のコマンドラインは？ docker _____

images

コンテナを作成・起動する Docker のコマンドラインは？ docker _____

run

💡 知识点总结

- **虚拟化管理 (libvirt)**: 这是一个核心 API 库，它像一个“翻译官”，让 `virsh` 或 `virt-manager` 能够统一控制 KVM、QEMU、Xen 等不同的虚拟化技术。
- **XML 配置文件**: 在 `libvirt` 体系中，虚拟机的配置都是以 XML 格式存储的。
- **Docker 生命周期**:
 - `run` = 创建 + 启动。
 - `start` / `stop` / `restart` = 对现有容器的开关操作。
 - `rm` = 彻底删除容器（注意：删除镜像用 `rmi`）。

🔍 选项解析与详细说明

4. 虚拟化管理利器：virsh

- `libvirt`: 几乎所有 Linux 虚拟化工具都在底层调用这个库。它是开源虚拟化管理的**标准接口**。
- `dumpxml`: 这是一个非常形象的命令。“Dump”意为“倾倒/导出”。它把虚拟机的配置像倒水一样，以 XML 格式“倒”出来供你查看或备份。
- `destroy`: 虽然名字叫“销毁”，但在 `virsh` 中它实际上代表「**强制終了**」。它相当于直接拔掉虚拟机的电源线（硬关机）。
- `shutdown`: 则是“文明关机”，向系统发送关机信号，让它正常保存数据后退出（软关机）。

2. Docker 容器管理

- `docker run`: 这是一个组合动作。如果你本地没有镜像，它还会自动执行 `pull`。

- `docker attach` : 想象你连接到了一个已经在跑的“黑匣子”（容器）上，去观察它的实时标准输入输出。
- `docker images` : 查看你电脑里存了哪些“光盘”（镜像）。
- `docker rm` : 移除容器。如果容器正在运行，通常需要先 `stop` 或者加 `-f`。

virsh 与 Docker 关机/删除对比表

操作类型	virsh (虚拟机)	Docker (容器)	状态说明
正常关闭	<code>shutdown</code>	<code>stop</code>	给程序时间存盘退出
强制关闭	<code>destroy</code>	<code>kill</code>	直接掐断进程
重新启动	<code>reboot</code>	<code>restart</code>	关了再开
彻底删除	<code>undefine</code>	<code>rm</code>	移除配置/容器实例

小学生级记忆法

1. `libvirt` (Library for Virtualization):
 - 名字里就写着呢：Lib (库) + Virt (虚拟化)。
2. `dumpxml` (导出配置):
 - Dump (导出) + XML (格式)。想看虚拟机的身份证信息，就把它 dump 出来。
3. `attach` (挂载/连接):
 - 想象你拿着一根电缆，Attach (贴/连) 到正在跑的容器排气管上，听它的动静（查看输出）。
4. `rm` (Remove):
 - 就像 Linux 删除文件叫 `rm`，Docker 删除容器也叫 `rm`。

考试小秘籍

- a. `docker run` 与 `docker start` 的区别:
 - 如果题目说「作成・起動」 $\rightarrow$ 选 `run`。
 - 如果题目说「既存のコンテナ」(已存在的容器) $\rightarrow$ 选 `start`。
- b. 强制关机的陷阱:
 - `virsh` 的强制关机是 `destroy`，千万别按字面意思理解成彻底删除（彻底删除是 `undefine`）。
- c. `dumpxml` 的用途:

- 考试中如果问到如何备份虚拟机配置，或者如何将配置迁移到另一台机器，答案往往涉及 `dumpxml`。
-

フォアグラウンドで実行中のジョブを一時停止したい。Ctrlキーと同時にどのキーを押下する必要があるか？

Z

一般的な Linux システムの起動順序（ブートプロセス）は？

BIOS または UEFI（英字）

ブートローダ（カタカナ）

カーネル（カタカナ）

init（英字）

「`/etc/inittab`」を設定ファイルとして使用しないinitプログラムは？（アルファベット順に2つ）

systemd / Upstart

メモリやスワップ領域の使用状況を、一定時間ごとに更新して表示するコマンドは？

top

ジョブ番号が1の一時停止中のジョブをバックグラウンド動作に切り替えたい。適切なコマンドは？

bg 1

フォアグラウンドで実行しているプロセスに割り込みシグナルを送信し、プロセスを終了したい。Ctrlキーと同時にどのキーを押下する必要があるか？

C

systemdが利用できるユニットの一覧を取得するためのsystemctlのサブコマンドは？

list-unit-files

直ちにシステムを停止してコンピュータの電源を切るコマンドは？（アルファベット順に2つ。必要ならオプションを指定する）

halt -p / poweroff

プロセスIDが10のプロセスのみの情報を表示したい。適切なコマンドは？（ハイフン無しで）

ps p 10

systemdが稼働するシステムにおいて、サービス foo を手動で再起動させたい。実行するコマンドは？（サービス名は短い形式で）

systemctl restart foo

psコマンドのオプションのうち、動作中のプロセスの親子関係をツリー状で表示するオプションは？

f

以下のコマンド名で実行されている全てのプロセスを終了したい。適切なコマンドは？

mycom
mycom2
mycom3

pskill mycom

バックグラウンドで動作中または一時停止中のジョブを、フォアグラウンド動作に切り替えるコマンドは？

fg

psコマンドのオプションのうち、親プロセスのPIDや実行優先度を決定するnice値なども併せて表示するオプションは？ (ハイフン無しで)

|

psコマンドのオプションのうち、現在実行中の全てのプロセスを表示するオプションは (アルファベット順に2つ、また複数のオプションを組み合わせる場合もアルファベット順に指定)(「-」で始まるUNIXオプションは、完全なフォーマットでプロセスを表示)

ax / -ef



知识点总结

掌握 Linux 启动流程 (Boot Process)、作业控制 (Job Control)、以及现代系统管理核心工具 `systemd` 与进程查看工具 `ps` 的进阶用法。



选项解析与详细说明

2.1 Linux 启动流程 (Boot Process)

Linux 启动是一个环环相扣的过程：

1. BIOS/UEFI: 硬件自检，读取磁盘第一个扇区。
2. ブートローダ (Boot Loader): 如 GRUB，负责加载内核。
3. カーネル (Kernel): 解压并初始化硬件驱动，挂载根文件系统。
4. init / systemd: 系统启动的第一个进程 (PID 1)，负责启动其他所有服务。

2.2 作业控制 (Job Control)

- `Ctrl + Z`: 暂停 (Suspend) 当前任务，将其丢入后台。
- `Ctrl + C`: 强制中断 (Interrupt) 当前任务。
- `bg` (BackGround): 让后台暂停的任务开始运行。

- `fg` (Foreground): 把后台任务抓回前台。

2.3 进程查看 (`ps`) 风格

- BSD 风格 (无横杠): `ps aux` (`a` 所有用户, `u` 详细信息, `x` 包含无终端进程) 。
- UNIX 风格 (有横杠): `ps -ef` (`-e` 全选, `-f` 全格式输出) 。

进程与系统控制命令对照表

操作功能 (日本語)	命令/快捷键	关键点
一時停止 (Suspend)	<code>Ctrl + Z</code>	状态变为 Stopped
割り込み (Interrupt)	<code>Ctrl + C</code>	直接结束进程
再起動 (服务)	<code>systemctl restart</code>	先 stop 再 start
電源を切る (Power off)	<code>poweroff</code>	也可以是 <code>halt -p</code>
ツリー表示 (Tree)	<code>ps f</code>	显示父子关系

小学生级记忆法

1. `Ctrl + Z` 与 `Ctrl + C` :
 - `Z` 是睡眠 (睡觉 **Zzz**) , 所以是 “暂停” 。
 - `C` 是 **C**ancel (取消) 或者 **C**lose, 所以是 “结束” 。
2. `systemctl restart` :
 - `systemd` 就像系统的总管 (**ctl** = control) , 你想让他 “重新开始 (**restart**) ” 某项工作。
3. `ps` 的 `l` 选项:
 - `l` 代表 **L**ong (长格式) 。因为要显示 nice 值、PID 等一堆信息, 所以列表会变得很 “长” 。
4. `pkill` (批量杀):
 - `kill` 只能一个一个瞄准 PID, `pkill` 像散弹枪, 只要名字匹配 (Pattern) , 统统干掉。

考试小秘籍

- a. `list-units` vs `list-unit-files` :

- `list-units` 显示的是**当前内存中**的状态。
- `list-unit-files` 显示的是**磁盘上安装的所有**单元文件及其开机自启状态（考试常考这个）。

b. `ps` 的字母排序:

- 题目要求「**アルファベット順（字母顺序）**」。所以是 `ax` 而不是 `xa`。

c. 电源切断命令:

- 记得 `halt` 必须配合 `-p` (poweroff) 才能真正切断物理电源，否则可能只停留在关机界面。

d. `init` 程序演进:

- 老旧系统用 `SysVinit`（使用 `/etc/inittab`）。
- 现代系统用 `systemd` 或 `Upstart`，它们不再依赖那个陈旧的配置文件。

`systemd`が稼働するシステムにおいて、システム起動時にサービス `foo` が自動起動するように設定したい。実行するコマンドは？（サービス名は短い形式で）

`systemctl enable foo`

プロセスに終了や再起動などのシグナル(命令)を送ることができるコマンドは？(短い方から先に3つ)

`kill` / `pkill` / `killall`

「`ping local > log`」コマンドをログアウト後もバックグラウンドで実行し続けたい。適切なコマンドは？

`nohup ping local > log &`

端末上で複数の仮想端末を開くことのできるプログラム名を答えよ。（アルファベット順に）

`screen` / `tmux`

プロセスの稼働状況を一定時間ごとに更新して表示するコマンドは？

`top`

Xサーバへの接続に使用される資格情報を表示したり、クライアント認証ファイルを編集するコマンドは？

`xauth`

「`ps ax | grep httpd`」を実行したが「`grep httpd`」が含まれており、1件不要である。解決するコマンドは？（パイプを使わず1コマンドで）

`pgrep httpd`

プログラムをバックグラウンドで動作させたい。コマンドラインの末尾に追加する文字は？

&

現在実行中のプロセスからtestユーザーが実行したプロセスのみを検索するコマンドは？

pgrep -u test

現在バックグラウンドで動作しているジョブと一時停止中のジョブを表示するコマンドは？

jobs

プロセスIDが10のプロセスをクリーンアップせずに強制的に終了させたい。適切なコマンドは？ (-s オプションを併用し、シグナル名は略さずに大文字で)

kill -s SIGKILL 10

psコマンドのオプションのうち、完全なフォーマットでプロセスを表示するUNIXオプションは？

-f

「killall mycom」コマンドと同等のコマンドを、シグナルを指定して実行したい。以下の下線部を埋めなさい。なお、-sオプションを必ず併用する事。（短い順に3つ。シグナル名の場合は大文字で）

killall _____ mycom

-s 15

-s TERM

-s SIGTERM

一時停止中のジョブをバックグラウンド動作に切り替えるコマンドは？

bg

以下の特徴を持つinitプログラムの名前は？

- 扱う処理をUnitという単位で管理する
- サービスの並列起動によって高速なシステム起動や停止が行える
- cgroupsでプロセスのリソースを管理する

systemd

知识点总结

掌握 `systemd` 的服务管理、多进程批量控制、以及维持进程在后台稳定运行（nohup）的技巧。

选项解析与详细说明

2.4 现代化初始化系统：systemd

- `systemctl enable`：核心在于「自動起動（自启动）」。它会在对应的 Target 目录下创建符号链接。

- **Unit (单元)** : `systemd` 的最小管理单位, 通过 `cgroups` 确保进程被精准隔离和限制资源。
- **并列启动**: 与老旧的 `SysVinit` 按顺序启动不同, `systemd` 允许服务并行启动, 极大缩短了开机时间。

2.2 进程批量处理工具

- `killall` / `pkill` :
 - `killall` : 必须给出进程的 **精确名称**。
 - `pkill` : 支持 **部分名称** 或正则匹配。
- `pgrep` : 题目中提到的「**grep httpd 自身を除外**」。使用 `pgrep` 可以直接过滤掉搜索进程本身, 只返回目标的 PID。

2.3 终端复用与持久化

- `nohup` : **No Hang up**。即使关闭终端窗口, 进程也会忽略 `SIGHUP` 信号继续跑。
- `screen` / `tmux` : 虚拟终端工具。可以在一个窗口里开多个分屏, 即使断开连接, 下次连上来还能看到之前的操作。

进程控制命令对比表

命令	匹配方式	常用场景
kill	PID (数字)	精确打击某个特定进程
killall	完全名	杀掉所有叫 httpd 的进程
pkill	部分名/用户	杀掉 test 用户的所有进程 (-u test)
pgrep	部分名	仅查询 PID, 不发送信号

小学生级记忆法

1. `systemctl enable` (开机使能):
 - **Enable** 意为“开启功能”。开机自动启动就是开启了这项功能。
2. `nohup` (不挂断):
 - **No** (不) + **Hang up** (挂断电话)。就像老式电话不挂机, 线就不掉。
3. `tmux` / `screen` (分身术):

- 只要记住这两个名字是“虚拟终端”的双胞胎即可。

4. `xauth` (X 认证):

- X (图形界面) + **Auth** (认证/资格)。管理谁能连我的图形显示器。

考试小秘籍

a. `killall` 的默认信号:

- 如果不指定信号, `killall` 默认发送的是 `SIGTERM (15)`。题目要求按长度排序: `-s 15` (5字) < `-s TERM` (7字) < `-s SIGTERM` (10字)。

b. `nohup` 别忘了 `&`:

- `nohup` 只是忽略信号, 如果要想命令在后台跑不占用当前屏幕, 结尾必须加 `&`。

c. `pgrep` 的优势:

- 考试中如果问“如何只显示 PID 且不包含搜索命令自身”, 选 `pgrep` 是标准解。

d. `systemd` 拼写:

- 很多人会写成 `systemctl`, 注意题目问的是「プログラムの名前」, 答案是 `systemd` (末尾是 d, 代表 daemon)。

動作中のプロセスの親子関係をつリー状に表示できるコマンドは? (オプション無しでツリー状に表示できるコマンドとする)

`pstree`

GUIアプリケーションを表示する先のXサーバを指定する環境変数は?

`DISPLAY`

`ps` コマンドのオプションのうち、プロセスの実行ユーザ名も表示するオプションは?

`u`

「`killall mycom`」コマンドと同等のコマンドを、シグナルを指定して実行したい。以下の下線部を埋めなさい。但し、`-s` オプションは使用しない事。(短い順に3つ。シグナル名の場合は大文字で)

`killall _____ mycom`

`-15`

`-TERM`

`-SIGTERM`

`systemd` が稼働するシステムにおいて、サービスに設定ファイルを再読み込ませる `systemctl` のサブコマンドは?

`reload`

systemctlのサブコマンドでシステムを停止させることができるものは？（アルファベット順に2つ）

halt / poweroff

systemctlのサブコマンドで現在のターゲットを変更して直ちにレスキューモードに切り替えることができるものは？

rescue

systemdが稼働するシステムで、システム起動時に最初に読み込まれるUnitファイルは？（絶対パスで回答）

/etc/systemd/system/default.target

systemdが稼働するシステムにおいて、サービス foo を手動で起動したい。実行するコマンドは？（サービス名は短い形式で）

systemctl start foo

実行中の全てのbashプロセスのプロセスIDを表示したい。実行するコマンドは？（パイプを使わず1コマンドで）

systemctl start foo

systemctlコマンドで、起動中のgraphical.targetから即座にmulti-user.targetに変更したい。設定コマンドを入力せよ（拡張子もつけること）

systemctl isolate multi-user.target

現在実行中のプロセスを表示するコマンドは？

ps

Xクライアント192.168.0.2上で実行したグラフィカルなアプリケーションを、Xサーバ192.168.0.1上のデフォルトディスプレイで表示させたい。Xクライアントの環境変数DISPLAYの値は？（ディスプレイ番号とスクリーン番号も指定する事）

192.168.0.1:0.0

システムを再起動させるためのsystemctlコマンドのサブコマンドは？

reboot

systemdが稼働するシステムにおいて、システム起動時にサービス foo が自動起動しないようにしたい。実行するコマンドは？（サービス名は短い形式で）

systemctl disable foo

知识点总结

深入掌握 `systemd` 的运行目标（Target）切换、服务控制，以及 X Window 系统的显示原理与环境变量配置。

 选项解析与详细说明

2.4 systemd 的状态切换与 Target

- `systemctl isolate` : 这是一个非常核心的命令。「**isolate (隔离)**」用于立即切换系统的状态 (运行级别)。例如从图形界面 (`graphical.target`) 切换到纯文本模式 (`multi-user.target`)。
- `default.target` : 系统启动时的“北极星”，决定了开机进入什么模式。通常它是指向 `multi-user.target` 或 `graphical.target` 的符号链接。
- `rescue` : 救急模式，用于系统出现故障时进行维护。

2.2 X Window 系统的显示逻辑

- **DISPLAY 环境变量**: 格式为 `[主机名]:[显示器号].[屏幕号]`。
 - 例子: `192.168.0.1:0.0` 表示将画面发送到 IP 为 `192.168.0.1` 的机器的第 0 个显示器的第 0 个屏幕上。
- **X Client 与 X Server**:
 - **X Server**: 运行在你面前有显示器的那台机器上 (负责画图)。
 - **X Client**: 运行在远程服务器上的程序 (负责计算)。

2.3 信号发送的多种写法

- `killall` 的信号指定:
 - 即使不带 `-s`，也可以直接用 `-` 加信号名或数字。
 - 考试要求按长度排序: `-15` (3字) < `-TERM` (5字) < `-SIGTERM` (8字)。

 systemctl 常用操作对照表

目的 (日本語)	systemctl サブ コマンド	说明
启动服务	<code>start</code>	立即运行
停止服务	<code>stop</code>	立即停止
重新加载配置	<code>reload</code>	不停止进程，仅 重读配置
开机自启	<code>enable</code>	注册到启动项
取消自启	<code>disable</code>	从启动项移除
立即关机	<code>poweroff / halt</code>	二者略有不同， 但都用于关机

🧠 小学生级记忆法

1. `isolate` (切换状态):

- **Isolate** 意为“隔离”。想象把当前的运行状态全部切断，只留（隔离出）目标状态。

2. `DISPLAY` 变量格式:

- 记公式：**谁的脸 (IP)** : **哪只眼 (Display)** . **哪个瞳孔 (Screen)**。

3. `pstree` (进程树):

- **P**rocess (进程) + **S**tatus (状态) + **T**ree (树)。一眼看穿谁生了谁。

4. `u` 选项 (User):

- `ps u` 中的 `u` 就是 **U**ser。想看是谁运行的，就加 `u`。

📝 考试小秘籍

a. `default.target` 的位置:

- 绝对路径一定要记牢：`/etc/systemd/system/` 下的优先级最高，这才是系统真正读取的地方。

b. `reload` vs `restart`:

- 题目中如果是「設定ファイルを再読込（重新读取配置）」 $\rightarrow$ 选 `reload`。
- 如果是「サービスを再起動（重新启动服务）」 $\rightarrow$ 选 `restart`。

c. `DISPLAY` 变量细节:

- 注意题目要求包含「ディスプレイ番号とスクリーン番号」，哪怕是默认的也要写成 `:0.0`。

d. `pgrep` 的唯一性:

- 只要看到「1コマンドで」且「不要な `grep` 自身を除外」，答案必选 `pgrep`。

systemdのデフォルトのターゲットが格納されているディレクトリは？

`/etc/systemd/system/`

以下のコマンドを実行した。プロセスID「1234」に送られるシグナル名は？（シグナル名は略せずに大文字で）

`$ kill 1234`

SIGTERM

systemdが稼働するシステムにおいて、サービス `foo` を停止したい。実行するコマンドは？（サービス名は短い形式で）

`systemctl stop foo`

プロセスを割り込みにより終了するシグナル名は？ (省略した形、大文字で)

INT

数分後にシステムを停止するようshutdownコマンドを実行したが、キャンセルしたい。適切なコマンドは？

`shutdown -c`

60分後にサーバを停止するコマンドは？

`shutdown 60` (時間の数字だけを指定)

`shutdown -h +60` (オプション、+時間を指定)

23:00にサーバを停止するコマンドは？

`shutdown -h 23:00`

systemdの管理単位であるUnitの名称を以下の順に答えよ

- ・各種デバイスを管理するUnit **device**
- ・ファイルシステムのマウントを管理するUnit **mount**
- ・サービスを制御するUnit **service**
- ・スワップ領域を管理するUnit **swap**
- ・複数のサービスをひとつのグループにするためのUnit **target**

「Rebooting soon」というメッセージを表示して、23:00にサーバを再起動するコマンドは？

`shutdown -r 23:00 "Rebooting soon"`

プロセスを一時停止するシグナルは？ (省略した形、大文字で。アルファベット順に2つ)

STOP / TSTP

端末の切断時にプロセスを終了する、または設定ファイルを再度読み込ませるシグナル名は？ (省略した形、大文字で)

HUP

ジョブ番号2のジョブをフォアグラウンド動作に切り替えたい。適切なコマンドは？

`fg 2`

プロセスIDが10のプロセスをクリーンアップせずに強制的に終了させたい。適切なコマンドは？ (-s オプションは使用せず、またシグナル番号で指定する事)

`kill -9 10`

あるコマンドを入力したところ以下の出力を得た。空欄に入るものは？

```
# systemctl [ ] httpd
active
```

is-active

mycomコマンドのみを実行している全てのプロセスをクリーンアップして終了させたい。適切なコマンドは？

killall mycom

💡 知识点总结

掌握 `systemd` 的 Unit 类型及其管理命令，以及 `shutdown` 命令的定时操作与进程信号 (Signals) 的精确控制。

🔍 选项解析与详细说明

2.4 systemd 的 Unit 家族

`systemd` 将一切资源抽象为 **Unit**。

- **service** : 最核心，管理后台服务（如 httpd）。
- **target** : 一组服务的集合，类似传统的运行级别（Runlevel）。
- **mount** / **swap** : 负责挂载文件系统和虚拟内存。
- **device** : 识别与管理硬件设备。

2. shutdown 时间与消息管理

- **相对时间**: `+60` 或直接写 `60` 代表 60 分钟后。
- **绝对时间**: `23:00` 代表当天的特定时刻。
- **通知消息**: 引号内的字符串会发送给所有登录的用户。
- **取消**: 执行 `shutdown -c` (Cancel) 可以撤回排队的关机计划。

2.3 进程信号 (Signals) 的深层区别

- **SIGTERM (15)**: `kill` 的默认信号。优雅的终结者，给程序时间收尾。
- **SIGKILL (9)**: 暴力的执行者。不清理数据，直接内存清除。
- **INT**: 对应 `Ctrl + C`，发给前台程序的请求中断信号。
- **STOP / TSTP**:
 - **STOP**: 强制暂停（不可忽略）。
 - **TSTP**: 终端发出的暂停请求（对应 `Ctrl + Z`）。

进程控制信号对照表

信号名 (大写)	缩写	数值	键盘操作	作用说明 (日本語)
SIGTERM	TERM	15	-	クリーンアップして終了 (默认)
SIGKILL	KILL	9	-	強制終了 (不清理)
SIGINT	INT	2	Ctrl+C	割り込みによる終了
SIGTSTP	TSTP	20	Ctrl+Z	一時停止 (终端请求)
SIGHUP	HUP	1	-	設定再読込 / 端末切断

小学生级记忆法

- `kill -9` (九死一生):
 - 数字 **9** 就像一把镰刀。一旦使用 `-9`，进程没有任何生存机会，立刻消失。
- `shutdown -c` (Cancel):
 - 关机计划开启后，如果你后悔了，按下 **C** 键来 **Cancel**（取消）。
- `is-active` (活着吗):
 - 这是一个询问句：“它是 **Active**（活跃）的吗？”。常用于脚本判断服务状态。
- `target` (组团):
 - Target** 就像一个靶心，所有的服务（Unit）都向它看齐，聚成一个组。

考试小秘籍

- 默认信号:
 - 如果题目问：`kill 1234` 送出什么？千万别写数字 15，要写名称 **SIGTERM**。
- `shutdown` 的 `-h` :
 - 在老旧系统中，`-h` 代表 **Halt**（停止）。虽然现代 `shutdown` 默认也是停止，但考试时带上 `-h` 或 `-r` 更稳妥。
- Unit 的拼写:
 - `device`，`mount`，`service`，`swap`，`target` 这五个名称是 `systemctl list-units` 里的常客，务必能准确拼写。
- 作业控制 (Job Control):
 - 如果要抓回前台，用 `fg [作业号]`；如果在后台跑，用 `bg [作业号]`。

CUI環境からGUI環境に移行するコマンドは？

startx

systemdが稼働するシステムで次回起動時のターゲットを確認できるコマンドは？

systemctl get-default

システムの再起動のみを行えるコマンドは？

reboot

shutdownコマンドのオプションで、シャットダウン処理を行わず、メッセージ通知のみを行うものは？

-k

systemdを管理するためのコマンドは？

systemctl

あるコマンドを入力したところ以下の出力を得た。実行したコマンドは？（サービス名は短い形式で）

```
# [REDACTED]
httpd.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled)
  Active: active (running) since 月 2015-06-22 14:59:38 JST; 9min ago
  Main PID: 2329 (httpd)
  Status: "Total requests: 0; Current requests/sec: 0; Current traffic: 0
B/sec"
  CGroup: /system.slice/httpd.service
          tq2329 /usr/sbin/httpd -DFOREGROUND
          tq2330 /usr/sbin/httpd -DFOREGROUND
          tq2331 /usr/sbin/httpd -DFOREGROUND
          tq2332 /usr/sbin/httpd -DFOREGROUND
          tq2333 /usr/sbin/httpd -DFOREGROUND
          mq2334 /usr/sbin/httpd -DFOREGROUND

 6月 22 14:59:38 centos7.localdomain systemd[1]: Started The Apache HTTP Server.
```

systemctl status httpd

systemctlコマンドで次回起動時にシングルユーザーモードで起動するよう設定したい。設定コマンドを入力せよ（拡張子もつけること）

systemctl set-default rescue.target

psコマンドで動作中の全てのプロセスの親子関係をつリー状で表示したい。適切なコマンドは？（オプションはアルファベット順に指定）

ps afx

一時停止しているプロセスを再開するシグナル名は？（省略した形、大文字で）

システムの稼働時間や負荷平均などを表示するコマンドは? (アルファベット順に2つ)

top / uptime

💡 知识点总结

掌握 `systemd` 的默认运行目标设置、`ps` 命令的复合选项、以及用于监控系统整体健康状况（负载与运行时间）的命令。

🔍 选项解析与详细说明

2.4 运行目标 (Target) 的持久化设置

- `systemctl get-default` : 查看系统下次开机时默认进入的模式（如图形界面或命令行）。
- `systemctl set-default` : 修改下次开机的默认模式。
 - `rescue.target` : 相当于传统的单用户模式（Single User Mode），常用于系统修复。
- `startx` : 在已经登录的命令行（CUI）环境下，手动启动 X Window 图形界面。

2.2 系统状态监控

- `uptime` : 简洁地显示系统运行了多久、当前有多少用户登录、以及最近 1/5/15 分钟的 **平均负载 (Load Average)**。
- `top` : 动态实时视图，除了包含 `uptime` 的信息外，还能看到占用资源最多的进程。

2.3 信号与进程树

- `SIGCONT` (`CONT`): `CONTInue`。让被 `STOP` 或 `TSTP` 暂停的进程恢复运行。
 - `ps afx` :
 - `a` : 显示所有用户的进程。
 - `f` : `forest`。用 ASCII 字符画出进程的树状结构（父子关系）。
 - `x` : 显示没有控制终端的进程。
-



`systemctl` 常用查询命令表

目的 (日本語)	命令	说明
查看服务状态	<code>systemctl status [服务名]</code>	显示是否运行、日志摘要
查看当前默认模式	<code>systemctl get-default</code>	查看下次开机的 Target
设置默认模式	<code>systemctl set-default [Target]</code>	永久修改启动模式
查看单元文件状态	<code>systemctl list-unit-files</code>	查看所有 Service 的使能状态

小学生级记忆法

1. `SIGCONT` (继续游戏):

- 就像玩游戏按了暂停，按下 **CONT**inue 键，进程就继续跑了。

2. `uptime` (醒了多久):

- Up** (运行着) + **Time** (时间)。问问电脑：“你醒了多久了？累不累（负载）？”

3. `ps afx` (森林):

- 把 **f** 想成 **F**orest（森林）。森林里有很多树，所以它能显示“进程树”。

4. `-k` 选项 (Kidding):

- `shutdown -k` 发出的关机警告是假的。**k** 代表 **K**idding（开玩笑），只吓唬用户，不真的关机。

考试小秘籍

a. `status` 输出的辨识:

- 看到包含 `Loaded: ...`, `Active: ...`, `Main PID: ...` 以及最后的日志输出，这绝对是 `systemctl status` 的结果。

b. `set-default` 的参数:

- 题目要求「拡張子もつける（带上后缀）」，所以一定要写 `.target`，例如 `rescue.target`。

c. 负载平均值 (Load Average):

- 如果题目问“哪个命令可以查看最近 15 分钟的平均负载”，答案是 `uptime` 或 `top`。

d. `ps` 的字母顺序:

- 题目要求「アルファベット順（字母顺序）」，所以是 `afx`。

shutdownコマンドに関するmanページのセクション番号は？

8

bashシェルにおいて、一般ユーザのプロンプトの末尾に表示される記号は？

\$

設定されている全ての環境変数を一覧表示できるコマンドは？ (アルファベット順に4つ)

declare / env / printenv / set

ファイルの内容をページ(1画面)ごとに表示するが、編集は出来ないコマンドは？ (アルファベット順に2つ)

less / more

囲んだ部分を文字列とみなすメタキャラクタは？ (他のメタキャラクタをエスケープ。但し「`」、
「\」、 「'」を除く)

"

変数を表すために使用されるメタキャラクタは？

\$

whatisデータベースを作成するコマンドは？

makewhatis

bashシェルにおいて、スーパーユーザ(root)のプロンプトの末尾に表示される記号は？

#

/etc/crontabファイルに関するmanページを表示するコマンドは？

man 5 crontab

bashのシェル変数LPICを削除するコマンドは？

unset LPIC

絶対パスを指定しなくてもコマンドを実行できるようにするための環境変数は？

PATH

現在ログインしているユーザはtestである。testユーザのホームディレクトリに移動するコマンドは？ (短い順に3つ)

cd / cd ~ / cd ~test

ログインしているユーザ名を格納する環境変数は？

USER

次に来る文字 (メタキャラクタ) をエスケープするメタキャラクタは？

\

知识点总结

掌握 Linux 的基础环境配置，包括环境变量（Environment Variables）、Shell 元字符（Metacharacters）、以及帮助手册（man pages）的分段规则。

选项解析与详细说明

2.4 Man 手册的章节 (Sections)

Linux 的 `man` 手册是分章节存储的，理解章节号能帮你快速定位信息：

- **1:** 一般用户命令（如 `ls`，`cd`）。
- **5:** 配置文件格式（如 `/etc/crontab`，`/etc/passwd`）。
- **8:** 系统管理命令（如 `shutdown`，`fdisk`，通常需要 root 权限）。

2.2 元字符与转义 (Escaping)

- `\` (反斜杠)：转义下一个字符。
- `'` (单引号)：强引用。内部所有元字符都失去特殊含义，变为普通字符串。
- `"` (双引号)：弱引用。大部分元字符失效，但 `$`（变量）、```（反向引用）和 `\`（转义）依然有效。

2.3 变量查看命令的区别

- `env` / `printenv`：仅查看环境变量（导出的变量）。
- `set` / `declare`：查看所有变量（包括环境变量和局部 Shell 变量）。

Shell 变量与环境对照表

环境变量	作用 (日本語)
<code>\$PATH</code>	コマンドの検索パス
<code>\$USER</code>	現在のログインユーザ名
<code>\$HOME</code>	ユーザのホームディレクトリのパス
<code>\$SHELL</code>	使用中のシェル (例: <code>/bin/bash</code>)

小学生级记忆法

1. 用户符号记忆:

- `$` = Standard User (标准用户，普通人的钱袋子)。
- `#` = 井号（像一把沉重的锁或者王冠），代表拥有最高权力的 **root**。

2. `unset` (取消设置):

- 就像你要“退订”一个服务，**set** 是设置，**un-set** 就是撤回这个设置。

3. 转义符 `\`:

- 把它看成一个“变魔术的手”，指一下后面的特殊符号，那个符号就变回普通字符了。

4. `man 5` 与 `man 8`:

- **5** 字长得像一个钩子，用来钩住（描述）**配置文件**。
- **8** 字像两个圆（眼睛），**管理员**要睁大眼睛盯着系统管理命令。

考试小秘籍

a. `man` 命令的参数顺序:

- 如果要看配置文件的帮助，格式是 `man [章节号] [名字]`。如果不写 5，直接 `man crontab` 可能会搜到第 1 章节的命令说明。

b. 环境变量的大小写:

- Linux 环境变量（如 `PATH`，`USER`）通常全部是 **大写**，这是不成文的准则。

c. `cd` 切换家目录:

- 最短的命令就是 `cd`（什么都不带默认回老家）。其次是 `cd ~`。

d. `whatis` 数据库:

- `whatis` 命令能快速显示命令的一句话介绍，其后台数据库需要通过 `makewhatis`（或在某些系统中是 `mandb`）来生成。

sambaというキーワードを一部に含むmanページを検索するコマンドは?(アルファベット順に2つ)

`apropos samba` / `man -k samba`

カレントディレクトリのパスを表示するコマンドは?

`pwd`

シェルから起動したアプリケーションやコマンドでも有効な環境変数を作成したり、シェル変数を環境変数に変更したりできるコマンド（またはコマンドとそのオプション）は?(アルファベット順に2つ)

`declare -x` / `export`

多くのディストリビューションで採用される、デフォルトのシェルは?

`bash`

command1を実行し、続けてcommand2を実行したい。なお、command1の実行結果はcommand2を実行するか否かには関係しない。下線部にあてはまる語句は？

command1 __ command2

；

囲んだ部分を文字列とみなすメタキャラクタは？（他のメタキャラクタを全てエスケープ）

'

任意の1文字を表すメタキャラクタは？

？

ホスト名を格納する環境変数は？

HOSTNAME

passwdというキーワードと完全に一致した名前を含むmanページを検索するコマンドは？(アルファベット順に2つ)

man -f passwd / whatis passwd

lsコマンドのオンラインマニュアルを参照するコマンドは？

man ls

command1を実行し、続けてcommand2を実行させたい。ただし、command1が失敗した場合はcommand2を実行しないこととする。下線部にあてはまる語句は？

command1 __ command2

&&

カレントディレクトリの1つ上の階層（親ディレクトリ）を表す特殊記号は？

..

コマンド実行履歴保存ファイルの最大記録数を設定する環境変数は？

HISTFILESIZE

知识点总结

掌握 Linux Shell 的基本操作，包括命令的逻辑连接符（Logical Operators）、环境变量的导出、以及手册页（Manual Pages）的关键词检索。

选项解析与详细说明

2.4 命令逻辑连接符 (Job Chaining Operators)

在 Linux 命令行中，如何连接多个命令决定了它们的执行逻辑：

-  (分号)：顺序执行。不管前面的命令成功还是失败，后面的都会跑。

- `&&` (AND): 逻辑与。只有前面的命令 **成功 (Exit Status 0)**，后面的才会执行。
- `||` (OR): 逻辑或。只有前面的命令 **失败 (Exit Status 非0)**，后面的才会执行。

2.2 变量的“晋升”：环境控制

- `export` / `declare -x`：普通变量（シェル変数）只在当前进程有效。通过这些命令，可以将变量提升为 **环境变量**，这样从当前 Shell 启动的所有子进程都能访问到它。

2.3 Man 手册的精准搜索

- `apropos` (`man -k`): 模糊匹配。只要包含关键字（如 `samba`）的描述都会列出。
- `whatis` (`man -f`): 精确匹配。只列出命令名称与关键字完全一致的内容。

 环境变量与历史记录对照表

变量名	作用 (日本語)
<code>\$HOSTNAME</code>	システムのホスト名
<code>\$HISTSIZE</code>	メモリ上の履歴の最大数
<code>\$HISTFILESIZE</code>	ファイル (~/.bash_history) に保存される最大数
<code>\$PWD</code>	現在の作業ディレクトリ (Print Working Directory)

 小学生级记忆法

1. `&&` (步步为营):
 - 两个 `&` 连在一起，就像两个好朋友手拉手。只有第一个人平安到达了，第二个人才会跟上。
2. `..` (退一步海阔天空):
 - 两个点 `..` 像是一对向后看的眼睛，代表回到上面的父目录。
3. `?` (猜猜我是谁):
 - 问号 `?` 本身就是疑问，代表我不确定你是哪**一个字**（匹配任意 1 字符）。
4. `export` (出口):
 - **Export** 是出口、贸易。把变量“出口”给子程序使用。



考试小秘籍

a. 分号 vs 双与号：

- 考试题目中看到「関係しない（不相关）」 \$\rightarrow\$ 选 `;`。
- 看到「失敗した場合は実行しない（失败则不执行）」 \$\rightarrow\$ 选 `&&`。

b. 引号的强弱：

- 单引号 `'`：最强转义，连 `$` 变量名都会被当成普通字符。
- 双引号 `"`：允许 `$变量` 正常显示它的值。

c. 历史记录的两个变量：

- `HISTSIZE` 是现在窗口里能翻到的历史；`HISTFILESIZE` 是存在硬盘文件里的历史。题目问的是「保存ファイルの最大記録数」，所以选后者。

d. `whatis` 的别名：

- 记牢 `man -f` 对应 `whatis`，`man -k` 对应 `apropos`。这在选择题中极高频出现。

ロケールを格納する環境変数は？

LANG

コマンド実行履歴を保存するファイルのパスを格納する環境変数は？

HISTFILE

command1を実行し、command1が失敗した場合のみcommand2を続けて実行させたい。下線部にあてはまる語句は？

command1 __ command2

||

現在の環境変数PATHに「/usr/local/test」ディレクトリを追加したい。下線部にあてはまる語句は？（ディレクトリは値の最後に追加すること）

\$ export PATH=_____

\$PATH:/usr/local/test

ホームディレクトリに\workディレクトリを作成するコマンドは？（シングルクォーテーション、ダブルクォーテーションは使わないこと）

mkdir ~/\work

以下はhistoryコマンドを実行した結果である。「cat hoge.txt」を再実行するには？（記号と数字、記号とアルファベットの順に2つ。最少文字数にすること）

```
$ history
1  ls
2  pwd
3  cat hoge.txt
4  cp hoge.txt hoge2.txt
5  history
```

!3 / !ca

ログインしているユーザのホームディレクトリを格納する環境変数は？

HOME

現在ログインしているユーザはrootである。testユーザのホームディレクトリに移動するコマンドは？

cd ~test

bashシェルで実行したコマンドの履歴を表示するコマンドは？

history

0文字以上の文字列を表すメタキャラクタは？

*

シェル変数LPICの値をtestにするコマンドは？

LPIC=test

以下のようにしてシェル変数LPICを設定した。このシェル変数を環境変数にするコマンドは？(アルファベット順に2つ)

\$ LPIC=study_linux

declare -x LPIC / export LPIC

bashシェルで実行したコマンドの履歴を保存するファイルは？（ファイル名のみ）

.bash_history

設定されている全てのシェル変数を一覧表示するコマンドは？(アルファベット順に2つ)

declare / set

カレントディレクトリを表す特殊記号は？

.

bashシェルの文字補完機能を使うためのキーは？（英字3文字）

tab

囲んだ部分をコマンドとみなし、その実行結果を表示するメタキャラクタは？

'

カレントディレクトリのパスを格納する環境変数は？

PWD

シェルスクリプト「script.sh」を実行したい。下線部に入るコマンドは?なお「script.sh」には実行権がないものとする。「script.sh」のあるディレクトリは、PATHに登録済みのディレクトリである。

_____ script.sh (文字数の少ない順に3つ)

. / bash / source

現在のシェルでのコマンド履歴の最大数を格納する環境変数は?

HISTSIZE

这组题目聚焦于 **Bash 历史管理 (History)**、**特殊字符转义** 以及 **PATH 变量的修改**。这些是提升 Linux 操作效率的必备技巧。

💡 知识点总结

- **コマンド履歴 (History):** `!n` 执行第 \$n\$ 条命令, `!string` 执行最近一条以该字符串开头的命令。
- **パスの追加 (PATH):** 修改 `PATH` 时, 必须包含原有的 `$PATH`, 否则会导致基础命令失效。
- **エスケープ (Escape):** 要在文件名中使用特殊字符 (如 `\`), 需用另一个 `\` 来转义。
- **スクリプトの実行:** `.` 或 `source` 在当前进程执行脚本; `bash` 启动子进程执行。

🔍 选项解析与详细说明

3. 历史命令的“瞬间移动”

根据你提供的图片:

- `!3`: 直接指定编号 3 的命令 `cat hoge.txt`。
- `!ca`: 查找最近一条以 `ca` 开头的命令并执行。

2. PATH 变量的追加

- `$PATH:/usr/local/test`:
 - 冒号 `:` 是路径分隔符。
 - **绝对不要**直接写成 `PATH=/usr/local/test`, 这样会覆盖掉 `/bin` 等系统目录。

3. 特殊字符与转义

- `mkdir ~/\ \work`:
 - 第一个 `\` 是转义字符, 第二个 `\` 是你想创建的目录名的一部分。

- 结果会创建一个名为 `\work` 的文件夹。

4. 逻辑操作符 `||`

- `command1 || command2` :
 - 这叫“逻辑或”。只有当 `command1` 报错（失败）时，才会为了“止损”去执行 `command2`。

变量与命令执行对比表

类型	命令/变量	含义/作用
执行脚本	<code>./ source</code>	在 当前环境 执行（变量会生效）
执行脚本	<code>bash</code>	在 子进程 执行（不影响当前环境）
历史文件	<code>.bash_history</code>	存放在硬盘上的历史记录文件
逻辑失败执行	<code>**`</code>	
反引号功能	<code>`</code> (Backtick)	执行其中的命令并将 结果 返回

小学生级记忆法

1. `!3` (惊叹号):
 - 惊叹号就像一个放大镜，帮你“大喊”出以前执行过的第 3 号命令。
2. `||` (最后一道防线):
 - 就像两个并排的栅栏。如果第一个命令跌倒了（失败），第二个命令才会出来接住它。
3. `TAB` (补全):
 - 它是懒人神键。只要按一下，系统就帮你把剩下的字写完。
4. `*` (万能星):
 - 就像星星一样多得数不清，代表匹配 **0 个或更多** 字符。

考试小秘籍

- 反引号 vs 单引号:
 - 考试时看清楚：单引号 `'` 是字符串，反引号 ``` 是执行命令。现在推荐使用 `$(command)` 替代反引号，但 LPI 依然常考反引号。
- `.bash_history` 的点:
 - 这是一个隐藏文件，名字前面必须有 `.`。

- `source` 的简写:
 - 记住 `.` (点后面加空格) 效果等同于 `source`。
- `mkdir` 的转义:
 - 题目明确要求“不使用引号”，所以遇到 `\` 必须用 `\\` 表示。

1行あたりの最大文字数を指定してテキストを整形するコマンドは？

`fmt`

`cat`と`tr`コマンドを使用して、「file」ファイル内の英小文字を英大文字に変換して表示させたい。
適切なコマンドは?(パイプおよび文字クラスを使用する事。リダイレクト演算子は使用しない)

`cat file | tr [:lower:] [:upper:]`

WindowsとLinuxの改行コードをそれぞれ答えよ (アルファベット大文字)

CRLF (Windows) / LF (Linux)

「file」ファイルを1ページあたり30行として整形して表示したい。また1ページから2ページまでを表示したい。適切なコマンドは?(オプションは行数から指定)

`pr -l 30 +1:2 file`

`tr`コマンドで使用する文字クラスのうち、英字を表すものは？

`[:alpha:]`

「file」ファイルの2番目のフィールドを基準に、数値として昇順にソートしたい。区切り文字は「,」（カンマ）とする。適切なコマンドは?(オプションはアルファベット順にそれぞれ指定)

`sort -k 2 -n -t , file`

指定した行数でファイルを分割するコマンドは？

`split`

`file1`と`file2`ファイルはそれぞれの2番目のフィールドが共通している部分がある。その行を結合して表示するコマンドは？

`join -j 2 file1 file2`

「file」ファイルの行頭のタブをスペース1つに置き換えて表示したい。適切なコマンドは？ (オプションはアルファベット順にそれぞれ指定)

`expand -i -t 1 file`

「file」ファイルの内容を1行あたり30文字に整形して表示したい。適切なコマンドは？

`fmt -w 30 file`

「file」ファイルの文字(バイト)数のみを表示したい。適切なコマンドは？

`wc -c file`

「file」 ファイルの先頭500バイトを表示したい。適切なコマンドは？

```
head -c 500 file
```

💡 知识点总结

掌握 Linux 文本处理工具流，包括格式整形 (fmt/pr)、字符转换 (tr)、排序 (sort) 以及文件切分 (split) 等进阶操作。

🔍 选项解析与详细说明

4.1 文本格式整形 (Formatting)

- **fmt**: 快速文本格式化工具。使用 **-w** 指定宽度（最大字数）。
- **pr**: 专门用于 **打印准备** 的格式化工具。可以指定每页行数 (**-l**) 和页码范围 (**+开始:结束**)。
- **expand**: 将 **Tab** 转换为 **空格**。使用 **-i** 仅转换行首的 Tab, **-t** 指定转换后的空格数。

4.2 强大的字符转换 (tr)

- **tr** (translate): 只能通过管道或重定向接收输入。
- **文字クラス (Character Classes)**:
 - **[:lower:]** / **[:upper:]**: 小写/大写字母。
 - **[:alpha:]**: 所有英文字母。
 - **[:digit:]**: 数字。

4.3 数据排序与合并 (sort & join)

- **sort**:
 - **-n**: **numeric** (按数值大小排, 否则 10 会排在 2 前面)。
 - **-k**: **key** (指定第几列)。
 - **-t**: 分隔符 (默认是空格)。
- **join**: 类似 SQL 的 JOIN 操作, 根据共同字段将两个文件的行合并。

4.4 换行符的差异 (Newline Codes)

- **Linux: LF** (Line Feed, **\n**)。
 - **Windows: CRLF** (Carriage Return + Line Feed, **\r\n**)。
-

常用文件信息查看命令表

操作需求	命令	常用选项
统计信息	<code>wc</code>	<code>-l</code> (行), <code>-w</code> (词), <code>-c</code> (字节/文字)
查看开头	<code>head</code>	<code>-n</code> (行数), <code>-c</code> (字节数)
文件分割	<code>split</code>	<code>-l</code> (按行分), <code>-b</code> (按容量分)

小学生级记忆法

1. `fmt` (Format):
 - 它是最基础的格式化。你想让一段乱糟糟的文字变整齐 (`fomat`)，就找它。
2. `tr` (Transformer):
 - 就像变形金刚 (`Transformers`)，它能把小写字母“变身”成大写字母。
3. `sort -n` (Number):
 - 告诉 `sort` 别傻傻地看字符，要像看数字 (`Number`) 一样去排大小。
4. `expand` (扩张):
 - 把一个窄窄的 Tab 键“扩张 (`expand`)”成好几个空格。

考试小秘籍

- a. `sort` 选项的字母序:
 - 题目要求「アルファベット順 (字母顺序)」。如果用到多个选项，如 `-k`，`-n`，`-t`，虽然功能上顺序没关系，但答题时注意题目要求的写法。
- b. `wc` 的 `-c` 与 `-m`:
 - 在传统 LPI 考试中，查看字节数 (文字数) 通常使用 `-c`。
- c. `pr` 的分页语法:
 - 记住 `+` 号代表页码。 `+1:2` 表示从第 1 页到第 2 页。
- d. `expand` 的行首限制:
 - `-i` (initial) 是关键点。如果只转换行首的 Tab，一定要加这个选项。

タブをスペースに変換するコマンドは？

expand

「test.txt」 ファイルの空の行を削除して表示したい。以下のコマンドの空欄に当てはまる正規表現は？

```
$ sed '/____/d' file
```

```
^$
```

「file」 ファイルを500行ごとに分割したものを別のファイルに保存したい。また、保存するファイル名は「hogeaa」、「hogeab」などのようにしたい。適切なコマンドは?(-lオプションを使わずに行数を指定する書式で)

```
split -500 file hoge
```

指定した文字を変換・削除するコマンドは？(2文字のコマンド)

```
tr
```

「file」 ファイルを行番号をつけて出力したい。適切なコマンドは?なお、行番号は空行を除いた行に付けることとする。(アルファベット順に2つ。省略可能なオプションは省略する)

```
cat -b file / nl file
```

指定したファイルの先頭部分を表示するコマンドは？

```
head
```

「/bin/ls」 ファイル(lsコマンドのバイナリファイル)の内容を8進数で表示したい。適切なコマンドは? (オプションを省略しない事)

```
od -t o /bin/ls
```

ファイルの内容を8進数や16進数で表示するコマンドは？

```
od
```

「file」 ファイルを整形する際、ヘッダに表示されるファイル名を「test」に変更したい。また、1ページあたり30行としたい。適切なコマンドは?(オプションはアルファベット順に指定)

```
pr -h test -l 30 file
```

「file1」と「file2」 ファイルを行単位で区切り文字「:」を使用して結合したい。適切なコマンドは？

```
paste -d : file1 file2
```

行単位でファイルの内容をソートするコマンドは？

```
sort
```

「file」 ファイルにあるスペース2つつ分(行頭以外にあるスペースも含む)をタブに置き換えて表示したい。適切なコマンドは？(オプションが少ない方から2つ。オプションはアルファベット順にそれぞれ指定)

```
unexpand -t 2 file / unexpand -a -t 2 file
```

知识点总结

掌握 Linux 文本流的“变身术”，包括空格与 Tab 的互换（expand/unexpand）、文件的水平/垂直合并（paste/split）、以及二进制数据的查看（od）。

选项解析与详细说明

4.5 空格与 Tab 的互换

- `expand` : 将 **Tab** 转换为 **空格**。
- `unexpand` : 将 **空格** 转换为 **Tab**。
 - 默认情况下，`unexpand` 只转换行首的空格。
 - `-a (all)`: 转换**所有**位置的空格。
 - `-t (tabs)`: 指定多少个空格合成一个 Tab。

4.2 文件合并与分割 (Horizontal & Vertical)

- `paste` : **水平合并**。把两个文件的行并排拼在一起。使用 `-d` 指定连接符（如 `:`）。
- `split` : **垂直分割**。
 - 格式: `split -[行数] [原文件] [前缀]`。
 - 例子: `split -500 file hoge` 会生成 `hogeaa`, `hogeab` 等。

4.3 行号显示技巧

- `nl` : 专门用于编号的命令。默认**不给空行编号**。
- `cat -b` : 等同于 `nl`，跳过空行编号。
- `cat -n` : 无论是否空行，一律编号。

4.4 二进制与非文本查看

- `od (Octal Dump)`: 用于查看非文本文件（如二进制、库文件）。
 - `-t o` : 以 **8 进数** (Octal) 显示。
 - `-t x` : 以 **16 进数** (Hex) 显示。

常用正则表达式与 sed 操作

需求 (日本語)	正则表达式	说明
空行 (空の行)	<code>^\$</code>	<code>^</code> (行首) 紧跟 <code>\$</code> (行尾)
行頭文字	<code>^abc</code>	以 <code>abc</code> 开头的行
削除アクション	<code>/d</code>	<code>delete</code>

🧠 小学生级记忆法

1. `unexpand` (反向扩张):
 - `expand` 是把一个点变成一排空格，`un-expand` 就是反过来，把一堆空格缩成一个 Tab。
2. `^$` (空空如也):
 - `^` 是开始，`$` 是结束。中间什么都没有，所以它代表“空行”。
3. `paste` (粘贴):
 - 就像用胶水把两个文件的侧面粘在一起，变成一个更宽的文件。
4. `od` (奇怪的数字):
 - 把 **O**ctal **D**ump 想成 **O**dd **D**igits（奇怪的数字），因为它能把你看得懂的文字变成看不懂的 8 进制。

📝 考试小秘籍

- a. `unexpand` 的隐藏陷阱:
 - 题目如果提到「行頭以外にあるスペースも含む（包含行首以外的空格）」，一定要优先想到添加 `-a` 选项。
- b. `nl` 与 `cat -b`:
 - 这是一个常见的等价替代考点。两者都默认忽略空行。
- c. `split` 的参数顺序:
 - 命令 → 行数 → 源文件 → 输出前缀。如果缺了输出前缀，默认是 `x`（生成 `xaa`，`xab`）。
- d. `sed` 的基本语法:
 - `sed ' /模式/ 动作 ' 文件名`。删除空行就是 `sed '/^$/d' file`。

「/bin/ls」ファイル(lsコマンドのバイナリファイル)の内容をASCII文字とエスケープ文字で表示したい。適切なコマンドは? (オプションを省略しない事)

```
od -t c /bin/ls
```


ソートされているファイルにある、連続して重複した行を1行にまとめて表示するコマンドは？

`uniq`

テキストファイルの各行から指定したフィールドを取り出して表示するコマンドは？

`cut`

`tr`コマンドを使用して「file」ファイル内にある文字「c」を削除したい。適切なコマンドは？(1つ目はリダイレクト演算子、2つ目は`cat`コマンドとパイプを使用する事)

`tr -d c < file / cat file | tr -d c`

`tr`コマンドを使用して「file」ファイル内の英小文字を英大文字に変換して表示させたい。適切なコマンドは？(リダイレクトを使用し、文字クラスは使用しない事)

`tr a-z A-Z < file`

全ての行が数字のみの「file」ファイルについて、数字を文字ではなく、数値として昇順にソートしたい。適切なコマンドは？

`sort -n file`

`tr`コマンドで使用する文字クラスのうち、英数字を表すものは？

`[:alnum:]`

`tr`コマンドを使用して、「file」ファイル内でスペースが連続している場合、スペース1つに置き換えて表示させたい。適切なコマンドは？(1つ目はリダイレクト演算子、2つ目は`cat`コマンドとパイプを使用する事。また両方とも文字クラスを使用する事。)

`tr -s [:space:] < file / cat file | tr -s [:space:]`

「/bin/ls」ファイル(`ls`コマンドのバイナリファイル)の内容を16進数で表示したい。適切なコマンドは？(オプションを省略しない事)

`od -tx /bin/ls`

「file」ファイルの単語数のみを表示したい。適切なコマンドは？

`wc -w file`

「file」ファイルのタブをスペース1つに置き換えて表示したい。適切なコマンドは？

`expand -t 1 file`

ファイル内の文字数や行数を調べることができるコマンドは？

`wc`

知识点总结

掌握 Linux 文本流的“裁剪与压缩”技巧，包括字段提取（`cut`）、重复行处理（`uniq`）、以及 `tr` 命令在删除、去重和转换方面的多种用法。

选项解析与详细说明

4.5 字段提取与重复处理

- `cut` : 像剪刀一样切出你需要的列。常用 `-f` 指定字段 (field) , `-d` 指定分隔符 (delimiter) 。
- `uniq` : **UNIQue**。它只能处理「連続して重複した (连续重复)」的行。所以通常先用 `sort` 排序, 再接 `uniq` 。

2. `tr` 命令的高级技巧

- `-d (delete)`: 删除指定的字符。
- `-s (squeeze)`: 「連続している場合、1つに置き換えて (压缩重复)」。常用于将多个连续空格压缩为一个。
- 输入方式: `tr` 不接受文件名作为参数, 必须通过 **リダイレクト** `<` 或 **パイプ** `|` 输入。
- 文字クラス:
 - `[:alnum:]` : **A**lphabetic + **N**umeric (英数字) 。
 - `[:space:]` : 空白字符 (包括空格、Tab 等) 。

4.3 二进制查看器 `od` (续)

- `-t c` : **C**haracter。以 **ASCII文字** 或转义字符 (如 `\n` , `\t`) 显示内容。
- `-t x` : **H**exadecimal。以 **16進数** 显示。

`WC` 命令选项速查表

操作需求 (日本語)	选项	记忆点
行数を表示	<code>-l</code>	L ines
単語数を表示	<code>-w</code>	W ords
文字(バイト)数を表示	<code>-c</code>	C haracters / B ytes

小学生级记忆法

1. `cut` (剪纸):
 - 想象一张报纸, 你只想要中间那一纵列的内容, 用 `cut` 把它剪下来。
2. `tr -s` (Squeeze/挤压):
 - 想象一排长长的海绵 (连续空格), 你用力一挤 (Squeeze), 它们就缩成了一个点。

3. `[:alnum:]` (全能字母数字):

- Alphabet (字母) + Number (数字) = Alnum。

4. `od -t c` (See characters):

- 这里的 `c` 就是 Character (字符)，让你能看到二进制文件里隐藏的文字。

考试小秘籍

a. `tr` 的输入重定向:

- 考试时如果要求“使用リダイレクト”，千万不要写 `tr -d c file`，这是错误的！必须是 `tr -d c < file`。

b. `uniq` 的前提条件:

- 只要看到「重複した行を1行に（重复行合为一）」，一定要联想到这个命令。如果题目问如何处理不连续的重复行，答案是先 `sort` 再 `uniq`。

c. `[:alnum:]` vs `[:alpha:]`:

- `[:alpha:]` 只有英文字母。
- `[:alnum:]` 包含英文和数字。看清题目要求的是否包含“数字”。

d. `expand -t`:

- 默认转换是 8 个空格。如果题目要求「スペース1つ（1个空格）」，必须指定 `-t 1`。

2つのファイルを読み込み、共通するフィールドがある行を結合して表示するコマンドは？

join

「file」ファイルの末尾5行を表示したい。適切なコマンドは？(オプションが短い方から先に2つ)

tail -5 file / tail -n 5 file

「file」ファイルにある連続して重複した行を1行にまとめて、「newfile」ファイルに出力したい。適切なコマンドは？なお、「file」ファイルはソートされている。

uniq file newfile

指定したファイルの末尾部分を表示するコマンドは？

tail

行頭にある連続したスペースをタブに変換するコマンドは？

unexpand

「file」ファイルの先頭5行を表示したい。適切なコマンドは？(オプションが短い方から先に2つ)

head -5 file / head -n 5 file

「file」 ファイルの行数、単語数、文字(バイト)数を表示したい。適切なコマンドは?(オプションは省略する事)

`wc file`

「file」 ファイルの末尾500バイトを表示したい。適切なコマンドは?

`tail -c 500 file`

複数のファイルを行単位で結合するコマンドは?

`paste`

ファイルの内容を一度にすべて表示し、編集は出来ないコマンドは?

`cat`

「file」 ファイルの各行の2番目の文字を取り出して表示したい。適切なコマンドは?

`cut -c 2 file`

「/var/log/messages」 ファイルに追記されるログをリアルタイムで表示したい。適切なコマンドは?

`tail -f /var/log/messages`

ファイルを印刷用に整形するコマンドは?

`pr`

「:」 で区切られている「/etc/passwd」 ファイルの1番目と7番目のフィールドを取り出して表示したい。適切なコマンドは?(オプションはアルファベット順に指定)

`cut -d : -f 1,7 /etc/passwd`

「file」 ファイルの行数のみを表示したい。適切なコマンドは?

`wc -l file`

trコマンドで使用する文字クラスのうち、数字を表すものは?

`[:digit:]`

「file」 ファイルを行番号をつけて出力したい。適切なコマンドは?なお、行番号は空行を含めた全ての行に付けることとする。(アルファベット順に2つ)

`cat -n file / nl -b a file`

这组题目涵盖了 Linux 文本处理中关于**文件尾部监控 (tail)**、****字段切割 (cut)** 以及**文件合并 (join/paste)** **的核心用法。

知识点总结

- **实时监控:** `tail -f` 是排查服务器日志 (Logs) 的“神技”，它能捕捉文件末尾的新增内容。

- **字段提取：** `cut` 可以通过字符位置（`-c`）或分隔符（`-d`）配合字段序号（`-f`）来提取信息。
 - **行号显示：** `cat -n` 和 `nl -b a` 都是给**所有行**（包括空行）加编号的常用方法。
 - **数据结合：** `join` 像数据库一样关联两个文件，而 `paste` 只是简单的并排粘贴。
-

选项解析与详细说明

1. `tail` 的灵活用法

- **查看行数：** `tail -5` 或 `tail -n 5` 效果一致，显示最后 5 行。
- **查看容量：** `tail -c 500` 显示最后 500 字节（Bytes）。
- **实时追踪：** `-f` (follow) 选项。它会保持文件打开状态，一旦有程序写入新日志，屏幕就会立即刷新。

2. `cut` 的精准切割

- **字符切割（`-c`）：** 如 `cut -c 2` 表示只拿每行的第 2 个字符。
- **字段切割（`-d` & `-f`）：**
 - `-d` ：指定冒号为**分隔符**（Delimiter）。
 - `-f 1,7` 指定取第 1 和第 7 个**字段**（Fields）。常用于处理 `/etc/passwd` 这种格式化文件。

5. 文件连接的差异

- **`paste`（水平粘贴）：**
 - 文件 A: 1, 2
 - 文件 B: a, b
 - 结果: 1 a, 2 b
 - **`join`（逻辑结合）：**
 - 需要两个文件有**共同的列**（Key），类似于 SQL 的 `INNER JOIN`。
-

文本处理命令功能表

操作需求 (日本語)	命令	关键选项	记忆点
实时日志监控	tail	-f	Follow
所有行加编号	cat / nl	-n / -b a	number / body all
字段(列)提取	cut	-f, -d	field, delimiter
字符转换	tr	[:digit:]	数字类

🧠 小学生级记忆法

1. `tail -f` (跟屁虫):
 - 把 `-f` 想成 Follow (跟随)。日志走到哪，你就跟到哪，永远盯着最后一行。
2. `cut -d` (切蛋糕):
 - `-d` 就是那把刀 (Dagger/刀)，用来指定在哪里切开 (分隔符)。
3. `[:digit:]` (手指头):
 - Digit 既是“数字”也是“手指”。数数当然用数字。
4. `unexpand` (收缩):
 - 看到“行头连续空格转 Tab”，就是要把散乱的空格收缩回一个 Tab 键。

📝 考试小秘籍

- o `nl` 的选项细节:
 - 默认的 `nl` 不给空行编号。如果题目要求「空行を含めた全ての行 (包含空行在内的所有行)」，必须写成 `nl -b a`。
- o `head` / `tail` 的数字写法:
 - 在 LPI 考试中，`-n 5` 和 `-5` 都是被认可的，但在要求写出两个选项时，记得把这两种形式都写上。
- o `wc` 的字节数:
 - 再次强调，字节数/文字数 对应的是 `-c`。
- o `cut` 字段序号:
 - 多个字段之间用逗号 `,` 分隔 (如 `1,7`)，中间不要加空格。

grepコマンドで、検索パターンにマッチした行を表示する際、行頭に行番号を付けて表示させたい。適切なオプションは？

正規表現で、直前の文字の0回もしくは1回の繰り返しを意味する記号は？

？

trコマンドを使用して、「file」ファイルにある「PING」を「ping」に変換し既存の「hoge」ファイルに上書きしたい。適切なコマンドは？(パイプを使用せずにリダイレクト演算子のみを使用すること)

```
tr PING ping < file > hoge
```

sedコマンドを使用して、「text」ファイルの各行の最初に現れる「ping」という文字列のみを「hoge」に置換したい。適切なコマンドは？(省略できるオプションは省略すること)

```
sed s/ping/hoge/ text
```

grepコマンドで検索する際、大文字と小文字の区別をせずに検索したい。適切なオプションは？

```
-i
```

grepコマンドを使用して「text」ファイルの行頭が「#」の行をカウントしたい。適切なコマンドは？(全体の文字数が少ないほうから先に2つ)

```
grep -c ^# text / grep ^# text | wc -l
```

grepコマンドを使用し、「text」ファイルのうち大文字アルファベットを含まない行を表示したい。適切なコマンドは？(正規表現は5文字以内にし、「|」で囲うこと)

```
grep -v '[A-Z]' text
```

trコマンドを使用して、「file」ファイルにある「PING」を「ping」に変換し既存の「hoge」ファイルに追記したい。適切なコマンドは？(パイプを使用せずにリダイレクト演算子のみを使用すること)

```
tr PING ping < file >> hoge
```

grepコマンドを使用して「text」ファイルから「ping」または「hoge」という文字列を含む行を抽出したい。適切なコマンドは？(正規表現は「|」で囲い、「ping」から指定すること)

```
grep -E 'ping|hoge' text
```

sedコマンドを使用して「text」ファイルの内容を編集して表示する際、編集コマンドを記述したスクリプトファイル「edit」を使用したい。適切なコマンドは？

```
sed -f edit text
```

💡 知识点总结

- **grep 搜索技巧：** `-n`（行号）、`-i`（忽略大小写）、`-v`（反向选择）、`-c`（计数）是四大高频选项。
- **正则表达式 (Regex)：** 掌握锚点（`^`、`$`）、量词（`?`、`*`）以及逻辑或（`|`）。
- **sed 流处理：** 默认只处理每行第一个匹配项，使用 `-f` 可以调用外部脚本。

- **重定向差异：** `>` 表示覆盖（上書き），`>>` 表示追加（追記）。

选项解析与详细说明

6. grep 的进阶搜索

- `-n (number)`：在输出结果前加上原始文件的行号。
- `-i (ignore)`：忽略大小写，不再区分 `Ping` 和 `ping`。
- `-v (reverse)`：显示**不匹配**的行。例如 `grep -v '[A-Z]'` 就是显示没有大写字母的行。
- `-E (Extended Regex)`：启用扩展正则。只有开启它，才能直接使用 `|`（或）这种特殊符号。

2. sed 的置换逻辑

- `sed s/old/new/ file`：基本语法。`s` 代表 substitute（置换）。
- 注意：如果不加 `g` 选项，`sed` 默认只置换每一行中**第一次出现**的匹配项。
- `-f (file)`：当置换逻辑太复杂时，可以写在文件里，用 `-f` 来读取执行。

3. 正则表达式的量词

- `?`：匹配 0 次或 1 次（即：该字符可有可无）。
- `^ (Hat)`：表示匹配**行头**。如 `^#` 匹配所有以井号开头的行（通常是注释）。

grep 统计与逻辑对比表

需求 (日本語)	实现方式 A	实现方式 B (管道组合)	关键点
匹配行数统计	<code>grep -c</code>	<code>grep ... wc -l</code>	<code>-c</code> 更简洁
逻辑“或”搜索	<code>grep -E 'A B'</code>	<code>egrep 'A B'</code>	扩展正则必不可少
反向匹配	<code>grep -v</code>	-	排除不需要的内容

小学生级记忆法

1. `grep -n (Number)`:
 - `n` 就是 **Number**（数字/行号）。
2. `grep -i (Ignore)`:
 - `i` 就是 **Ignore**（忽略）。不要管它是大写还是小写，统统带走。

3. `sed -f` (File):

- 把命令写在 File 里的。

4. `>` 与 `>>` (箭头开口):

- 一个 `>` 是覆盖，像把旧水倒掉换新的。
- 两个 `>>` 是追加，像在水杯里继续倒水。

考试小秘籍

a. `tr` 命令的局限性:

- `tr` 只能替换单个字符或字符集合 (PING -> ping)，它**不能**替换字符串 (如把 apple 换成 orange)。如果题目涉及字符串替换，请务必选 `sed`。

b. 正则表达式的引号:

- 在 Linux 命令行中使用正则时，强烈建议用 **单引号** `' '` 括起来，防止 Shell 错误解析 `*` 或 `|` 等符号。

c. 扩展正则 `grep -E`:

- 如果你发现你的 `|` 或 `+` 号不起作用，记得检查是不是忘了加 `-E`。

d. 行头统计:

- 看到“行头”，脑子里第一反应必须是 `^`。

正規表現で、行末を意味する記号は？

\$

コマンドの標準出力先をディスプレイからファイルに切り替えることが出来るリダイレクト演算子
は？ (短い方から先に2つ)

> / >>

正規表現で、[]内の先頭において後続の文字以外を意味する記号は？

^

標準入力から渡された文字列を、指定のコマンドの引数として実行するコマンドは？

xargs

egrepコマンドを使用して「text」ファイルから「ping」または「hoge」という文字列を含まない
行を抽出したい。適切なコマンドは？ (正規表現は「|」で囲い、「ping」から指定すること)

egrep -v 'ping|hoge' text

egrepコマンドを使用して「text」ファイルから「ping」または「hoge」という文字列を含む行を
抽出したい。適切なコマンドは？ (正規表現は「|」で囲い、「ping」から指定すること)

egrep 'ping|hoge' text

正規表現で、左右いずれかの文字列を意味する記号は？

|

コマンド1の標準出力と標準エラー出力をコマンド2の標準入力に送りたい。空欄にあてはまるリダイレクト演算子は？

\$ コマンド1 [|] コマンド2

2>&1

teeコマンドを使用する際、標準入力の内容を標準出力に出力すると共にファイルに追記したい。適切なオプションは？

-a

💡 知识点总结

- **正则表达式锚点：** `$` 锁定行尾，`^` 在 `[]` 之外锁定行头，在 `[]` 之内表示“非”。
- **重定向高级技巧：** `2>&1` 是将错误信息（Stderr）“嫁接”到标准输出（Stdout）的经典写法。
- **数据流分叉与传递：** `tee` 实现了一路变两路（显示并保存），`xargs` 实现了从“字符串”到“命令参数”的转换。

🔍 选项解析与详细说明

4. 正则表达式的“方位”与“否定”

- `$`：锚定行末。例如 `ping$` 匹配以 ping 结尾的行。
- `^` (Hat):
 - 在外部时（如 `^#`）：匹配行首。
 - 在内部时（如 `[^0-9]`）：表示「後続の文字以外（非...）」，即匹配非数字。

2. 标准输出与错误的合并

- `2>&1`：
 - `1` 代表标准输出（Stdout），`2` 代表标准错误（Stderr）。
 - `2>&1` 的意思是：把本该发给“2”的内容全部丢给“1”。常用于 `&>` 的替代写法，将所有日志保存到同一个文件。

3. `xargs` 与 `tee` 的妙用

- `xargs` : 它是管道符 `|` 的强力搭档。有些命令（如 `rm`）不接受管道传输的字符串作为输入，这时必须用 `xargs` 把字符串转换成「引数（参数）」。
- `tee -a` : `tee` 就像一个 T 型水管。使用 `-a` (`append`) 选项可以实现追记，而不是覆盖。

重定向符号与功能对照表

符号	功能说明 (日本語)	记忆点
<code>></code>	標準出力を上書き	覆盖
<code>>></code>	標準出力を追記	追加
<code>2>&1</code>	エラー出力を標準出力に結合	合并流
<code>**`</code>	<code>`**`</code>	前の出力を次の標準入力へ
<code>xargs</code>	入力をコマンドの引数にする	转换器

小学生级记忆法

1. `$` (美元在最后):
 - 想象你买完东西最后才付钱（\$），所以它代表 **行末**。
2. `[^ ]` (反骨仔):
 - `^` 缩在框框 `[ ]` 里面时，像是在顶嘴说“不”，所以它代表 **排除**。
3. `|` (或是这，或是那):
 - 像一根栅栏把两个东西隔开，你要么选左边，要么选右边。
4. `tee` (T型管):
 - 名字长得就像一个 T 字接头，水流进来后分成两路走。

考试小秘籍

- a. `grep -E` vs `egrep` :
 - 它们是等价的。考试时如果问到包含 `|` 的正则表达式，记得两者都可以使用，但题目如果指定 `egrep`，答案就要严格遵循。
- b. `xargs` 的必要性:
 - 如果题目问：“如何删除 `find` 查找到的所有文件？”，答案通常是 `find ... | xargs rm`。直接用管道给 `rm` 是行不通的。
- c. `2>&1` 的顺序:

- 记住必须先写重定向 `>`，再写合并目标。顺序不能乱。

d. `tee` 的追记:

- 很多人会漏掉 `-a`。如果题目提到「追記したい」，这是唯一的正确选项。

grepコマンドで検索する際、条件にマッチしなかった行を表示したい。適切なオプションは？

-v

dmesgコマンドの標準出力を、ディスプレイに表示すると共に「log」ファイルに保存したい。適切なコマンドは？

dmesg | tee log

grepコマンドを使用して「text」ファイルから、「*」という文字列を含む行を抽出したい。適切なコマンドは？ (オプションを使用するほうから先に2つ、検索パターンは「|」で囲うこと)

grep -F '*' text / grep '\.*' text

標準入力元をキーボードからファイルに切り替えることが出来るリダイレクト演算子は？

<

sedコマンドを使用して、「text」ファイル内の全ての「a」を「A」に、また「b」を「B」に置換したい。適切なコマンドは？ (オプションは使用せず、編集コマンドは上記の順に指定)

sed y/ab/AB/ text

「rm ./」コマンドを実行したときに出力されるエラーメッセージのみを「rm.log」ファイルに出力したい。適切なコマンドは？ (ファイルが存在する場合は上書き)

rm ./ 2> rm.log

sedコマンドを使用して、「text」ファイルの行頭が「#」の行を削除して表示したい。適切なコマンドは？ (省略できるオプションは省略すること)

sed /^#/d text

grepコマンドで検索する際、検索パターンを正規表現ではなく固定文字列として検索したい。適切なオプションは？

-F

sedコマンドを使用して、「text」ファイル内の全ての「ping」という文字列を「hoge」に置換し、また行頭が「#」の行を削除して表示したい。適切なコマンドは？ (編集コマンドは上記の順に指定すること)

sed -e s/ping/hoge/g -e /^#/d text

grepコマンドを使用して「text」ファイルから、行末が「ping」の行を抽出したい。適切なコマンドは？ (正規表現は「|」で囲うこと)

grep 'ping\$' text

💡 知识点总结

- **grep 的字符串处理**：`-v` 用于排除，`-F` 用于取消正则特性，将字符视为普通文本。
- **sed 的多重处理**：使用多个 `-e` 选项可以按顺序执行多条指令；`y` 指令用于单字符的一对一转换。
- **重定向的文件描述符**：标准错误输出（Stderr）的编号是 `2`，使用 `2>` 专门捕获报错信息。
- **正则锚点（Anchor）**：`$` 用于锁定行尾，是搜索特定结尾字符串的关键。

🔍 选项解析与详细说明

3. 搜索“元字符”本身

如果你想搜索 `.*`（这在正则中代表任意长度字符），直接搜会被当成正则。

- **方法 A (`-F`)**: Fixed string。告诉 grep 把所有字符都当成普通的“死字”。
- **方法 B (`\`)**: 使用反斜杠对 `.` 和 `*` 进行转义（Escape），让它们失去魔力。

2. sed 的指令集

- **y 指令**: 转换（Transliterate）。`y/ab/AB/` 会把所有的 `a` 换成 `A`，所有的 `b` 换成 `B`。它比 `s` 指令在处理单字符批量转换时更高效。
- **-e 选项**: expression。当你需要在一个 sed 命令里既做替换（`s`）又做删除（`d`）时，每个指令前都要加 `-e`。

3. I/O 重定向：捕获错误

- `2> log` :
 - `1>` 或 `>` 默认只管“好消息”（Stdout）。
 - `2>` 专门盯着“坏消息”（Stderr）。这在脚本调试中非常重要，可以确保屏幕干净，同时记录报错。

📊 sed 常用编辑指令对照表

指令符号	功能名称	示例	作用说明
s	Substitute	s/old/new/g	字符串替换（g代表全部）
d	Delete	/^#/d	删除匹配的行（如注释）
y	Transliterate	y/abc/ABC/	字符一对一转换
-e	Expression	-e ... -e ...	连接多个编辑动作

🧠 小学生级记忆法

1. `grep -v` (Vanish):

- 把 `-v` 想成 Vanish (消失)。匹配到的行统统“消失”，只显示没匹配到的。

2. `sed y/` (Yes/同步):

- `y` 就像照镜子，左边的 `a` 对应右边的 `A`，整整齐齐，同步变换。

3. `2>` (二号位报错):

- 1号是冠军 (正确输出)，2号是亚军 (错误输出)。所以 `2>` 用来接错误。

4. `grep -F` (Frozen/冰冻):

- 把正则元字符“冰冻”起来，让它们动弹不得，变成普通字符。

📝 考试小秘籍

a. `s` 指令的 `g` 标记:

- 题目如果要求「全ての (所有的)」字符串替换，别忘了在 `s/ping/hoge/` 后面加一个 `g` (global)。

b. `sed` 的删除语法:

- 语法是 `/模式/d`。行首是 `^`，井号是 `#`，所以连起来是 `/^#/d`。

c. `tee` 的作用:

- 题目如果说「ディスプレイに表示すると共に (显示的同时保存)」，不要犹豫，直接写 `tee`。

d. 正则匹配行末:

- 看到「行末が...」，关键词就是 `$`。如果是搜索 `ping` 结尾，一定要写成 `'ping$'`。

dateコマンドを実行した結果を「log.txt」ファイルに上書き保存したい。適切なコマンドは？ (リダイレクト演算子は1文字で)

date > log.txt

正規表現で、[]内において範囲指定を意味する記号は？

-

sedコマンドを使用して、「text」ファイルの1行目から3行目を削除して表示したい。適切なコマンドは？ (省略できるオプションは省略すること)

sed 1,3d text

正規表現で、直前の文字の0回以上の繰り返しを意味する記号は？

*

grepコマンドで検索する際、拡張正規表現を使用したい。適切なオプションは？

-E

catコマンドとリダイレクト演算子2つを使用して「file」ファイルの内容を、既存の「hoge」ファイルに追記したい。適切なコマンドは？

cat < file >> hoge

sedコマンドを使用して、「text」ファイルに含まれる全ての「ping」という文字列を「hoge」に置換したい。適切なコマンドは？(省略できるオプションは省略すること)

sed s/ping/hoge/g text

grepコマンドで検索する際、検索パターンをファイルから読み込みたい。適切なオプションは？

-f

以下のコマンドの出力結果を答えよ。

```
$ echo 02 | sed 's/2/1&3/'
```

0123

正規表現で、次の1文字をエスケープする記号は？

\

💡 知识点总结

- sed 的范围指定：使用逗号 `,`，连接起始行号和结束行号，配合 `d` 指令可实现批量删除。
- 正则表达式的重复（Quantifiers）：`*` 代表 0 次或多次，与通配符中的含义不同。
- grep 的进阶输入：使用 `-f` 选项可以从文件中批量导入关键词，而不需要在命令行中手动输入。
- 字符串切分 (Echo & Cut)：通过管道将字符串传给 `cut`，利用 `-c` 可以实现精准的字符截取。

🔍 选项解析与详细说明

4. sed 的行处理

- `sed 1,3d text`：
 - `1,3` 是地址范围（Address Range）。
 - `d` 是删除指令（Delete）。
 - 意思是：只在显示时“抹掉”1到3行，原文件内容不会改变。

2. 强大的扩展正则 (ERE)

- `grep -E`:
 - 允许使用更丰富的符号，如 `+` (1次以上)、`?` (0次或1次)、`|` (或者)。如果不加 `-E`，这些符号会被视为普通字符。

3. 正则表达式的“连字符”

- `[a-z]`: 在方括号内，`-` 表示一个「範圍 (范围)」。
- `\*`: 这是一个常见的考点。
 - 在 正規表現 中: 代表直前字符的 0 次以上重复。
 - 在 ワイルドカード (Wildcard) 中: 代表任意长度字符串。

sed 与 grep 的指令/选项对比表

功能	命令与选项	作用说明 (日本語)
行指定削除	<code>sed 1,3d</code>	1行目から3行目までを削除
文字列一括置換	<code>sed s/old/new/g</code>	全ての該当文字列を置換
拡張正規表現	<code>grep -E</code>	メタキャラクタを有効化
パターンファイル	<code>grep -f</code>	ファイルから検索条件を読み込

小学生级记忆法

- `sed 1,3d` (点名):
 - 像老师点名一样: “1号到3号，今天不用来 (delete)。”
- `grep -f` (File):
 - `f` 代表 file。说明搜索的目标 (Pattern) 藏在文件里。
- `*` (星号 = 影分身):
 - 它是前一个字符的影分身，可以变成 0 个 (消失)，也可以变成无数个。
- `cut -c` (Char):
 - `c` 就是 character。专门剪掉字符的剪刀。

考试小秘籍

- `sed` 的 `g` 标记:

- 题目问「全ての（所有的）」替换，结尾必须带 `g`。如果只是替换每一行第一个，则不带 `g`。

2. `cat` 的输入输出重定向:

- 题目要求使用「リダイレクト演算子2つ」，所以必须写成标准的 `cat < 输入文件 >> 输出文件` 形式。

3. `cut` 的索引:

- 注意 `cut` 的计数是从 1 开始的。 `1-4` 代表截取前 4 个字符。

4. `\` 的转义作用:

- 任何时候想让一个有特殊权力的符号（如 `*` 或 `$`）“闭嘴”变回普通字符，就在它前面加一个 `\`。

コマンドの標準出力を標準エラー出力にリダイレクトするリダイレクト演算子は? (省略しないこと)

1>&2

catコマンドとリダイレクト演算子を1つ使用して「file」ファイルの内容を、既存の「hoge」ファイルに追記したい。適切なコマンドは?

cat file >> hoge

正規表現で、行頭を意味する記号は?

^

正規表現で、直前の文字の1回以上の繰り返しを意味する記号は?

+

grepコマンドを使用して「text」ファイルから「hoge」という文字列を含む行を抽出したい。適切なコマンドは?

grep hoge text

「file」ファイルの行数を出力したい空欄にあてはまるコマンドは?

\$ cat file | wc -l

wc

終了文字が現れるまで、コマンドの標準入力に入力を続けるリダイレクト演算子は? (記号と、その呼び名)

<< / ヒアドキュメント

catコマンドとgrepコマンドを使用して「text」ファイルから「hoge」という文字列を含む行を抽出したい。適切なコマンドは?

cat text | grep hoge

fgrepコマンドを使用して「text」ファイルから、「.*」という文字列を含む行を抽出したい。適切なコマンドは？(文字列は「」で囲うこと)

```
fgrep '.*' text
```

コマンドの標準エラー出力先をディスプレイからファイルに切り替えることが出来るリダイレクト演算子は？(短い方から先に2つ)

```
2> / 2>>
```

💡 知识点总结

- **ファイル記述子 (File Descriptors):** 标准输出 (Stdout) 是 **1**，标准错误 (Stderr) 是 **2**。重定向时 `1>&2` 可以将正常信息“推”给错误流。
- **ヒアドキュメント (<<):** 一种特殊的重定向方式，允许用户在命令行或脚本中直接输入多行文本，直到遇到指定的结束符（如 `EOF`）。
- **fgrep (Fixed grep):** 等同于 `grep -F`。它完全不理睬正则表达式，把所有字符都当成普通的固定字符串。

🔍 选项解析与详细说明

4. 数据的“并流”与“分流”

- `1>&2` :
 - 将本该显示在屏幕上的标准输出（1）重定向到标准错误（2）所在的地方。
 - 这常用于脚本中，当你希望某些提示信息被当作“错误或警告”记录到错误日志时。

2. ヒアドキュメント (Here Document)

- `<<` :
 - 语法：`command << 終了文字`。
 - 比如在脚本中自动输入多行配置，而不需要手动一行行交互。

3. 文字处理的“暴力搜索”

- `fgrep '.*'` :
 - 如果用普通 `grep`，它会找“任意字符”。
 - 用 `fgrep`，它会老老实实地去找文件里真的写着「.」和「*」的地方。

📊 重定向与追加操作对照表

操作需求 (日本語)	符号	示例	说明
標準出力の追記	>>	cat f1 >> f2	在原文件末尾添加
標準エラーの出力	2>	ls nofile 2> err.log	只接住报错信息
標準エラーの追記	2>>	ls nofile 2>> err.log	将报错信息追加到日志
入力の終了指定	<<	cat << EOF	ヒアドキュメント启动

小学生级记忆法

- +** (加号 = 至少有一个):
 - 就像数学里的加法，你必须有东西才能加。所以它代表 **1 次以上** 的重复。
- fgrep** (Fast/Fixed):
 - 把它想成 **Fixed** (固定的)。字符全被“固定”住了，不再有正则变身的能力。
- <<** (向左开口的漏斗):
 - 就像一个大漏斗，你可以一直往里倒水 (输入文字)，直到看到“塞子 (结束字符)”为止。
- ^** (Hat/帽子):
 - 帽子戴在头顶，所以它代表 **行头**。

考试小秘籍

- grep** 的三种形态:
 - grep**: 基础款，支持基本正则。
 - egrep**: 增强款 (**-E**)，支持 **+**，**?**，**|**。
 - fgrep**: 简单款 (**-F**)，不支持正则，速度最快。
- 行数统计的管道:
 - `cat file | wc -l` 这种写法虽然稍微多了一步 **cat**，但在考试中是非常经典的管道组合题。
- <<** 的中文名称:
 - 考试有时会考术语，记得它叫「**ヒアドキュメント**」 (Here Document)。
- 2>** 与 **2>>**:
 - 看到「**ディスプレイからファイルに切り替える** (从显示器切换到文件)」，根据是覆盖还是追记，分别对应 **2>** 和 **2>>**。

「file」ファイルに記載されているファイル名を元に、複数の新規ファイルを一度に作成したい。適切なコマンドは？（使用するコマンドはxargs/cat/touchの3つ。リダイレクト演算子は使用しない）

```
cat file | xargs touch
```

正規表現で、任意の1文字を意味する記号は？

.

sedコマンドを使用して、「text」ファイルの各行の最初に現れる文字列「ping」のみを大文字と小文字を区別せずに「hoge」に置換したい。適切なコマンドは？（省略できるオプションは省略すること）

```
sed s/ping/hoge/i text
```

標準入力の内容をディスプレイとファイルに出力するコマンドは？

```
tee
```

正規表現で、囲んだ部分のいずれかの一文字を意味する記号は？（2文字で）

```
[]
```

「sh test.sh」コマンドを実行した時に出力される標準出力と標準エラー出力の両方を既存の「log」ファイルに追記したい。適切なコマンドは？（標準エラー出力先を変更すること）

```
sh test.sh >> log 2>&1
```

dmesgコマンドの標準出力を、lessコマンドの標準入力に渡したい。適切なコマンドは？

```
dmesg | less
```

コマンドの標準エラー出力を標準出力にリダイレクトするリダイレクト演算子は？

```
2>&1
```

grepコマンドを使用して「text」ファイルから、行頭が「#」ではない行を抽出したい。適切なコマンドは？（オプションを指定した方法とすること。また、正規表現は「|」で囲うこと。）

```
grep -v '^#' text
```

grepコマンドで、検索パターンにマッチした行の行数のみを表示させたい。適切なオプションは？

```
-C
```

trコマンドを使用して、「file1」ファイル内の連続した「#」を1つに置き換えて「file2」ファイルに出力したい。適切なコマンドは？（パイプを使用せずにリダイレクト演算子のみを使用すること）

```
tr -s '#' < file1 > file2
```

知识点总结

掌握 Linux 命令行的高级组合技：通过 `xargs` 实现参数传递、利用 `sed` 的忽略大小写标志、以及对标准输出与错误输出的精准合并。

选项解析与详细说明

1. `xargs` 的神奇转换

- `cat file | xargs touch`:
 - 如果 `file` 里面写着 `a.txt`, `b.txt`, `c.txt`。
 - 直接 `cat file | touch` 是行不通的, 因为 `touch` 不接受管道传来的字符串。
 - `xargs` 的作用就是把 `cat` 出来的字符串, 变成 `touch` 后面跟着的「引数 (参数)」。

3.1 `sed` 的忽略大小写置换

- `sed s/ping/hoge/i`:
 - 这里的 `i` 代表 **ignore-case**。
 - 它能让 `sed` 同时匹配到 `ping`, `Ping`, `PING` 并统一置换为 `hoge`。

3.3 完美的日志捕获

- `>> log 2>&1`:
 - 这是运维中最常用的指令。
 - `>> log`: 把正常输出 (1) 追加到 `log`。
 - `2>&1`: 把错误输出 (2) 也扔到标准输出 (1) 去。
 - 结果: 无论是正常的还是报错的, 全部乖乖存进 `log` 文件。

正则表达式元字符对照表

符号	含义 (日本語)	记忆点
.	任意の1文字	像一个占位符, 代表任何字
[]	囲んだ内のいずれか	像一个篮子, 选篮子里的一个
^#	行頭が #	井号在最前面 (通常是注释)
[^#]	# 以外の文字	在方括号里表示 “排除”

小学生级记忆法

1. `xargs` (快递员):

- 它就像一个快递员，把前一个命令产生的一堆名字（字符串），搬到后一个命令的办公桌上作为参数。

2. `sed ... /i` (I don't care):

- 这个 `i` 就是在说：“我不介意 (Ignore) 它是大写还是小写。”

3. `grep -c` (Count):

- `c` 就是 `count` (计数)。它不给你看内容，只给你看一个数字。

4. `tr -s` (Squeeze/挤压):

- 看到「連続した文字を1つに」就要想到挤压。把一排重复的符号挤成一个。

考试小秘籍

1. `2>&1` 的写法:

- 考试时不能缩写。虽然现代 Bash 支持 `&>>`，但 LPI 考试标准写法依然是 `>> log 2>&1`。

2. `grep -v` 的妙用:

- 题目问「...ではない行 (不是...的行)」，第一反应必须是反向选择 `-v`。

3. `tr` 的重定向方向:

- `tr` 永远是 `<` 输入，`>` 输出。它是从左往右流动的。

4. `sed` 的置换范围:

- 除非题目说“置换所有”，否则默认不加 `g`；如果说“忽略大小写”，别忘了加 `i`。

viエディタで編集しているファイルを、変更内容を強制保存しエディタを終了したい。適切なviコマンドは？

:wq!

viエディタで、カーソル位置からファイルの先頭に向かって文字列「hoge」を検索したい。適切なviコマンドは？

?hoge

viエディタのコマンドモードにおいて、カーソルのある行を含めて5行をバッファにコピーしたい。適切なviコマンドは？

5yy

viエディタでコマンドモードから入力モードに移行する際、カーソルの行の先頭から文字を入力したい。適切なviコマンドは？

|

viエディタのコマンドモードで、1画面分、下方を表示することが出来るviコマンドは？

Ctrl+f

viエディタでコマンドモードから入力モードに移行する際、カーソルのある行の末尾から文字を入力したい。適切なviコマンドは？

A

viエディタのコマンドモードにおいて、カーソル位置の単語を削除するviコマンドは？

dw

viエディタでコマンドモードから入力モードに移行する際、カーソルのある行の上に空白行を挿入し、その行で文字を入力したい。適切なviコマンドは？

O

viエディタのコマンドモードにおいて、直前の編集を再実行するviコマンドは？

.

viエディタで編集しているファイルをエディタを終了することなく保存したい。適切なviコマンドは？

:w

viエディタのコマンドモードで、カーソルを1文字下へ移動することが出来るviコマンドは？

j

viエディタで、「/hoge」として文字列「hoge」をカーソル位置からファイルの末尾に向かって検索している。次の候補を検索するviコマンドは？

n

viエディタで、カーソル位置からファイルの末尾に向かって文字列「hoge」を検索したい。適切なviコマンドは？

/hoge

知识点总结

掌握 Linux 的灵魂编辑器 —— `vi` (vim) 的三种模式转换、光标移动、编辑指令以及末行命令 (Ex command) 。

选项解析与详细说明

3.4 模式转换的细节（进入输入模式）

进入输入模式 (Insert Mode) 不仅仅是按 `i`，位置很重要：

- `i` (insert): カーソル位置から（当前光标前）。

- **I (Shift + i): 行の先頭から**（行首）。
- **a (append): カーソル位置の直後から**（当前光标后）。
- **A (Shift + a): 行の末尾从**（行尾）。
- **o**: カーソルの下に新しい行（在下方插入新行）。
- **O**: **カーソルの上**に新しい行（在上方插入新行）。

3.2 搜索与重复操作

- **/hoge**: **下方向**（文件末尾方向）搜索。
- **?hoge**: **上方向**（文件开头方向）搜索。
- **n (next): 按照搜索的同方向查找下一个**。
- **.** (**dot**): 这是一个极强大的命令，用于**重复执行**上一次的编辑动作（如删除、粘贴等）。

3.3 移动与翻页

- **h, j, k, l**: 左、下、上、右。
- **Ctrl + f (forward): 向前翻一页**（即：画面**下方**移动）。
- **Ctrl + b (backward): 向后翻一页**（即：画面**上方**移动）。

 常用编辑命令对照表

目的 (日本語)	コマンド	记忆技巧
复制行	yy	Yank（提取）
粘贴	p	Put（放置）
删除/剪切行	dd	Delete Delete
删除单词	dw	Delete Word
撤销	u	Undo

 小学生级记忆法

1. **j (下移):**
 - **j** 的形状像一个小钩子，勾向**下方**。
2. **yy (复制):**
 - 把 **y** 想象成一个“丫”形的叉子，把这一行文字给**叉（复制）**起来。
3. **A 与 I (端点输入):**

- I 是单词 Inside 或行首那一竖。
- A 是单词 After，移动到后面再开始。

4. dw (干掉单词):

- Delete Word。精准打击，只删掉光标下的那个词。

考试小秘籍

1. 强制保存并退出:

- 题目如果提到「強制保存しエディタを終了」，别忘了末尾的感叹号，即 :wq!。

2. 搜索方向:

- 记住“撇号向右下方划”对应 / (下搜)，“问号向上”对应 ? (上搜)。

3. 组合键技巧:

- 在命令模式下，数字 + 命令 = 重复。比如 5yy 是复制 5 行，10dd 是删除 10 行。

4. 画面移动:

- Ctrl + f 的 f 是 Forward (前进/下翻)。
- Ctrl + b 的 b 是 Backward (后退/上翻)。

viエディタを終了せずに、現在編集しているファイルを閉じて、別の「newfile」ファイルを開きたい。ただし、編集集中のファイルが変更されている場合は、最後に保存した状態に戻してから別ファイルを開くようにする。適切なviコマンドは？

:e! newfile

viエディタのコマンドモードにおいて、カーソルの左にバッファの内容を貼り付けるviコマンドは？

P

viエディタのコマンドモードにおいて、カーソル位置の行を削除し、バッファにコピーしたい。適切なviコマンドは？

dd

viエディタを終了せずに、現在編集しているファイルを閉じて、別の「newfile」ファイルを開きたい。ただし、編集集中のファイルが変更されている場合は、警告が出て別ファイルを開けないようにする。適切なviコマンドは？

:e newfile

viエディタのコマンドモードで、カーソルをファイルの5行目へ移動することが出来るviコマンドは？ (記号を使う方から先に2つ)

:5 / 5G

viエディタのコマンドモードで、1画面分、上方を表示することが出来るviコマンドは？

Ctrl+b

viエディタのコマンドモードで、カーソルを1文字右へ移動することが出来るviコマンドは？

h

viエディタのコマンドモードにおいて、カーソルのある行を含めて3行をコピーし、カーソルの上に挿入したい。適切なviコマンドは？（3文字で）

3YP

viエディタでファイルを編集する際、行番号を付けてファイルを表示させたい。適切なコマンドは？(短い方から先に2つ)

:set nu / :set number

viエディタのコマンドモードで、カーソルを画面の最上行へ移動することが出来るviコマンドは？

H

開発環境として便利な機能を備え、ユーザ独自の拡張機能をカスタマイズできる高機能なテキストエディタは？

Emacs

viエディタでコマンドモードから入力モードに移行する際、カーソルの左から文字を入力したい。適切なviコマンドは？

i

viエディタのコマンドモードにおいて、カーソルのある行をコピーし、下の行に挿入したい。適切なviコマンドは？（3文字で）

yyp

デフォルトで起動するエディタを設定する環境変数は？

EDITOR

viエディタのコマンドモードで、カーソルを行頭へ移動することが出来るviコマンドは？（数字で回答）

0

viエディタのコマンドモードで、カーソルをファイルの先頭へ移動することが出来るviコマンドは？

gg

viエディタのコマンドモードにおいて、カーソルのある行を含めて5行を切り取り、バッファにコピーしたい。適切なviコマンドは？

5dd

viエディタで編集しているファイルにある「ping」という文字列を、全て「hoge」に置換したい。適切なコマンドは？

:%s/ping/hoge/g

💡 知识点总结

掌握 `vi` 的高级文件切换指令 (Edit)、寄存器操作 (Yank/Paste)、全局替换 (Substitute) 以及快速光标跳转 (Move)。

🔍 选项解析与详细说明

3.4 文件的“无缝”切换与保护

在不退出 `vi` 的情况下打开新文件：

- `:e newfile` : edit。如果当前文件没保存，`vi` 会报错警告你，保护你的劳动成果。
- `:e! newfile` : 强制切换。不管当前改了什么，直接**丢弃修改**，回到最后一次保存的状态并打开新文件。

3.2 粘贴位置的精细控制

- `p` (小写): 贴在光标**之后** (若是行，则贴在当前行**之下**)。
- `P` (大写): 贴在光标**之前** (若是行，则贴在当前行**之上**)。
- 例子: `3YP` 的逻辑是: 3行复制 (3Y) + 贴在上面 (P)。

3.3 全局替换的语法

- `:%s/old/new/g` :
 - `%` : 代表整篇文档 (所有行)。
 - `s` : substitute (替换)。
 - `g` : global (行内所有的匹配项都换掉)。

3.4 光标的“瞬间移动”

- `gg` : 回到文件第一行。
 - `G` : 去到文件最后一行。
 - `:n` 或 `nG` : 跳到第 `$n$` 行。
 - `H` : High。跳到当前屏幕显示的最上方那一行。
 - `0` : 数字零，跳到当前行的行首。
-

📊 `vi` 常用跳转与设置对照表

目的 (日本語)	コマンド	記憶点
行番号を表示	:set nu	number
指定行へ移動	:5 / 5G	第 5 行
画面最上行へ	H	High
ファイル先頭へ	gg	go go (start)
ファイル末尾へ	G	Grave (end/大写G更有力)

🧠 小学生级记忆法

1. `yyp` (克隆行):

- `yy` (复制) + `p` (贴在下面)。这是 `vi` 里最常用的“复制并粘贴当前行”的连招。

2. `H` / `M` / `L` (屏幕坐标):

- `H` (高/上), `M` (中), `L` (低/下)。这三个键让你在屏幕内快速跳跃。

3. `:e!` (不管了!):

- 那个感叹号 `!` 就像你在对 `vi` 吼: “别废话, 强制执行 (丢弃修改)!”

4. `EDITOR` 变量:

- 这是一个全局变量。告诉系统: 谁是我的 `EDITOR` (主编/编辑器)?

📝 考试小秘籍

1. 复制粘贴的连招:

- 如果题目问“复制并粘贴到下方”, 答案是 `yyp`。
- 如果是“复制并粘贴到上方”, 答案是 `yyP`。

2. 强制操作的标志:

- 在 `vi` 的末行模式下, 只要看到“强制”、“丢弃修改”, 答案里一定带感叹号 `!`。

3. 环境变量的写法:

- 注意 `EDITOR` 全大写, 它是 Linux 默认编辑器的“指挥官”。

4. 跳转到特定行:

- 记住有两种方式: 一种是冒号开头的末行命令 `:5`, 一种是普通模式下的 `5G`。

操作キーとそれに対応する機能が画面下部に表示されており、初心者でも使いやすい軽量のテキストエディタは?

nano

viエディタのコマンドモードにおいて、カーソル位置の文字を削除するviコマンドは？

x

viエディタのコマンドモードにおいて、直前の編集を取りやめるviコマンドは？

u

viエディタのコマンドモードにおいて、カーソルのある行をバッファにコピーしたい。適切なviコマンドは？

yy

viエディタで入力モードからコマンドモードへ変更するキーは？

Esc

viエディタのコマンドモードで、カーソルを1文字左へ移動することが出来るviコマンドは？

h

viエディタでコマンドモードから入力モードに移行する際、カーソルのある行の下に空白行を挿入し、その行で文字を入力したい。適切なviコマンドは？

o

viから派生した高機能なテキストエディタは？

Vim

viエディタを終了したい。ただし、開いているファイルの内容が変更されている場合は終了したくない。適切なviコマンドは？

:q

viエディタのコマンドモードで、カーソルをファイルの最終行へ移動することが出来るviコマンドは？

G

viエディタで、「/hoge」として文字列「hoge」をカーソル位置からファイルの末尾に向かって検索している。次の候補を逆方向に検索するviコマンドは？

N

viエディタで編集しているファイルを、変更内容を保存しエディタを終了したい。適切なviコマンドは？ (記号を使用するほうから先に、またアルファベット順に3つ)

:wq / :x / ZZ

viエディタのコマンドモードで、カーソルを画面の最下行へ移動することが出来るviコマンドは？

L

 知识点总结

掌握 `vi` (Vim) 的基本编辑操作、搜索方向切换、多种退出方式，以及轻量级编辑器 `nano` 的特点。

 选项解析与详细说明

3.5 退出 vi 的三种姿势

- `:wq` : write (保存) + quit (退出) 。最标准的做法。
- `:x` : 类似于 `:wq` ，但仅在文件有修改时才写入硬盘。
- `ZZ` : 大写连按两次。这是命令模式下的快捷键，功能等同于 `:x` (保存并退出) 。

3.2 搜索方向的逆转

- `n` (next): 按照你之前的搜索方向 (`/` 或 `?`) 查找下一个。
- `N` (Shift + n): 逆方向查找。如果你是用 `/` 向下搜，按 `N` 就会向上找。

3.3 进入输入模式的“新行”操作

- `o` (小写): 在当前行「の下 (下方)」插入新行并进入输入模式。
- `O` (大写): 在当前行「の上 (上方)」插入新行。

3.4 初学者友好的 `nano`

- `nano` : 相比于 `vi` 的“深不可测”，`nano` 在屏幕下方直接列出了常用快捷键 (如 `^O` 保存，`^X` 退出) ，非常适合临时编辑配置文件。

 光标移动与屏幕跳转对照表

目的 (日本語)	コマンド	记忆点
左へ移動	<code>h</code>	最左边的键
画面最下行へ	<code>L</code>	Low (底部)
ファイルの最終行へ	<code>G</code>	Go to end
直前の操作を取消	<code>u</code>	Undo (撤销)

 小学生级记忆法

1. `x` (打叉削除):
 - 就像在纸上写错字打个 `x` 删掉一样，它只删掉光标下的那一个字符。

2. **Esc** (逃生键):

- 输入模式太危险，按下 **Escape** (逃跑) 回到安全的命令模式。

3. **h j k l** (左右排列):

- **h** 在最左边，所以是左移；**l** 在最右边，所以是右移。

4. **o** (Open new line):

- 想象在地上钻了一个 **o** 型的洞 (新行)，然后跳进去写字。

考试小秘籍

1. 退出命令的陷阱:

- 题目问「変更されている場合は終了したくない」 $\rightarrow$ 选 **:q**。因为它不带感叹号，有修改时会报错阻止退出。

2. 搜索的“大小写 n”:

- **n** = 顺着找。
- **N** = 倒着找 (逆方向)。

3. Vim 的定义:

- 看到「viから派生した高機能 (vi派生的高性能)」编辑器，答案必然是 **Vim** (Vi Improved)。

4. 保存退出的顺序:

- 题目要求按字母顺序或特定顺序时，务必记牢 **:wq**，**:x**，**ZZ** 这三个高频考点。

viエディタでコマンドモードから入力モードに移行する際、カーソルの右から文字を入力したい。
適切なviコマンドは？

a

viエディタでファイルを編集しているが、viエディタを終了せずにファイルを最後に保存した状態に戻したい。適切なviコマンドは？

:e!

viエディタで、あるファイルを編集しているが、viエディタを終了することなくカレントディレクトリにあるファイルを確認したい。適切なコマンドは？

!:ls

viエディタのコマンドモードで、カーソルを行の先頭の文字へ移動するviコマンドは？

^

viエディタのコマンドモードでカーソルを1文字上へ移動することが出来るviコマンドは？

k

viエディタで読み取り専用でファイルを開くためのオプションは？

-R

viエディタでファイルを編集する際、行番号が表示していたが、行番号を非表示にしたい。適切なviコマンドは？ (短い方から先に2つ)

:set nonu / :set nonumber

viエディタでファイルを編集する際、タブの幅を10にしたい。適切なviコマンドは？ (短い方から先に2つ)

:set ts=10 / :set tabstop=10

viエディタのコマンドモードにおいて、カーソル位置の単語をバッファにコピーするviコマンドは？

yw

viエディタであるファイルを編集しているが、別ファイルである「text」ファイルの内容をカレント行の次の行に読み込みたい。適切なviコマンドは？

:r text

ファイルの変更内容を保存せずにviエディタを終了したい。適切なviコマンドは？

:q!

viエディタのコマンドモードにおいて、カーソルの右にバッファの内容を貼り付けるviコマンドは？

p

viエディタのコマンドモードにおいて、カーソル位置の左の文字を削除するviコマンドは？

X

viエディタのコマンドモードで、カーソルを行末へ移動することが出来るviコマンドは？

\$

知识点总结

掌握 `vi` 的插入模式细节、外部命令调用 (Shell escape)、行内快速定位以及各种环境参数 (set) 的自定义配置。



选项解析与详细说明

3.5 插入位置的微妙差别

- `a` (append): 在光标「の右 (右侧)」开始输入。

- **i** (insert): 在光标「の左（左侧）」开始输入。
- **I**: 在「行の先頭（行首）」开始输入。
- **A**: 在「行の末尾（行尾）」开始输入。

3.2 末行模式的高级操作

- **:!ls**: 在 **vi** 中执行外部 Shell 命令。**!** 符号告诉 **vi** 临时跳出编辑器去执行后面的命令。
- **:r text**: read。将 **text** 文件的内容读取并插入到当前行之后。
- **:e!**: 强制重新加载文件。当你改乱了想「保存した状態に戻す（恢复到保存状态）」时非常有用。

3.3 行内跳转的符号

- **^**: 移动到行内第一个非空白字符（行头文字）。
- **0**: 移动到绝对行首（包含空格）。
- **\$**: 移动到「行末」。

3.4 环境设置 (Set Options)

- **tabstop (ts)**: 设置缩进宽度。
- **number (nu)**: 控制行号显示。在命令前加 **no** 即可取消该设置（如 **nonu**）。

字符级编辑与移动对照表

目的 (日本語)	コマンド	记忆技巧
上へ移動	k	k 长得高，向上指
右側を削除	x	直接打叉（小写）
左側を削除	X	背后打叉（大写）
単語をコピー	yw	Yank Word
読み取り専用	-R	Read-only

小学生级记忆法

1. **a** (After):
 - **a** 代表 after（在...之后）。所以在光标后面（右侧）输入。
2. **k** (King/向上):
 - 想象国王（King）高高在上，所以 **k** 是往上走。

3. \$ (钱在最后):

- 就像买东西最后才付钱，\$ 永远代表一行的最后。

4. :q! (紧急出口):

- 那个感叹号 ! 就像是你在大喊：“别管保存了，快让我出去！”

考试小秘籍

1. p 与 P 的左右:

- 在处理字符粘贴时，小写 p 贴在光标 右侧，大写 P 贴在光标 左侧。

2. 转义执行命令:

- 只要题目提到「エディタを終了することなく...を確認したい（不退出编辑器确认...）」，关键词就是 :!。

3. 单语操作:

- 记住 w 代表 word。所以删除单词是 dw，复制单词是 yw。

4. 选项缩写:

- 考试中 nu 和 number，ts 和 tabstop 经常作为一组出现，务必记住简写形式。